源代码说明文档

如画 Lumira 摄影辅助应用 V1.0.0

本文档为「如画 Lumira 摄影辅助应用」软件著作权登记申请的核心材料，包含源代码总体说明与核心模块源代码摘录。

- 软件全称：如画 Lumira 摄影辅助应用
- 软件简称：如画 Lumira
- 版本号：V1.0.0
- 编程语言：Dart 2.19.6

第一部分　代码总体说明

一、开发语言

本项目以 Dart 作为唯一业务逻辑开发语言，基于 Flutter 框架实现跨平台编译；Kotlin 与 Swift 仅用于各平台原生工程的入口与权限桥接，不承载业务逻辑。

语言	版本	用途	代码行数（约）
Dart	2.19.6	全部业务逻辑（UI、状态管理、数据持久化、相机控制、图像处理）	34,691 行
Kotlin	1.8+	Android 原生工程（权限申请、生命周期、平台通道）	约 800 行
Swift	5.0+	iOS 原生工程（权限申请、生命周期、平台通道）	约 600 行
测试代码	Dart 2.19.6	单元测试 / Widget 测试（test 目录 41 个文	10,606 行

		件）	
合计	-	-	约 45,297 行

二、开发框架与依赖

类别	名称	版本	用途
框架	Flutter	3.7+	跨平台 UI 框架（Android / iOS / Web / 桌面）
状态管理	flutter_riverpod	^2.3.6	声明式状态管理（Provider 模式）
路由	go_router	^6.5.7	声明式路由（34 个路由 1:1 对应 uni-app pages.json）
数据库	sqflite	^2.2.6	SQLite 本地数据库（模板、场景、相册、用户进度持久化）
相机	camerawesome	^1.4.0	跨平台相机控制（预览、拍照、前后置切换）
图像处理	gpu_image	^1.0.0	GPU 加速图像滤镜
图像处理	image	^4.0.16	像素级图像处理（LUT、暗角、颗粒、锐化）
文件系统	path_provider	^2.0.15	平台文档目录访问
路径处理	path	^1.8.3	路径拼接工具
工具	flutter_lints	^2.0.1	代码静态分析规则

三、代码结构

```
lumira_app_flutter/
├── lib/                                  # 应用源代码（101 个 Dart 文件，
34,691 行）
│   ├── main.dart                         # 应用入口（ProviderScope + MyApp）
│   ├── app/
│   │   └── router.dart                   # GoRouter 路由配置（34 个路由）
│   ├── core/                             # 核心层（与业务无关的基础设施）
│   │   ├── db/
│   │   │   ├── database_provider.dart    # 数据库 Provider（lazy 初始化 + 平台
降级）
│   │   │   ├── tables.dart               # 表名/列名常量定义
│   │   │   └── dao/                      # 数据访问对象
```

```
│   │   │       ├── gallery_dao.dart    # 相册 DAO
│   │   │       ├── scenes_dao.dart     # 场景 DAO
│   │   │       └── templates_dao.dart  # 模板 DAO
│   │   ├── router/
│   │   │   ├── route_names.dart        # 路由路径常量 + URL 构造工具
│   │   │   └── route_observers.dart    # 路由观察者（拍摄页状态清理）
│   │   ├── theme/
│   │   │   ├── app_theme.dart          # AppThemeData（tokens + style → 
ThemeData）
│   │   │   ├── theme_controller.dart   # 主题状态 Provider
│   │   │   └── theme_tokens.dart       # 8 套主题色卡 + 阴影系统
│   │   └── utils/
│   │       └── number_format.dart      # 数字格式化工具
│   ├── features/                       # 业务模块层（Feature-first 架构，10 
个模块）
│   │   ├── capture/                    # 拍摄模块
│   │   ├── challenge/                  # 挑战模块（每日挑战 + 合拍挑战）
│   │   ├── gallery/                    # 相册模块
│   │   ├── home/                       # 首页模块（8 个 section）
│   │   ├── inspiration/                # 灵感模块
│   │   ├── profile/                    # 个人中心
│   │   ├── scenes/                     # 场景模块
│   │   ├── shootkit/                   # 拍摄套件编辑器
│   │   ├── splash/                     # 启动页
│   │   └── templates/                  # 模板模块
│   └── shared/widgets/                 # 共享组件层
│       ├── buttons/                    # 按钮组件
│       ├── cards/                      # 卡片组件
│       ├── common/                     # 通用组件（fade_up / 
glass_background）
│       ├── nav/                        # 导航组件（lumira_nav）
│       └── tabbar/
│           └── floating_tabbar.dart    # 悬浮胶囊 Tab 栏
├── test/                               # 测试代码（41 个文件，10,606 行）
└── 各平台原生工程（android / ios / ohos / web / windows / macos / linux）
```

四、代码规范与设计模式

4.1 Feature-first 架构

采用按业务功能划分目录的 Feature-first 架构，每个 feature 模块内部自治， 包含 `pages/`（页面）、`widgets/`（模块私有组件）、`data/`（状态与数据）三层。 模块之间通过 `core/` 层与 `shared/widgets/` 层解耦，禁止横向依赖。

4.2 Riverpod Provider 模式

全局状态、单例服务、数据库连接均通过 Riverpod Provider 暴露：

• Provider<T>：只读服务（如 databaseProvider、appThemeProvider）

• FutureProvider<T>：异步初始化（如 templatesDaoProvider）

• StateProvider<T>：可变状态（如 themeKeyProvider、isFullscreenProvider）

• ConsumerWidget / ConsumerStatefulWidget：订阅 Provider 的 Widget 基类

4.3 DAO（Data Access Object）模式

数据库访问统一通过 DAO 类封装，每个表对应一个 DAO：`TemplatesDao`（模板）、`ScenesDao`（场景）、`GalleryDao`（相册）。DAO 接收 `Database` 实例（由 `databaseProvider` 注入），上层通过 `templatesDaoProvider` 等 FutureProvider 获取 DAO 实例。

4.4 Immutable 模式

数据模型类全部字段声明为 `final`，构造函数为 `const`，确保实例创建后不可变。状态变更通过 Provider 的 `notifier.state = newValue` 替换整个对象。

4.5 平台降级策略

数据库初始化采用三级降级策略，适配多平台环境：

1. 优先使用 path_provider.getApplicationDocumentsDirectory()（标准路径）
2. 捕获 PlatformException 时降级到 sqflite.getDatabasePath()（HarmonyOS 适配）
3. 捕获其他异常时再次降级到同一默认路径

4.6 主题 Token 系统

主题系统采用 Token 化设计，将颜色、阴影等视觉属性抽象为 `ThemeTokens` 不可变对象。8 套主题（warmWhite / ink / retro / fresh / cozy / macaron / morandi / rosegold）各自定义一套 Token，4 种 UI 风格（neumorphic / flat / glass / female）在同一 Token 基础上切换阴影与模糊效果。

4.7 路由观察者模式

`LumiraRouteObserver` 实现 `NavigatorObserver`，在 `didPop` 离开拍摄页时 自动调用 `CaptureState.resetAll` 清理状态，修复了 uni-app 版本中 "退出拍摄页后 currentTemplateId 残留影响后续自由拍摄"的 bug。

4.8 单元测试覆盖

`test/` 目录包含 41 个测试文件，覆盖 DAO、路由、主题、状态管理、 Widget 渲染等核心模块，总计约 10,606 行测试代码， 采用 Flutter 原生 `test` 与 `flutter_test` 框架。

第二部分　源代码清单

按软著规范摘录实际源代码。共 60 页（前 30 页 + 后 30 页），
每页约 50 行，行号在文件内连续不中断（跨页时继续递增），
每页页眉标注「如画 Lumira 摄影辅助应用 V1.0.0」，右上角标注连续页码 1-60。

摘录顺序：
- 前 30 页（核心基础层）：应用入口 → 路由 → 数据库 → 主题 → 工具
- 后 30 页（业务页面 + 共享组件）：拍摄 → 模板 → 首页 → 相册 → 个人中心 → 挑战 → 灵感 → 场景 → 启动 → 共享组件

模块 1：应用入口 + 主题控制器（lib/main.dart + lib/core/theme/theme_controller.dart）

第 1 页

如画 Lumira 摄影辅助应用 V1.0.0　　第 1 页

```
1  // ===== lib/main.dart =====
```

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'app/router.dart';
import 'core/theme/theme_controller.dart';
/// 应用根 Widget（接入 ProviderScope + routerProvider + appThemeProvider）
///
/// 修复 Minor finding #32：Task 2.2 遗留状态——main.dart 仍是初始模板，
/// 未接入 ProviderScope / routerProvider / appThemeProvider，导致真实运行
/// app 时无法启动到任何路由（所有 Riverpod provider 读取会抛异常）。
class MyApp extends ConsumerWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final router = ref.watch(routerProvider);
    final appTheme = ref.watch(appThemeProvider);
    return MaterialApp.router(
      title: '如画 Lumira',
      debugShowCheckedModeBanner: false,
      theme: appTheme.toThemeData(),
      routerConfig: router,
    );
  }
}
void main() {
  runApp(const ProviderScope(child: MyApp()));
}
// ===== lib/core/theme/theme_controller.dart =====
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'theme_tokens.dart';
import 'app_theme.dart';
/// 当前主题 key（默认暖米白）
final themeKeyProvider = StateProvider<ThemeKey>((ref) => ThemeKey.warmWhite);
/// 当前 UI 风格（默认新拟态）
final uiStyleProvider = StateProvider<UIStyle>((ref) => UIStyle.neumorphic);
/// 主题 Token Provider（根据 themeKey 派生 ThemeTokens）
final themeTokensProvider = Provider<ThemeTokens>((ref) {
  final theme = ref.watch(themeKeyProvider);
  return ThemeTokens.of(theme);
});
/// 应用主题 Provider（tokens + style → AppThemeData）
final appThemeProvider = Provider<AppThemeData>((ref) {
  final tokens = ref.watch(themeTokensProvider);
  final style = ref.watch(uiStyleProvider);
  return AppThemeData(tokens: tokens, style: style);
});
```

模块 2：路由配置（lib/app/router.dart）

第 2 页


```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import '../core/router/route_names.dart';
import '../core/router/route_observers.dart';
import '../features/capture/pages/capture_page.dart';
import '../features/capture/pages/capture_preview_page.dart';
import
'../features/capture/pages/capture_preview_template_page.dart';
import '../features/capture/pages/capture_scene_detail_page.dart';
import '../features/capture/pages/capture_scene_guide_page.dart';
import '../features/capture/pages/capture_scene_manage_page.dart';
import '../features/challenge/pages/challenge_detail_page.dart';
import '../features/challenge/pages/challenge_page.dart';
import '../features/gallery/pages/gallery_detail_page.dart';
import '../features/gallery/pages/gallery_diary_page.dart';
import '../features/gallery/pages/gallery_monthly_digest_page.dart';
import '../features/gallery/pages/gallery_page.dart';
import '../features/home/pages/home_page.dart';
import '../features/inspiration/pages/inspiration_page.dart';
import '../features/profile/pages/profile_academy_detail_page.dart';
import '../features/profile/pages/profile_academy_page.dart';
import '../features/profile/pages/profile_about_page.dart';
import
'../features/profile/pages/profile_collection_detail_page.dart';
import '../features/profile/pages/profile_collections_page.dart';
import '../features/profile/pages/profile_growth_page.dart';
import '../features/profile/pages/profile_invite_page.dart';
import '../features/profile/pages/profile_my_templates_page.dart';
import '../features/profile/pages/profile_page.dart';
import '../features/profile/pages/profile_settings_page.dart';
import '../features/profile/pages/profile_theme_page.dart';
import '../features/scenes/pages/scenes_page.dart';
import '../features/shootkit/pages/shootkit_editor_page.dart';
import '../features/splash/pages/splash_page.dart';
import '../features/templates/pages/templates_all_page.dart';
import '../features/templates/pages/templates_detail_page.dart';
import '../features/templates/pages/templates_drafts_page.dart';
import '../features/templates/pages/templates_editor_page.dart';
import '../features/templates/pages/templates_page.dart';
import '../features/templates/pages/templates_recommend_page.dart';
import '../features/templates/pages/templates_unlock_page.dart';
/// GoRouter Provider
/// 34 个路由与 uni-app pages.json 1:1 对应
final routerProvider = Provider<GoRouter>((ref) {
  final observer = ref.watch(lumiraRouteObserverProvider);
  return GoRouter(
    initialLocation: RouteNames.splash,
    observers: [observer],
    routes: [
      // === 主流程 ===
```

```
   50        GoRoute(
```

第 3 页


```
   51          path: RouteNames.splash,
   52          name: 'splash',
   53          builder: (context, state) => const SplashPage(),
   54        ),
   55        GoRoute(
   56          path: RouteNames.home,
   57          name: 'home',
   58          builder: (context, state) => const HomePage(),
   59        ),
   60        // === 拍摄流程 ===
   61        GoRoute(
   62          path: RouteNames.capture,
   63          name: 'capture',
   64          builder: (context, state) {
   65            final templateId =
state.queryParams[RouteNames.paramTemplateId];
   66            return CapturePage(templateId: templateId);
   67          },
   68        ),
   69        GoRoute(
   70          path: RouteNames.capturePreview,
   71          name: 'capturePreview',
   72          builder: (context, state) {
   73            final photoUrl = state.queryParams['photoUrl'];
   74            return CapturePreviewPage(photoUrl: photoUrl);
   75          },
   76        ),
   77        GoRoute(
   78          path: RouteNames.capturePreviewTemplate,
   79          name: 'capturePreviewTemplate',
   80          builder: (context, state) {
   81            final templateId =
state.queryParams[RouteNames.paramTemplateId];
   82            final draftId = state.queryParams['draftId'];
   83            return CapturePreviewTemplatePage(
   84              templateId: templateId,
   85              draftId: draftId,
   86            );
   87          },
   88        ),
   89        GoRoute(
   90          path: RouteNames.captureSceneGuide,
   91          name: 'captureSceneGuide',
   92          builder: (context, state) {
   93            // 接收 scene 参数（uni-app: /pages/capture/scene-
guide?scene=xxx）
   94            final scene = state.queryParams[RouteNames.paramScene];
   95            return CaptureSceneGuidePage(scene: scene);
   96          },
```

```
 97        ),
 98        GoRoute(
 99          path: RouteNames.captureSceneManage,
100          name: 'captureSceneManage',
```

```
  101           builder: (context, state) {
  102             final tab = state.queryParams[RouteNames.paramTab];
  103             return CaptureSceneManagePage(initialTab: tab);
  104           },
  105         ),
  106         GoRoute(
  107           path: RouteNames.captureSceneDetail,
  108           name: 'captureSceneDetail',
  109           builder: (context, state) {
  110             final sceneId = state.queryParams[RouteNames.paramSceneId];
  111             return CaptureSceneDetailPage(sceneId: sceneId);
  112           },
  113         ),
  114         // === 模板 ===
  115         GoRoute(
  116           path: RouteNames.templates,
  117           name: 'templates',
  118           builder: (context, state) => const TemplatesPage(),
  119         ),
  120         GoRoute(
  121           path: RouteNames.templatesDetail,
  122           name: 'templatesDetail',
```

第 4 页


```
  123             final templateId =
state.queryParams[RouteNames.paramTemplateId];
  124             return TemplatesDetailPage(templateId: templateId);
  125           },
  126         ),
  127         GoRoute(
  128           path: RouteNames.templatesUnlock,
  129           name: 'templatesUnlock',
  130           builder: (context, state) {
  131             final templateId =
state.queryParams[RouteNames.paramTemplateId];
  132             return TemplatesUnlockPage(templateId: templateId);
  133           },
  134         ),
  135         GoRoute(
  136           path: RouteNames.templatesEditor,
  137           name: 'templatesEditor',
  138           builder: (context, state) {
  139             final templateId =
state.queryParams[RouteNames.paramTemplateId];
  140             final draftId = state.queryParams['draftId'];
  141             return TemplatesEditorPage(templateId: templateId, draftId:
draftId);
  142           },
  143         ),
  144         GoRoute(
  145           path: RouteNames.templatesDrafts,
  146           name: 'templatesDrafts',
  147           builder: (context, state) => const TemplatesDraftsPage(),
```

```
  148         ),
  149         GoRoute(
  150           path: RouteNames.templatesRecommend,
  151           name: 'templatesRecommend',
  152           builder: (context, state) => const TemplatesRecommendPage(),
  153         ),
  154         GoRoute(
  155           path: RouteNames.templatesAll,
  156           name: 'templatesAll',
  157           builder: (context, state) {
  158             final scene = state.queryParams[RouteNames.paramScene];
  159             return TemplatesAllPage(scene: scene);
  160           },
  161         ),
  162         // === 挑战 ===
  163         GoRoute(
  164           path: RouteNames.challenge,
  165           name: 'challenge',
  166           builder: (context, state) => const ChallengePage(),
  167         ),
  168         GoRoute(
  169           path: RouteNames.challengeDetail,
  170           name: 'challengeDetail',
  171           builder: (context, state) {
  172             final challengeId =
state.queryParams[RouteNames.paramChallengeId];
```

```
  173             return ChallengeDetailPage(challengeId: challengeId);
  174           },
  175         ),
  176         // === 灵感 / 相册 ===
  177         GoRoute(
  178           path: RouteNames.inspiration,
  179           name: 'inspiration',
  180           builder: (context, state) => const InspirationPage(),
  181         ),
  182         GoRoute(
  183           path: RouteNames.gallery,
  184           name: 'gallery',
  185           builder: (context, state) => const GalleryPage(),
  186         ),
  187         GoRoute(
  188           path: RouteNames.galleryDetail,
  189           name: 'galleryDetail',
  190           builder: (context, state) {
  191             final photoId = state.queryParams[RouteNames.paramPhotoId];
  192             return GalleryDetailPage(photoId: photoId);
  193           },
  194         ),
  195         GoRoute(
  196           path: RouteNames.galleryDiary,
```

第 5 页


```
  197           name: 'galleryDiary',
  198           builder: (context, state) => const GalleryDiaryPage(),
  199         ),
  200         GoRoute(
  201           path: RouteNames.galleryMonthlyDigest,
  202           name: 'galleryMonthlyDigest',
  203           builder: (context, state) => const
GalleryMonthlyDigestPage(),
  204         ),
  205         // === 个人中心 ===
  206         GoRoute(
  207           path: RouteNames.profile,
  208           name: 'profile',
  209           builder: (context, state) => const ProfilePage(),
  210         ),
  211         GoRoute(
  212           path: RouteNames.profileSettings,
  213           name: 'profileSettings',
  214           builder: (context, state) => const ProfileSettingsPage(),
  215         ),
  216         GoRoute(
  217           path: RouteNames.profileSettingsTheme,
  218           name: 'profileSettingsTheme',
  219           builder: (context, state) => const ProfileThemePage(),
  220         ),
  221         GoRoute(
```

```
  222           path: RouteNames.profileGrowth,
  223           name: 'profileGrowth',
  224           builder: (context, state) => const ProfileGrowthPage(),
  225         ),
  226         GoRoute(
  227           path: RouteNames.profileInvite,
  228           name: 'profileInvite',
  229           builder: (context, state) => const ProfileInvitePage(),
  230         ),
  231         GoRoute(
  232           path: RouteNames.profileAcademy,
  233           name: 'profileAcademy',
  234           builder: (context, state) => const ProfileAcademyPage(),
  235         ),
  236         GoRoute(
  237           path: RouteNames.profileAcademyDetail,
  238           name: 'profileAcademyDetail',
  239           builder: (context, state) {
  240             final academyId =
state.queryParams[RouteNames.paramAcademyId];
  241             return ProfileAcademyDetailPage(academyId: academyId);
  242           },
  243         ),
  244         GoRoute(
  245           path: RouteNames.profileCollections,
  246           name: 'profileCollections',
```

```
  247          builder: (context, state) => const ProfileCollectionsPage(),
  248        ),
  249        GoRoute(
  250          path: RouteNames.profileCollectionDetail,
  251          name: 'profileCollectionDetail',
  252          builder: (context, state) {
  253            final collectionId =
state.queryParams[RouteNames.paramCollectionId];
  254            return ProfileCollectionDetailPage(collectionId:
collectionId);
  255          },
  256        ),
  257        GoRoute(
  258          path: RouteNames.profileMyTemplates,
  259          name: 'profileMyTemplates',
  260          builder: (context, state) => const ProfileMyTemplatesPage(),
  261        ),
  262        GoRoute(
  263          path: RouteNames.profileAbout,
  264          name: 'profileAbout',
  265          builder: (context, state) => const ProfileAboutPage(),
  266        ),
  267        // === 场景 ===
  268        GoRoute(
  269          path: RouteNames.scenes,
  270          name: 'scenes',
  271          builder: (context, state) => const ScenesPage(),
  272        ),
  273        // === 拍摄套件 ===
  274        GoRoute(
  275          path: RouteNames.shootkitEditor,
  276          name: 'shootkitEditor',
  277          builder: (context, state) {
  278            final kitId = state.queryParams[RouteNames.paramKitId];
  279            final sceneId = state.queryParams[RouteNames.paramSceneId];
  280            return ShootkitEditorPage(kitId: kitId, sceneId: sceneId);
  281          },
  282        ),
  283      ],
  284      errorBuilder: (context, state) => Scaffold(
  285        body: Center(
  286          child: Text('Route not found: ${state.location}'),
  287        ),
  288      ),
  289    );
  290  });
```

模块 3：数据库 Provider

（lib/core/db/database_provider.dart）

第 6 页


```
import 'package:flutter/services.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:path/path.dart' as p;
import 'package:path_provider/path_provider.dart';
import 'package:sqflite/sqflite.dart';
import 'tables.dart';
import 'dao/templates_dao.dart';
import 'dao/scenes_dao.dart';
import 'dao/gallery_dao.dart';
const String _kDbName = 'lumira.db';
const int _kDbVersion = 1;
/// 数据库 Provider
/// 在应用启动时首次 read 时初始化（lazy），后续 read 返回同一实例
///
/// Forced fix: HarmonyOS 平台缺少 path_provider 原生实现，
/// 调用 getApplicationDocumentsDirectory() 会抛 MissingPluginException。
/// 降级到 sqflite.getDatabasePath()（sqflite 内置的默认路径）。
final databaseProvider = FutureProvider<Database>((ref) async {
  String dbPath;
  try {
    final docsDir = await getApplicationDocumentsDirectory();
    dbPath = p.join(docsDir.path, _kDbName);
  } on PlatformException catch (_) {
    // Forced fix: HarmonyOS 降级 - 用 sqflite.getDatabasePath
    final dbDir = await getDatabasesPath();
    dbPath = p.join(dbDir, _kDbName);
  } catch (_) {
    // Forced fix: 其他平台异常时再次降级
    final dbDir = await getDatabasesPath();
    dbPath = p.join(dbDir, _kDbName);
  }
  final db = await openDatabase(
    dbPath,
    version: _kDbVersion,
    onCreate: _onCreate,
    onUpgrade: _onUpgrade,
  );
  ref.onDispose(db.close);
  return db;
});
final templatesDaoProvider = FutureProvider<TemplatesDao>((ref) async {
  final db = await ref.watch(databaseProvider.future);
  return TemplatesDao(db);
});
final scenesDaoProvider = FutureProvider<ScenesDao>((ref) async {
  final db = await ref.watch(databaseProvider.future);
  return ScenesDao(db);
});
final galleryDaoProvider = FutureProvider<GalleryDao>((ref) async {
```

```
50    final db = await ref.watch(databaseProvider.future);
```

```
   51    return GalleryDao(db);
   52  });
   53  Future<void> _onCreate(Database db, int version) async {
   54    final batch = db.batch();
   55    // === custom_templates ===
   56    batch.execute('''
   57      CREATE TABLE IF NOT EXISTS ${Tables.customTemplates} (
   58        ${Tables.colId} TEXT PRIMARY KEY,
   59        ${Tables.colName} TEXT NOT NULL,
   60        ${Tables.colAuthor} TEXT NOT NULL DEFAULT '',
   61        ${Tables.colVersion} TEXT NOT NULL DEFAULT '1.0.0',
   62        ${Tables.colCategory} TEXT NOT NULL,
   63        ${Tables.colClassificationJson} TEXT NOT NULL DEFAULT '{}',
   64        ${Tables.colTagsJson} TEXT NOT NULL DEFAULT '[]',
   65        ${Tables.colTagIdsJson} TEXT NOT NULL DEFAULT '[]',
   66        ${Tables.colPrice} INTEGER NOT NULL DEFAULT 0,
   67        ${Tables.colCover} TEXT NOT NULL DEFAULT '',
   68        ${Tables.colDescription} TEXT NOT NULL DEFAULT '',
   69        ${Tables.colReferenceSource} TEXT NOT NULL DEFAULT '',
   70        ${Tables.colCompositionJson} TEXT NOT NULL DEFAULT '{}',
   71        ${Tables.colPoseJson} TEXT NOT NULL DEFAULT '{}',
   72        ${Tables.colCameraJson} TEXT NOT NULL DEFAULT '{}',
   73        ${Tables.colSceneGuideJson} TEXT NOT NULL DEFAULT '{}',
   74        ${Tables.colPostProcessJson} TEXT NOT NULL DEFAULT '{}',
   75        ${Tables.colCreatedAt} INTEGER NOT NULL,
   76        ${Tables.colUpdatedAt} INTEGER NOT NULL
   77      )
   78    ''');
   79    batch.execute('CREATE INDEX IF NOT EXISTS
idx_custom_templates_category ON
${Tables.customTemplates}(${Tables.colCategory})');
   80    batch.execute('CREATE INDEX IF NOT EXISTS
idx_custom_templates_created_at ON
${Tables.customTemplates}(${Tables.colCreatedAt} DESC)');
```

第 7 页


```
   81    // === scenes ===
   82    // 仅存储用户自定义场景 + 内置场景的 is_favorite 标记
   83    // 内置场景的完整数据由代码常量提供
   84    batch.execute('''
   85      CREATE TABLE IF NOT EXISTS ${Tables.scenes} (
   86        ${Tables.colId} TEXT PRIMARY KEY,
   87        ${Tables.colName} TEXT NOT NULL,
   88        ${Tables.colIcon} TEXT NOT NULL DEFAULT '',
   89        ${Tables.colCategory} TEXT NOT NULL,
   90        ${Tables.colStyle} TEXT NOT NULL DEFAULT '',
   91        ${Tables.colFilterJson} TEXT NOT NULL DEFAULT '{}',
   92        ${Tables.colVibe} TEXT NOT NULL DEFAULT '',
   93        ${Tables.colDescription} TEXT NOT NULL DEFAULT '',
   94        ${Tables.colExampleImagesJson} TEXT NOT NULL DEFAULT '[]',
   95        ${Tables.colTipsJson} TEXT NOT NULL DEFAULT '[]',
   96        ${Tables.colWhereToShoot} TEXT NOT NULL DEFAULT '',
```

```
   97        ${Tables.colBestTime} TEXT NOT NULL DEFAULT '',
   98        ${Tables.colSceneGuideJson} TEXT NOT NULL DEFAULT '{}',
   99        ${Tables.colRelatedCategory} TEXT NOT NULL DEFAULT '',
  100        ${Tables.colRecommendedTagIdsJson} TEXT NOT NULL DEFAULT '[]',
  101        ${Tables.colTagIdsJson} TEXT NOT NULL DEFAULT '[]',
  102        ${Tables.colCreator} TEXT NOT NULL DEFAULT 'user',
  103        ${Tables.colIsFavorite} INTEGER NOT NULL DEFAULT 0,
  104        ${Tables.colCreatedAt} INTEGER NOT NULL,
  105        ${Tables.colUpdatedAt} INTEGER NOT NULL
  106      )
  107    ''');
  108    batch.execute('CREATE INDEX IF NOT EXISTS idx_scenes_category ON
${Tables.scenes}(${Tables.colCategory})');
  109    batch.execute('CREATE INDEX IF NOT EXISTS idx_scenes_is_favorite
ON ${Tables.scenes}(${Tables.colIsFavorite}) WHERE ${Tables.colIsFavorite}
= 1');
  110    // === gallery_items ===
  111    // 图片本体优先存文件路径（file_path），data_url 保留兼容旧数据
  112    batch.execute('''
  113      CREATE TABLE IF NOT EXISTS ${Tables.galleryItems} (
  114        ${Tables.colId} TEXT PRIMARY KEY,
  115        ${Tables.colDataUrl} TEXT,
  116        ${Tables.colFilePath} TEXT,
  117        ${Tables.colSceneId} TEXT,
  118        ${Tables.colTemplateId} TEXT,
  119        ${Tables.colKitId} TEXT,
  120        ${Tables.colMood} TEXT,
  121        ${Tables.colLut} TEXT,
  122        ${Tables.colCreatedAt} INTEGER NOT NULL
  123      )
  124    ''');
  125    batch.execute('CREATE INDEX IF NOT EXISTS
idx_gallery_items_created_at ON
${Tables.galleryItems}(${Tables.colCreatedAt} DESC)');
  126    batch.execute('CREATE INDEX IF NOT EXISTS
idx_gallery_items_scene_id ON
${Tables.galleryItems}(${Tables.colSceneId})');
  127    // === user_progress (单行表，id=1) ===
  128    // uni-app 中未持久化，Flutter 端新增
  129    batch.execute('''
  130      CREATE TABLE IF NOT EXISTS ${Tables.userProgress} (
```

```
131         ${Tables.colId} INTEGER PRIMARY KEY DEFAULT 1,
132         ${Tables.colLevel} INTEGER NOT NULL DEFAULT 1,
133         ${Tables.colLevelName} TEXT NOT NULL DEFAULT '新手',
134         ${Tables.colXp} INTEGER NOT NULL DEFAULT 0,
135         ${Tables.colXpToNextLevel} INTEGER NOT NULL DEFAULT 100,
136         ${Tables.colTotalPhotos} INTEGER NOT NULL DEFAULT 0,
137         ${Tables.colUsedTemplates} INTEGER NOT NULL DEFAULT 0,
138         ${Tables.colFavorites} INTEGER NOT NULL DEFAULT 0,
139         ${Tables.colStreakDays} INTEGER NOT NULL DEFAULT 0,
140         ${Tables.colLastCheckInDate} TEXT,
141         ${Tables.colFragmentsJson} TEXT NOT NULL DEFAULT '[]',
142         ${Tables.colAchievementsJson} TEXT NOT NULL DEFAULT '[]',
143         ${Tables.colUpdatedAt} INTEGER NOT NULL
144       )
145     ''');
146     // 初始化单行
147     batch.insert(Tables.userProgress, {
148       Tables.colId: 1,
149       Tables.colUpdatedAt: DateTime.now().millisecondsSinceEpoch,
150     });
```

第 8 页


```
151     // === user_settings (单行表，id=1) ===
152     batch.execute('''
153       CREATE TABLE IF NOT EXISTS ${Tables.userSettings} (
154         ${Tables.colId} INTEGER PRIMARY KEY DEFAULT 1,
155         ${Tables.colThemeKey} TEXT NOT NULL DEFAULT 'warmWhite',
156         ${Tables.colUiStyle} TEXT NOT NULL DEFAULT 'neumorphic',
157         ${Tables.colFollowSystem} INTEGER NOT NULL DEFAULT 0,
158         ${Tables.colCaptureFullscreen} INTEGER NOT NULL DEFAULT 0,
159         ${Tables.colGridEnabled} INTEGER NOT NULL DEFAULT 0,
160         ${Tables.colLevelEnabled} INTEGER NOT NULL DEFAULT 0,
161         ${Tables.colShutterSound} INTEGER NOT NULL DEFAULT 1,
162         ${Tables.colWatermark} INTEGER NOT NULL DEFAULT 0,
163         ${Tables.colUpdatedAt} INTEGER NOT NULL
164       )
165     ''');
166     batch.insert(Tables.userSettings, {
167       Tables.colId: 1,
168       Tables.colUpdatedAt: DateTime.now().millisecondsSinceEpoch,
169     });
170     await batch.commit(noResult: true);
171   }
172   Future<void> _onUpgrade(Database db, int oldVersion, int newVersion)
async {
173     // 当前 v1，未来版本迁移在此扩展
174     // 不做 destructive 迁移（不 DROP TABLE），保留用户数据
175   }
176   // ===== lib/core/db/tables.dart =====
177   /// 所有数据库表名与列名常量。
178   /// 字段设计基于 uni-app 源码逆向研究：
179   /// e:\Project\photo_post\.superpowers\sdd\task-1.4-research.md
```

```
class Tables {
  Tables._();
  // === custom_templates ===
  static const String customTemplates = 'custom_templates';
  static const String colId = 'id';
  static const String colName = 'name';
  static const String colAuthor = 'author';
  static const String colVersion = 'version';
  static const String colCategory = 'category';
  static const String colClassificationJson = 'classification_json';
  static const String colTagsJson = 'tags_json';
  static const String colTagIdsJson = 'tag_ids_json';
  static const String colPrice = 'price';
  static const String colCover = 'cover';
  static const String colDescription = 'description';
  static const String colReferenceSource = 'reference_source';
  static const String colCompositionJson = 'composition_json';
  static const String colPoseJson = 'pose_json';
  static const String colCameraJson = 'camera_json';
  static const String colSceneGuideJson = 'scene_guide_json';
  static const String colPostProcessJson = 'post_process_json';
```

```
  201    static const String colCreatedAt = 'created_at';
  202    static const String colUpdatedAt = 'updated_at';
  203    // === scenes ===
  204    static const String scenes = 'scenes';
  205    static const String colIcon = 'icon';
  206    static const String colStyle = 'style';
  207    static const String colFilterJson = 'filter_json';
  208    static const String colVibe = 'vibe';
  209    static const String colExampleImagesJson = 'example_images_json';
  210    static const String colTipsJson = 'tips_json';
  211    static const String colWhereToShoot = 'where_to_shoot';
  212    static const String colBestTime = 'best_time';
  213    static const String colRelatedCategory = 'related_category';
  214    static const String colRecommendedTagIdsJson =
'recommended_tag_ids_json';
  215    // colTagIdsJson 已在 custom_templates 段声明，此处不重复声明
  216    static const String colCreator = 'creator';
  217    static const String colIsFavorite = 'is_favorite';
  218    // === gallery_items ===
  219    static const String galleryItems = 'gallery_items';
  220    static const String colDataUrl = 'data_url';
  221    static const String colFilePath = 'file_path';
  222    static const String colSceneId = 'scene_id';
  223    static const String colTemplateId = 'template_id';
  224    static const String colKitId = 'kit_id';
  225    static const String colMood = 'mood';
  226    static const String colLut = 'lut';
  227    // === user_progress ===
  228    static const String userProgress = 'user_progress';
  229    static const String colLevel = 'level';
  230    static const String colLevelName = 'level_name';
```

第 9 页


```
  231    static const String colXp = 'xp';
  232    static const String colXpToNextLevel = 'xp_to_next_level';
  233    static const String colTotalPhotos = 'total_photos';
  234    static const String colUsedTemplates = 'used_templates';
  235    static const String colFavorites = 'favorites';
  236    static const String colStreakDays = 'streak_days';
  237    static const String colLastCheckInDate = 'last_check_in_date';
  238    static const String colFragmentsJson = 'fragments_json';
  239    static const String colAchievementsJson = 'achievements_json';
  240    // === user_settings ===
  241    static const String userSettings = 'user_settings';
  242    static const String colThemeKey = 'theme_key';
  243    static const String colUiStyle = 'ui_style';
  244    static const String colFollowSystem = 'follow_system';
  245    static const String colCaptureFullscreen = 'capture_fullscreen';
  246    static const String colGridEnabled = 'grid_enabled';
  247    static const String colLevelEnabled = 'level_enabled';
  248    static const String colShutterSound = 'shutter_sound';
  249    static const String colWatermark = 'watermark';
  250  }
```

模块 4：模板 DAO（lib/core/db/dao/templates_dao.dart）

第 10 页

如画 Lumira 摄影辅助应用 V1.0.0 第 10 页

```
import 'dart:convert';
import 'package:sqflite/sqflite.dart';
import '../tables.dart';
/// 简化的模板实体（仅常用字段；完整 PhotoTemplate 在
lib/features/templates/ 中定义）
/// 本任务范围：DB 层只负责存取原始 JSON 数据
class TemplateRecord {
  final String id;
  final String name;
  final String author;
  final String version;
  final String category;
  final Map<String, dynamic> classification;
  final List<String> tags;
  final List<String> tagIds;
  final int price;
  final String cover;
  final String description;
  final String referenceSource;
  final Map<String, dynamic> composition;
  final Map<String, dynamic> pose;
  final Map<String, dynamic> camera;
  final Map<String, dynamic> sceneGuide;
  final Map<String, dynamic> postProcess;
  final int createdAt;
  final int updatedAt;
  TemplateRecord({
    required this.id,
    required this.name,
    required this.author,
    required this.version,
    required this.category,
    required this.classification,
    required this.tags,
    required this.tagIds,
    required this.price,
    required this.cover,
    required this.description,
    required this.referenceSource,
    required this.composition,
    required this.pose,
    required this.camera,
    required this.sceneGuide,
    required this.postProcess,
    required this.createdAt,
```

```
45      required this.updatedAt,
46    });
47    Map<String, Object?> toRow() {
48      return {
49        Tables.colId: id,
50        Tables.colName: name,
```

```
   51        Tables.colAuthor: author,
   52        Tables.colVersion: version,
   53        Tables.colCategory: category,
   54        Tables.colClassificationJson: jsonEncode(classification),
   55        Tables.colTagsJson: jsonEncode(tags),
   56        Tables.colTagIdsJson: jsonEncode(tagIds),
   57        Tables.colPrice: price,
   58        Tables.colCover: cover,
   59        Tables.colDescription: description,
   60        Tables.colReferenceSource: referenceSource,
   61        Tables.colCompositionJson: jsonEncode(composition),
   62        Tables.colPoseJson: jsonEncode(pose),
   63        Tables.colCameraJson: jsonEncode(camera),
   64        Tables.colSceneGuideJson: jsonEncode(sceneGuide),
   65        Tables.colPostProcessJson: jsonEncode(postProcess),
   66        Tables.colCreatedAt: createdAt,
   67        Tables.colUpdatedAt: updatedAt,
   68      };
   69    }
   70    static TemplateRecord fromRow(Map<String, Object?> row) {
   71      return TemplateRecord(
   72        id: row[Tables.colId] as String,
   73        name: row[Tables.colName] as String,
   74        author: (row[Tables.colAuthor] as String?) ?? '',
   75        version: (row[Tables.colVersion] as String?) ?? '1.0.0',
   76        category: row[Tables.colCategory] as String,
   77        classification:
_decodeJsonMap(row[Tables.colClassificationJson]),
   78        tags: _decodeJsonList(row[Tables.colTagsJson]),
   79        tagIds: _decodeJsonList(row[Tables.colTagIdsJson]),
   80        price: (row[Tables.colPrice] as num?)?.toInt() ?? 0,
```

第 11 页


```
   81        cover: (row[Tables.colCover] as String?) ?? '',
   82        description: (row[Tables.colDescription] as String?) ?? '',
   83        referenceSource: (row[Tables.colReferenceSource] as String?) ??
'',
   84        composition: _decodeJsonMap(row[Tables.colCompositionJson]),
   85        pose: _decodeJsonMap(row[Tables.colPoseJson]),
   86        camera: _decodeJsonMap(row[Tables.colCameraJson]),
   87        sceneGuide: _decodeJsonMap(row[Tables.colSceneGuideJson]),
   88        postProcess: _decodeJsonMap(row[Tables.colPostProcessJson]),
   89        createdAt: (row[Tables.colCreatedAt] as num).toInt(),
   90        updatedAt: (row[Tables.colUpdatedAt] as num).toInt(),
   91      );
   92    }
   93    static Map<String, dynamic> _decodeJsonMap(Object? raw) {
   94      if (raw is String && raw.isNotEmpty) {
   95        return jsonDecode(raw) as Map<String, dynamic>;
   96      }
   97      return <String, dynamic>{};
   98    }
   99    static List<String> _decodeJsonList(Object? raw) {
```

```
  100      if (raw is String && raw.isNotEmpty) {
  101        final list = jsonDecode(raw) as List<dynamic>;
  102        return list.cast<String>();
  103      }
  104      return <String>[];
  105    }
  106  }
  107  class TemplatesDao {
  108    TemplatesDao(this._db);
  109    final Database _db;
  110    Future<List<TemplateRecord>> getAll({String? category}) async {
  111      final where = category != null ? '${Tables.colCategory} = ?' :
null;
  112      final whereArgs = category != null ? [category] : null;
  113      final rows = await _db.query(
  114        Tables.customTemplates,
  115        where: where,
  116        whereArgs: whereArgs,
  117        orderBy: '${Tables.colCreatedAt} DESC',
  118      );
  119      return rows.map(TemplateRecord.fromRow).toList();
  120    }
  121    Future<TemplateRecord?> getById(String id) async {
  122      final rows = await _db.query(
  123        Tables.customTemplates,
  124        where: '${Tables.colId} = ?',
  125        whereArgs: [id],
  126        limit: 1,
  127      );
  128      if (rows.isEmpty) return null;
  129      return TemplateRecord.fromRow(rows.first);
  130    }
```

```
  131    /// Upsert: 按 id 插入或更新
  132    Future<void> upsert(TemplateRecord record) async {
  133      await _db.insert(
  134        Tables.customTemplates,
  135        record.toRow(),
  136        conflictAlgorithm: ConflictAlgorithm.replace,
  137      );
  138    }
  139    Future<int> delete(String id) async {
  140      return _db.delete(
  141        Tables.customTemplates,
  142        where: '${Tables.colId} = ?',
  143        whereArgs: [id],
  144      );
  145    }
  146    Future<int> count() async {
  147      final rows = await _db.rawQuery('SELECT COUNT(*) AS cnt FROM
${Tables.customTemplates}');
  148      return Sqflite.firstIntValue(rows) ?? 0;
  149    }
  150  }
```

模块 5：场景 DAO（lib/core/db/dao/scenes_dao.dart）

第 12 页

如画 Lumira 摄影辅助应用 V1.0.0 第 12 页

```
    1  import 'dart:convert';
    2  import 'package:sqflite/sqflite.dart';
    3  import '../tables.dart';
    4  class SceneRecord {
    5    final String id;
    6    final String name;
    7    final String icon;
    8    final String category;
    9    final String style;
   10    final Map<String, dynamic> filter;
   11    final String vibe;
   12    final String description;
   13    final List<String> exampleImages;
   14    final List<String> tips;
   15    final String whereToShoot;
   16    final String bestTime;
   17    final Map<String, dynamic> sceneGuide;
   18    final String relatedCategory;
   19    final List<String> recommendedTagIds;
   20    final List<String> tagIds;
   21    final String creator; // 'user' | 'system'
   22    final bool isFavorite;
   23    final int createdAt;
```

```
24    final int updatedAt;
25    SceneRecord({
26      required this.id,
27      required this.name,
28      required this.icon,
29      required this.category,
30      required this.style,
31      required this.filter,
32      required this.vibe,
33      required this.description,
34      required this.exampleImages,
35      required this.tips,
36      required this.whereToShoot,
37      required this.bestTime,
38      required this.sceneGuide,
39      required this.relatedCategory,
40      required this.recommendedTagIds,
41      required this.tagIds,
42      required this.creator,
43      required this.isFavorite,
44      required this.createdAt,
45      required this.updatedAt,
46    });
47    Map<String, Object?> toRow() {
48      return {
49        Tables.colId: id,
50        Tables.colName: name,
```

```
   51        Tables.colIcon: icon,
   52        Tables.colCategory: category,
   53        Tables.colStyle: style,
   54        Tables.colFilterJson: jsonEncode(filter),
   55        Tables.colVibe: vibe,
   56        Tables.colDescription: description,
   57        Tables.colExampleImagesJson: jsonEncode(exampleImages),
   58        Tables.colTipsJson: jsonEncode(tips),
   59        Tables.colWhereToShoot: whereToShoot,
   60        Tables.colBestTime: bestTime,
   61        Tables.colSceneGuideJson: jsonEncode(sceneGuide),
   62        Tables.colRelatedCategory: relatedCategory,
   63        Tables.colRecommendedTagIdsJson: jsonEncode(recommendedTagIds),
   64        Tables.colTagIdsJson: jsonEncode(tagIds),
   65        Tables.colCreator: creator,
   66        Tables.colIsFavorite: isFavorite ? 1 : 0,
   67        Tables.colCreatedAt: createdAt,
   68        Tables.colUpdatedAt: updatedAt,
   69      };
   70    }
   71    static SceneRecord fromRow(Map<String, Object?> row) {
   72      return SceneRecord(
   73        id: row[Tables.colId] as String,
   74        name: row[Tables.colName] as String,
   75        icon: (row[Tables.colIcon] as String?) ?? '',
   76        category: row[Tables.colCategory] as String,
   77        style: (row[Tables.colStyle] as String?) ?? '',
   78        filter: _decodeJsonMap(row[Tables.colFilterJson]),
   79        vibe: (row[Tables.colVibe] as String?) ?? '',
   80        description: (row[Tables.colDescription] as String?) ?? '',
```

第 13 页


```
   81        exampleImages:
_decodeJsonList(row[Tables.colExampleImagesJson]),
   82        tips: _decodeJsonList(row[Tables.colTipsJson]),
   83        whereToShoot: (row[Tables.colWhereToShoot] as String?) ?? '',
   84        bestTime: (row[Tables.colBestTime] as String?) ?? '',
   85        sceneGuide: _decodeJsonMap(row[Tables.colSceneGuideJson]),
   86        relatedCategory: (row[Tables.colRelatedCategory] as String?) ??
'',
   87        recommendedTagIds:
_decodeJsonList(row[Tables.colRecommendedTagIdsJson]),
   88        tagIds: _decodeJsonList(row[Tables.colTagIdsJson]),
   89        creator: (row[Tables.colCreator] as String?) ?? 'user',
   90        isFavorite: (row[Tables.colIsFavorite] as num?)?.toInt() == 1,
   91        createdAt: (row[Tables.colCreatedAt] as num).toInt(),
   92        updatedAt: (row[Tables.colUpdatedAt] as num).toInt(),
   93      );
   94    }
   95    static Map<String, dynamic> _decodeJsonMap(Object? raw) {
   96      if (raw is String && raw.isNotEmpty) {
   97        return jsonDecode(raw) as Map<String, dynamic>;
   98      }
```

```
 99      return <String, dynamic>{};
100    }
101    static List<String> _decodeJsonList(Object? raw) {
102      if (raw is String && raw.isNotEmpty) {
103        final list = jsonDecode(raw) as List<dynamic>;
104        return list.cast<String>();
105      }
106      return <String>[];
107    }
108  }
109  class ScenesDao {
110    ScenesDao(this._db);
111    final Database _db;
112    Future<List<SceneRecord>> getCustomScenes() async {
113      final rows = await _db.query(
114        Tables.scenes,
115        where: '${Tables.colCreator} = ?',
116        whereArgs: ['user'],
117        orderBy: '${Tables.colCreatedAt} DESC',
118      );
119      return rows.map(SceneRecord.fromRow).toList();
120    }
121    Future<List<SceneRecord>> getFavorites() async {
122      final rows = await _db.query(
123        Tables.scenes,
124        where: '${Tables.colIsFavorite} = ?',
125        whereArgs: [1],
126      );
127      return rows.map(SceneRecord.fromRow).toList();
128    }
129    /// 切换收藏标记（upsert minimal row 仅含 id + is_favorite）
130    /// 用于内置场景：内置场景完整数据由代码常量提供，DB 只持久化收藏标记
```

```
  131    Future<void> toggleFavorite(String sceneId, bool favorite) async {
  132      final now = DateTime.now().millisecondsSinceEpoch;
  133      final existing = await _db.query(
  134        Tables.scenes,
  135        where: '${Tables.colId} = ?',
  136        whereArgs: [sceneId],
  137        limit: 1,
  138      );
  139      if (existing.isEmpty) {
  140        // 内置场景首次收藏：插入最小行（仅 id + is_favorite + 必填字段）
  141        await _db.insert(Tables.scenes, {
  142          Tables.colId: sceneId,
  143          Tables.colName: '',
  144          Tables.colCategory: '',
  145          Tables.colCreator: 'system',
  146          Tables.colIsFavorite: favorite ? 1 : 0,
  147          Tables.colCreatedAt: now,
  148          Tables.colUpdatedAt: now,
  149        });
  150      } else {
```

第 14 页


```
  151        await _db.update(
  152          Tables.scenes,
  153          {
  154            Tables.colIsFavorite: favorite ? 1 : 0,
  155            Tables.colUpdatedAt: now,
  156          },
  157          where: '${Tables.colId} = ?',
  158          whereArgs: [sceneId],
  159        );
  160      }
  161    }
  162    Future<void> upsert(SceneRecord record) async {
  163      await _db.insert(
  164        Tables.scenes,
  165        record.toRow(),
  166        conflictAlgorithm: ConflictAlgorithm.replace,
  167      );
  168    }
  169    Future<int> delete(String id) async {
  170      return _db.delete(
  171        Tables.scenes,
  172        where: '${Tables.colId} = ?',
  173        whereArgs: [id],
  174      );
  175    }
  176    Future<int> countCustom() async {
  177      final rows = await _db.rawQuery(
  178        'SELECT COUNT(*) AS cnt FROM ${Tables.scenes} WHERE
${Tables.colCreator} = ?',
  179        ['user'],
  180      );
```

```
    return Sqflite.firstIntValue(rows) ?? 0;
  }
}
// ===== lib/core/db/dao/gallery_dao.dart =====
import 'package:sqflite/sqflite.dart';
import '../tables.dart';
class GalleryItemRecord {
  final String id;
  final String? dataUrl;
  final String? filePath;
  final String? sceneId;
  final String? templateId;
  final String? kitId;
  final String? mood;
  final String? lut;
  final int createdAt;
  GalleryItemRecord({
    required this.id,
    required this.dataUrl,
    required this.filePath,
```

```
  201       required this.sceneId,
  202       required this.templateId,
  203       required this.kitId,
  204       required this.mood,
  205       required this.lut,
  206       required this.createdAt,
  207     });
  208     Map<String, Object?> toRow() {
  209       return {
  210         Tables.colId: id,
  211         Tables.colDataUrl: dataUrl,
  212         Tables.colFilePath: filePath,
  213         Tables.colSceneId: sceneId,
  214         Tables.colTemplateId: templateId,
  215         Tables.colKitId: kitId,
  216         Tables.colMood: mood,
  217         Tables.colLut: lut,
  218         Tables.colCreatedAt: createdAt,
  219       };
  220     }
  221     static GalleryItemRecord fromRow(Map<String, Object?> row) {
  222       return GalleryItemRecord(
  223         id: row[Tables.colId] as String,
  224         dataUrl: row[Tables.colDataUrl] as String?,
  225         filePath: row[Tables.colFilePath] as String?,
  226         sceneId: row[Tables.colSceneId] as String?,
  227         templateId: row[Tables.colTemplateId] as String?,
  228         kitId: row[Tables.colKitId] as String?,
  229         mood: row[Tables.colMood] as String?,
  230         lut: row[Tables.colLut] as String?,
```

第 15 页


```
  231         createdAt: (row[Tables.colCreatedAt] as num).toInt(),
  232       );
  233     }
  234   }
  235   class GalleryDao {
  236     GalleryDao(this._db);
  237     final Database _db;
  238     /// 获取所有照片（最新在前，对应 uni-app 数组头部插入的语义）
  239     Future<List<GalleryItemRecord>> getAll({int? limit, int? offset})
async {
  240       final rows = await _db.query(
  241         Tables.galleryItems,
  242         orderBy: '${Tables.colCreatedAt} DESC',
  243         limit: limit,
  244         offset: offset,
  245       );
  246       return rows.map(GalleryItemRecord.fromRow).toList();
  247     }
  248     Future<List<GalleryItemRecord>> getByScene(String sceneId) async {
  249       final rows = await _db.query(
  250         Tables.galleryItems,
```

```
251         where: '${Tables.colSceneId} = ?',
252         whereArgs: [sceneId],
253         orderBy: '${Tables.colCreatedAt} DESC',
254       );
255       return rows.map(GalleryItemRecord.fromRow).toList();
256     }
257     Future<GalleryItemRecord?> getById(String id) async {
258       final rows = await _db.query(
259         Tables.galleryItems,
260         where: '${Tables.colId} = ?',
261         whereArgs: [id],
262         limit: 1,
263       );
264       if (rows.isEmpty) return null;
265       return GalleryItemRecord.fromRow(rows.first);
266     }
267     Future<void> insert(GalleryItemRecord record) async {
268       await _db.insert(
269         Tables.galleryItems,
270         record.toRow(),
271         conflictAlgorithm: ConflictAlgorithm.abort,
272       );
273     }
274     Future<int> updateScene(String photoId, String? sceneId) async {
275       return _db.update(
276         Tables.galleryItems,
277         {Tables.colSceneId: sceneId},
278         where: '${Tables.colId} = ?',
279         whereArgs: [photoId],
280       );
```

```
  281    }
  281    Future<int> delete(String id) async {
  282      return _db.delete(
  283        Tables.galleryItems,
  284        where: '${Tables.colId} = ?',
  285        whereArgs: [id],
  286      );
  287    }
  288    Future<int> count() async {
  289      final rows = await _db.rawQuery('SELECT COUNT(*) AS cnt FROM
${Tables.galleryItems}');
  290      return Sqflite.firstIntValue(rows) ?? 0;
  291    }
  292    Future<int> countByScene(String sceneId) async {
  293      final rows = await _db.rawQuery(
  294        'SELECT COUNT(*) AS cnt FROM ${Tables.galleryItems} WHERE
${Tables.colSceneId} = ?',
  295        [sceneId],
  296      );
  297      return Sqflite.firstIntValue(rows) ?? 0;
  298    }
  299    /// 按月分组统计（用于 monthly-digest 页）
  300    Future<List<Map<String, dynamic>>> monthlyCounts() async {
  301      return _db.rawQuery('''
  302        SELECT
  303          strftime('%Y-%m', ${Tables.colCreatedAt} / 1000, 'unixepoch',
'localtime') AS month,
  304          COUNT(*) AS cnt
  305        FROM ${Tables.galleryItems}
  306        GROUP BY month
  307        ORDER BY month DESC
  308      ''');
  309    }
  310  }
```

模块 6：主题 Token 系统（lib/core/theme/theme_tokens.dart）

第 16 页

如画 Lumira 摄影辅助应用 V1.0.0 第 16 页

```
    1  import 'package:flutter/material.dart';
    2  enum ThemeKey {
    3    warmWhite,
    4    ink,
    5    retro,
    6    fresh,
    7    cozy,
    8    macaron,
    9    morandi,
   10    rosegold,
```

```
}
enum UIStyle {
  neumorphic,
  flat,
  glass,
  female,
}
class ThemeTokens {
  final Color canvas;
  final Color surface;
  final Color surfaceAlt;
  final Color canvasDeep;
  final Color textPrimary;
  final Color textSecondary;
  final Color textTertiary;
  final Color textInverse;
  final Color divider;
  final Color brand;
  final Color brandDeep;
  final Color brandLight; // 注意：warmWhite/ink/retro/fresh 主题在App.vue 中未显式定义 --color-brand-light，需用 brand 色稍亮派生
  final Color brandSubtle;
  final Color brandText;
  final Color danger;
  final Color dangerSubtle;
  final Color success;
  final Color successSubtle;
  final List<BoxShadow> shadowConvex;
  final List<BoxShadow> shadowConvexSubtle;
  final List<BoxShadow> shadowConvexBrand;
  final List<BoxShadow> shadowConcave;
  final List<BoxShadow> shadowConcaveSubtle;
  final List<BoxShadow> shadowPressed;
  final List<BoxShadow> shadowFloat;
  const ThemeTokens({
    required this.canvas,
    required this.surface,
    required this.surfaceAlt,
    required this.canvasDeep,
    required this.textPrimary,
    required this.textSecondary,
```

```
51      required this.textTertiary,
52      required this.textInverse,
53      required this.divider,
54      required this.brand,
55      required this.brandDeep,
56      required this.brandLight,
57      required this.brandSubtle,
58      required this.brandText,
59      required this.danger,
60      required this.dangerSubtle,
61      required this.success,
62      required this.successSubtle,
63      required this.shadowConvex,
64      required this.shadowConvexSubtle,
65      required this.shadowConvexBrand,
66      required this.shadowConcave,
67      required this.shadowConcaveSubtle,
68      required this.shadowPressed,
69      required this.shadowFloat,
70    });
71    static ThemeTokens of(ThemeKey theme) {
72      switch (theme) {
73        case ThemeKey.warmWhite:
74          return _warmWhiteTokens;
75        case ThemeKey.ink:
76          return _inkTokens;
77        case ThemeKey.retro:
78          return _retroTokens;
79        case ThemeKey.fresh:
80          return _freshTokens;
81        case ThemeKey.cozy:
82          return _cozyTokens;
83        case ThemeKey.macaron:
84          return _macaronTokens;
85        case ThemeKey.morandi:
86          return _morandiTokens;
87        case ThemeKey.rosegold:
88          return _rosegoldTokens;
89      }
90    }
91    // === 暖米白主题（默认） ===
92    // 来源: App.vue :root (line 40-80)
93    static const _warmWhiteTokens = ThemeTokens(
94      canvas: Color(0xFFFAF7F2),
95      surface: Color(0xFFFFFFFF),
96      surfaceAlt: Color(0xFFF2EEE6),
97      canvasDeep: Color(0xFFF5F1EB),
98      textPrimary: Color(0xFF1A1A1A),
99      textSecondary: Color(0xFF5C5852),
100     textTertiary: Color(0xFF9C9690),
```

```
  101      textInverse: Color(0xFFFAF7F2),
  102      divider: Color(0xFFEAE5DC),
  103      brand: Color(0xFFC9A96E),
  104      brandDeep: Color(0xFFA88550),
  105      brandLight: Color(0xFFD4B57A), // App.vue 显式定义
  106      brandSubtle: Color(0xFFF5EDDB),
  107      brandText: Color(0xFF8C7340),
  108      danger: Color(0xFFB85450),
  109      dangerSubtle: Color(0xFFF5E3E0),
  110      success: Color(0xFF7A8B5C),
  111      successSubtle: Color(0xFFEBEEE2),
```

第 17 页


```
  112      shadowConvex: [
  113        BoxShadow(color: Color(0xFFD8D4CC), offset: Offset(6, 6),
blurRadius: 14),
  114        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-6, -6),
blurRadius: 14),
  115      ],
  116      shadowConvexSubtle: [
  117        BoxShadow(color: Color(0xFFE0DCD4), offset: Offset(3, 3),
blurRadius: 6),
  118        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-3, -3),
blurRadius: 6),
  119      ],
  120      shadowConvexBrand: [
  121        BoxShadow(color: Color(0xFFB89A5E), offset: Offset(4, 4),
blurRadius: 10),
  122        BoxShadow(color: Color(0xFFDABB82), offset: Offset(-4, -4),
blurRadius: 10),
  123      ],
  124      shadowConcave: [
  125        BoxShadow(color: Color(0xFFE0DCD4), offset: Offset(4, 4),
blurRadius: 10, blurStyle: BlurStyle.inner),
  126        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-4, -4),
blurRadius: 10, blurStyle: BlurStyle.inner),
  127      ],
  128      shadowConcaveSubtle: [
  129        BoxShadow(color: Color(0xFFE5E0D8), offset: Offset(2, 2),
blurRadius: 5, blurStyle: BlurStyle.inner),
  130        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-2, -2),
blurRadius: 5, blurStyle: BlurStyle.inner),
  131      ],
  132      shadowPressed: [
  133        BoxShadow(color: Color(0xFFE0DCD4), offset: Offset(3, 3),
blurRadius: 8, blurStyle: BlurStyle.inner),
  134        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-3, -3),
blurRadius: 8, blurStyle: BlurStyle.inner),
  135      ],
  136      shadowFloat: [
  137        BoxShadow(color: Color(0x141A1A1A), offset: Offset(0, 8),
blurRadius: 32),
  138      ],
```

```
139    );
140    // === 浓墨主题 ===
141    // 来源: App.vue [data-theme="ink"] (line 83-110)
142    static const _inkTokens = ThemeTokens(
143      canvas: Color(0xFF1C1A17),
144      surface: Color(0xFF262320),
145      surfaceAlt: Color(0xFF2E2B27),
146      canvasDeep: Color(0xFF151310),
147      textPrimary: Color(0xFFF2EEE6),
148      textSecondary: Color(0xFFA39D94),
149      textTertiary: Color(0xFF6E695F),
150      textInverse: Color(0xFF1C1A17),
151      divider: Color(0xFF3A3630),
152      brand: Color(0xFFD4B57A),
153      brandDeep: Color(0xFFB8985A),
154      brandLight: Color(0xFFE0C68A),
155      brandSubtle: Color(0xFF2E2820),
156      brandText: Color(0xFFD4B57A),
157      danger: Color(0xFFD4706C),
158      dangerSubtle: Color(0xFF2E201E),
159      success: Color(0xFF8FA06A),
160      successSubtle: Color(0xFF22251D),
161      shadowConvex: [
```

```
  162        BoxShadow(color: Color(0xFF13110E), offset: Offset(6, 6),
blurRadius: 14),
  163        BoxShadow(color: Color(0xFF29251F), offset: Offset(-6, -6),
blurRadius: 14),
  164      ],
  165      shadowConvexSubtle: [
  166        BoxShadow(color: Color(0xFF1A1714), offset: Offset(3, 3),
blurRadius: 6),
  167        BoxShadow(color: Color(0xFF2E2B24), offset: Offset(-3, -3),
blurRadius: 6),
  168      ],
  169      shadowConvexBrand: [
  170        BoxShadow(color: Color(0xFF1A1610), offset: Offset(4, 4),
blurRadius: 10),
  171        BoxShadow(color: Color(0xFF3E3624), offset: Offset(-4, -4),
blurRadius: 10),
  172      ],
  173      shadowConcave: [
  174        BoxShadow(color: Color(0xFF141210), offset: Offset(4, 4),
blurRadius: 10, blurStyle: BlurStyle.inner),
  175        BoxShadow(color: Color(0xFF302C25), offset: Offset(-4, -4),
blurRadius: 10, blurStyle: BlurStyle.inner),
  176      ],
  177      shadowConcaveSubtle: [
  178        BoxShadow(color: Color(0xFF1A1714), offset: Offset(2, 2),
blurRadius: 5, blurStyle: BlurStyle.inner),
  179        BoxShadow(color: Color(0xFF2E2B24), offset: Offset(-2, -2),
blurRadius: 5, blurStyle: BlurStyle.inner),
  180      ],
  181      shadowPressed: [
  182        BoxShadow(color: Color(0xFF141210), offset: Offset(3, 3),
blurRadius: 8, blurStyle: BlurStyle.inner),
  183        BoxShadow(color: Color(0xFF302C25), offset: Offset(-3, -3),
blurRadius: 8, blurStyle: BlurStyle.inner),
  184      ],
  185      shadowFloat: [
  186        BoxShadow(color: Color(0x4D000000), offset: Offset(0, 8),
blurRadius: 32),
  187      ],
  188    );
```

第 18 页


```
  189    // === 胶片复古主题 ===
  190    // 来源: App.vue [data-theme="retro"] (line 112-140)
  191    // 注意: retro 主题 App.vue 未显式定义 --color-brand-light，用
brandLight=brand 稍亮派生 (0xFFD4A57A)
  192    static const _retroTokens = ThemeTokens(
  193      canvas: Color(0xFFF5E6D3),
  194      surface: Color(0xFFFFF8F0),
  195      surfaceAlt: Color(0xFFEBDAC4),
  196      canvasDeep: Color(0xFFEBDAC4),
  197      textPrimary: Color(0xFF3D2817),
```

```
  198      textSecondary: Color(0xFF6B4C2F),
  199      textTertiary: Color(0xFF9C8060),
  200      textInverse: Color(0xFFF5E6D3),
  201      divider: Color(0xFFD9C9B3),
  202      brand: Color(0xFFC4956A),
  203      brandDeep: Color(0xFFA67B52),
  204      brandLight: Color(0xFFD4A57A),
  205      brandSubtle: Color(0xFFF0E0C8),
  206      brandText: Color(0xFF8C5A30),
  207      danger: Color(0xFFA04030),
  208      dangerSubtle: Color(0xFFF0D8D0),
  209      success: Color(0xFF6B7B4C),
  210      successSubtle: Color(0xFFE8EDDF),
  211      shadowConvex: [
  212        BoxShadow(color: Color(0xFFCFC0AB), offset: Offset(5, 5),
blurRadius: 12),
  213        BoxShadow(color: Color(0xFFFFFDF7), offset: Offset(-5, -5),
blurRadius: 12),
  214      ],
  215      shadowConvexSubtle: [
  216        BoxShadow(color: Color(0xFFD5C6B0), offset: Offset(3, 3),
blurRadius: 6),
  217        BoxShadow(color: Color(0xFFFFFDF7), offset: Offset(-3, -3),
blurRadius: 6),
  218      ],
  219      shadowConvexBrand: [
  220        BoxShadow(color: Color(0xFFB08560), offset: Offset(4, 4),
blurRadius: 10),
  221        BoxShadow(color: Color(0xFFDAA577), offset: Offset(-4, -4),
blurRadius: 10),
  222      ],
  223      shadowConcave: [
  224        BoxShadow(color: Color(0xFFD0C1AC), offset: Offset(4, 4),
blurRadius: 8, blurStyle: BlurStyle.inner),
  225        BoxShadow(color: Color(0xFFFFFDF7), offset: Offset(-4, -4),
blurRadius: 8, blurStyle: BlurStyle.inner),
  226      ],
  227      shadowConcaveSubtle: [
  228        BoxShadow(color: Color(0xFFD5C6B0), offset: Offset(2, 2),
blurRadius: 5, blurStyle: BlurStyle.inner),
  229        BoxShadow(color: Color(0xFFFFFDF7), offset: Offset(-2, -2),
blurRadius: 5, blurStyle: BlurStyle.inner),
  230      ],
  231      shadowPressed: [
  232        BoxShadow(color: Color(0xFFD0C1AC), offset: Offset(3, 3),
blurRadius: 6, blurStyle: BlurStyle.inner),
  233        BoxShadow(color: Color(0xFFFFFDF7), offset: Offset(-3, -3),
blurRadius: 6, blurStyle: BlurStyle.inner),
  234      ],
  235      shadowFloat: [
  236        BoxShadow(color: Color(0x1A3D2817), offset: Offset(0, 8),
blurRadius: 32),
  237      ],
  238    );
```

```
  239    // === 日系清新主题 ===
  240    // 来源: App.vue [data-theme="fresh"] (line 142-170)
  241    // 注意: fresh 主题 App.vue 未显式定义 --color-brand-light，用
brandLight=brand 稍亮派生 (0xFFA8D8A0)
  242    static const _freshTokens = ThemeTokens(
  243      canvas: Color(0xFFF8FAF6),
  244      surface: Color(0xFFFFFFFF),
  245      surfaceAlt: Color(0xFFEDF2EB),
  246      canvasDeep: Color(0xFFE8EDE5),
  247      textPrimary: Color(0xFF4A3F35),
  248      textSecondary: Color(0xFF8C7F70),
  249      textTertiary: Color(0xFFB8AEA0),
  250      textInverse: Color(0xFFF8FAF6),
  251      divider: Color(0xFFDDE5D8),
  252      brand: Color(0xFF8BAD72),
  253      brandDeep: Color(0xFF6E9458),
  254      brandLight: Color(0xFFA8D8A0),
  255      brandSubtle: Color(0xFFE8F0E2),
   256      brandText: Color(0xFF5E8348),
  257      danger: Color(0xFFC87878),
  258      dangerSubtle: Color(0xFFF5E0E0),
   259      success: Color(0xFF9AAB7C),
  260      successSubtle: Color(0xFFEDF2E8),
```

第 19 页


```
  261      shadowConvex: [
  262        BoxShadow(color: Color(0xFFD4DBD0), offset: Offset(6, 6),
blurRadius: 14),
  263        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-6, -6),
blurRadius: 14),
  264      ],
  265      shadowConvexSubtle: [
  266        BoxShadow(color: Color(0xFFD8DFD4), offset: Offset(3, 3),
blurRadius: 6),
  267        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-3, -3),
blurRadius: 6),
  268      ],
  269      shadowConvexBrand: [
  270        BoxShadow(color: Color(0xFF7A9B62), offset: Offset(4, 4),
blurRadius: 10),
  271        BoxShadow(color: Color(0xFF9CC084), offset: Offset(-4, -4),
blurRadius: 10),
  272      ],
  273      shadowConcave: [
  274        BoxShadow(color: Color(0xFFD6DDD2), offset: Offset(4, 4),
blurRadius: 10, blurStyle: BlurStyle.inner),
  275        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-4, -4),
blurRadius: 10, blurStyle: BlurStyle.inner),
  276      ],
  277      shadowConcaveSubtle: [
  278        BoxShadow(color: Color(0xFFD8DFD4), offset: Offset(2, 2),
blurRadius: 5, blurStyle: BlurStyle.inner),
```

```
  279        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-2, -2),
blurRadius: 5, blurStyle: BlurStyle.inner),
   280      ],
  281      shadowPressed: [
  282        BoxShadow(color: Color(0xFFD6DDD2), offset: Offset(3, 3),
blurRadius: 8, blurStyle: BlurStyle.inner),
  283        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-3, -3),
blurRadius: 8, blurStyle: BlurStyle.inner),
  284      ],
  285      shadowFloat: [
  286        BoxShadow(color: Color(0x144A3F35), offset: Offset(0, 8),
blurRadius: 32),
  287      ],
  288    );
  289    // === 温馨粉主题 ===
  290    // 来源: App.vue [data-theme="cozy"] (line 172-201)
  291    static const _cozyTokens = ThemeTokens(
  292      canvas: Color(0xFFFFF5F5),
  293      surface: Color(0xFFFFFFFF),
  294      surfaceAlt: Color(0xFFFAEDED),
  295      canvasDeep: Color(0xFFF5EAEA),
  296      textPrimary: Color(0xFF4A3A3A),
  297      textSecondary: Color(0xFF8C7070),
  298      textTertiary: Color(0xFFB89A9A),
  299      textInverse: Color(0xFFFFF5F5),
  300      divider: Color(0xFFF0E0E0),
  301      brand: Color(0xFFE8A0A0),
  302      brandDeep: Color(0xFFD4858A),
  303      brandLight: Color(0xFFF0B5B5),
  304      brandSubtle: Color(0xFFFCE8E8),
  305      brandText: Color(0xFFC47070),
  306      danger: Color(0xFFD47070),
  307      dangerSubtle: Color(0xFFFCE8E8),
  308      success: Color(0xFF8FB088),
  309      successSubtle: Color(0xFFEDF2E8),
  310      shadowConvex: [
```

```
  311        BoxShadow(color: Color(0xFFF0E0E0), offset: Offset(6, 6),
blurRadius: 14),
  312        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-6, -6),
blurRadius: 14),
  313      ],
  314      shadowConvexSubtle: [
  315        BoxShadow(color: Color(0xFFF2E2E2), offset: Offset(3, 3),
blurRadius: 6),
  316        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-3, -3),
blurRadius: 6),
  317      ],
  318      shadowConvexBrand: [
  319        BoxShadow(color: Color(0xFFD4858A), offset: Offset(4, 4),
blurRadius: 10),
  320        BoxShadow(color: Color(0xFFF0B5B5), offset: Offset(-4, -4),
blurRadius: 10),
  321      ],
  322      shadowConcave: [
  323        BoxShadow(color: Color(0xFFF0E0E0), offset: Offset(4, 4),
blurRadius: 10, blurStyle: BlurStyle.inner),
  324        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-4, -4),
blurRadius: 10, blurStyle: BlurStyle.inner),
  325      ],
  326      shadowConcaveSubtle: [
  327        BoxShadow(color: Color(0xFFF2E2E2), offset: Offset(2, 2),
blurRadius: 5, blurStyle: BlurStyle.inner),
  328        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-2, -2),
blurRadius: 5, blurStyle: BlurStyle.inner),
   329      ],
  330      shadowPressed: [
  331        BoxShadow(color: Color(0xFFF0E0E0), offset: Offset(3, 3),
blurRadius: 8, blurStyle: BlurStyle.inner),
   332        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-3, -3),
blurRadius: 8, blurStyle: BlurStyle.inner),
   333      ],
  334      shadowFloat: [
   335        BoxShadow(color: Color(0x144A3A3A), offset: Offset(0, 8),
blurRadius: 32),
   336      ],
   337    );
```

第 20 页


```
  338    // === 马卡龙主题 ===
  339    // 来源: App.vue [data-theme="macaron"] (line 203-232)
  340    static const _macaronTokens = ThemeTokens(
  341      canvas: Color(0xFFFFF8F0),
  342      surface: Color(0xFFFFFFFF),
  343      surfaceAlt: Color(0xFFF5F0E8),
  344      canvasDeep: Color(0xFFF0EAE0),
  345      textPrimary: Color(0xFF5A4A4A),
  346      textSecondary: Color(0xFF8C7A7A),
  347      textTertiary: Color(0xFFB8A8A0),
```

```
  348      textInverse: Color(0xFFFFF8F0),
  349      divider: Color(0xFFE8E0D5),
  350      brand: Color(0xFFA8D8C8),
  351      brandDeep: Color(0xFF8CC5B5),
  352      brandLight: Color(0xFFC5E8DD),
  353      brandSubtle: Color(0xFFE0F0EA),
  354      brandText: Color(0xFF5E9882),
  355      danger: Color(0xFFE8A0A0),
  356      dangerSubtle: Color(0xFFFCE8E8),
  357      success: Color(0xFFA8D8C8),
  358      successSubtle: Color(0xFFE0F0EA),
  359      shadowConvex: [
  360        BoxShadow(color: Color(0xFFE8E0D5), offset: Offset(6, 6),
blurRadius: 14),
  361        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-6, -6),
blurRadius: 14),
  362      ],
  363      shadowConvexSubtle: [
  364        BoxShadow(color: Color(0xFFEDE5D8), offset: Offset(3, 3),
blurRadius: 6),
  365        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-3, -3),
blurRadius: 6),
  366      ],
  367      shadowConvexBrand: [
  368        BoxShadow(color: Color(0xFF8CC5B5), offset: Offset(4, 4),
blurRadius: 10),
  369        BoxShadow(color: Color(0xFFC5E8DD), offset: Offset(-4, -4),
blurRadius: 10),
  370      ],
  371      shadowConcave: [
  372        BoxShadow(color: Color(0xFFE8E0D5), offset: Offset(4, 4),
blurRadius: 10, blurStyle: BlurStyle.inner),
  373        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-4, -4),
blurRadius: 10, blurStyle: BlurStyle.inner),
  374      ],
  375      shadowConcaveSubtle: [
  376        BoxShadow(color: Color(0xFFEDE5D8), offset: Offset(2, 2),
blurRadius: 5, blurStyle: BlurStyle.inner),
  377        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-2, -2),
blurRadius: 5, blurStyle: BlurStyle.inner),
  378      ],
  379      shadowPressed: [
  380        BoxShadow(color: Color(0xFFE8E0D5), offset: Offset(3, 3),
blurRadius: 8, blurStyle: BlurStyle.inner),
  381        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-3, -3),
blurRadius: 8, blurStyle: BlurStyle.inner),
  382      ],
  383      shadowFloat: [
  384        BoxShadow(color: Color(0x145A4A4A), offset: Offset(0, 8),
blurRadius: 32),
  385      ],
  386    );
  387    // === 莫兰迪主题 ===
```

```
  388    // 来源: App.vue [data-theme="morandi"] (line 234-263)
  389    static const _morandiTokens = ThemeTokens(
  390      canvas: Color(0xFFE8E4E0),
  391      surface: Color(0xFFF2EFEA),
  392      surfaceAlt: Color(0xFFE0DCD6),
   393      canvasDeep: Color(0xFFDDD9D3),
   394      textPrimary: Color(0xFF4A4540),
   395      textSecondary: Color(0xFF7A7570),
   396      textTertiary: Color(0xFFA8A29C),
   397      textInverse: Color(0xFFE8E4E0),
   398      divider: Color(0xFFD5D0CA),
   399      brand: Color(0xFF8B9DAF),
  400      brandDeep: Color(0xFF6B7D8F),
  401      brandLight: Color(0xFFA8B8C8),
  402      brandSubtle: Color(0xFFD5DDE5),
   403      brandText: Color(0xFF5B6D7F),
  404      danger: Color(0xFFA88080),
  405      dangerSubtle: Color(0xFFE8DDDD),
  406      success: Color(0xFF8FA590),
   407      successSubtle: Color(0xFFDDE5DD),
   408      shadowConvex: [
  409        BoxShadow(color: Color(0xFFD5D0CA), offset: Offset(6, 6),
blurRadius: 14),
  410        BoxShadow(color: Color(0xFFF2EFEA), offset: Offset(-6, -6),
blurRadius: 14),
   411      ],
   412      shadowConvexSubtle: [
   413        BoxShadow(color: Color(0xFFD8D3CD), offset: Offset(3, 3),
blurRadius: 6),
   414        BoxShadow(color: Color(0xFFF2EFEA), offset: Offset(-3, -3),
blurRadius: 6),
   415      ],
   416      shadowConvexBrand: [
   417        BoxShadow(color: Color(0xFF6B7D8F), offset: Offset(4, 4),
blurRadius: 10),
   418        BoxShadow(color: Color(0xFFA8B8C8), offset: Offset(-4, -4),
blurRadius: 10),
   419      ],
   420      shadowConcave: [
   421        BoxShadow(color: Color(0xFFD5D0CA), offset: Offset(4, 4),
blurRadius: 10, blurStyle: BlurStyle.inner),
   422        BoxShadow(color: Color(0xFFF2EFEA), offset: Offset(-4, -4),
blurRadius: 10, blurStyle: BlurStyle.inner),
   423      ],
   424      shadowConcaveSubtle: [
   425        BoxShadow(color: Color(0xFFD8D3CD), offset: Offset(2, 2),
blurRadius: 5, blurStyle: BlurStyle.inner),
   426        BoxShadow(color: Color(0xFFF2EFEA), offset: Offset(-2, -2),
blurRadius: 5, blurStyle: BlurStyle.inner),
   427      ],
   428      shadowPressed: [
   429        BoxShadow(color: Color(0xFFD5D0CA), offset: Offset(3, 3),
blurRadius: 8, blurStyle: BlurStyle.inner),
   430        BoxShadow(color: Color(0xFFF2EFEA), offset: Offset(-3, -3),
blurRadius: 8, blurStyle: BlurStyle.inner),
   431      ],
```

```
   432      shadowFloat: [
   433        BoxShadow(color: Color(0x144A4540), offset: Offset(0, 8),
blurRadius: 32),
   434      ],
   435    );
```

第 21 页

如画 Lumira 摄影辅助应用 V1.0.0 第 21 页

```
  436    // === 玫瑰金主题 ===
  437    // 来源: App.vue [data-theme="rosegold"] (line 265-294)
  438    static const _rosegoldTokens = ThemeTokens(
  439      canvas: Color(0xFFFAF6F2),
  440      surface: Color(0xFFFFFFFF),
  441      surfaceAlt: Color(0xFFF5EDE8),
  442      canvasDeep: Color(0xFFF0E8E2),
  443      textPrimary: Color(0xFF3D2E2A),
  444      textSecondary: Color(0xFF6B5450),
  445      textTertiary: Color(0xFFA89088),
  446      textInverse: Color(0xFFFAF6F2),
  447      divider: Color(0xFFE8DDD5),
  448      brand: Color(0xFFC9A0A0),
  449      brandDeep: Color(0xFFB08585),
  450      brandLight: Color(0xFFDDB8B8),
  451      brandSubtle: Color(0xFFF0E0E0),
  452      brandText: Color(0xFFA06868),
  453      danger: Color(0xFFC47878),
  454      dangerSubtle: Color(0xFFF0E0E0),
   455      success: Color(0xFF9AB088),
  456      successSubtle: Color(0xFFE8F0E0),
  457      shadowConvex: [
  458        BoxShadow(color: Color(0xFFE8DDD5), offset: Offset(6, 6),
blurRadius: 14),
  459        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-6, -6),
blurRadius: 14),
  460      ],
  461      shadowConvexSubtle: [
  462        BoxShadow(color: Color(0xFFEDE2DA), offset: Offset(3, 3),
blurRadius: 6),
  463        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-3, -3),
blurRadius: 6),
  464      ],
  465      shadowConvexBrand: [
  466        BoxShadow(color: Color(0xFFB08585), offset: Offset(4, 4),
blurRadius: 10),
  467        BoxShadow(color: Color(0xFFDDB8B8), offset: Offset(-4, -4),
blurRadius: 10),
   468      ],
   469      shadowConcave: [
   470        BoxShadow(color: Color(0xFFE8DDD5), offset: Offset(4, 4),
blurRadius: 10, blurStyle: BlurStyle.inner),
  471        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-4, -4),
blurRadius: 10, blurStyle: BlurStyle.inner),
  472      ],
```

```
   473      shadowConcaveSubtle: [
   474        BoxShadow(color: Color(0xFFEDE2DA), offset: Offset(2, 2),
blurRadius: 5, blurStyle: BlurStyle.inner),
   475        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-2, -2),
blurRadius: 5, blurStyle: BlurStyle.inner),
   476      ],
   477      shadowPressed: [
   478        BoxShadow(color: Color(0xFFE8DDD5), offset: Offset(3, 3),
blurRadius: 8, blurStyle: BlurStyle.inner),
  479        BoxShadow(color: Color(0xFFFFFFFF), offset: Offset(-3, -3),
blurRadius: 8, blurStyle: BlurStyle.inner),
   480      ],
   481      shadowFloat: [
  482        BoxShadow(color: Color(0x143D2E2A), offset: Offset(0, 8),
blurRadius: 32),
   483      ],
   484    );
   485  }
```

```
// ===== lib/core/theme/app_theme.dart =====
import 'package:flutter/material.dart';
import 'theme_tokens.dart';
class AppThemeData {
  final ThemeTokens tokens;
  final UIStyle style;
  const AppThemeData({
    required this.tokens,
    required this.style,
  });
  /// 卡片圆角（dp）。映射 App.vue --card-radius (rpx ÷ 2 ≈ dp)
  /// neumorphic: 28rpx → 14dp ... 注意: 此处按 brief 原值保留
28/20/28/48，rpx→dp 转换在 Widget 层处理
  double get cardRadius {
    switch (style) {
      case UIStyle.neumorphic:
        return 28;
      case UIStyle.flat:
        return 20;
      case UIStyle.glass:
        return 28;
      case UIStyle.female:
        return 48;
    }
  }
  /// 表面透明度（对应 App.vue --surface-alpha）
```

第 22 页


```
  double get surfaceAlpha {
    switch (style) {
      case UIStyle.neumorphic:
        return 1.0;
      case UIStyle.flat:
        return 1.0;
      case UIStyle.glass:
        return 0.55;
      case UIStyle.female:
        return 0.75;
    }
  }
  /// 卡片边框（对应 App.vue --card-border）
  Border? get cardBorder {
    switch (style) {
      case UIStyle.neumorphic:
        return null;
      case UIStyle.flat:
        return Border.all(color: tokens.divider, width: 1);
      case UIStyle.glass:
        return Border.all(color: const
Color(0xFFFFFFFF).withOpacity(0.3), width: 1);
      case UIStyle.female:
        return null;
```

```
534       }
535     }
536     /// 卡片阴影（对应 App.vue --shadow-convex）
537     List<BoxShadow> get cardShadow {
538       switch (style) {
539         case UIStyle.neumorphic:
540           return tokens.shadowConvex;
541         case UIStyle.flat:
542           return const [];
543         case UIStyle.glass:
544           return const [
545             BoxShadow(color: Color(0x14000000), offset: Offset(0, 8),
blurRadius: 32),
546           ];
547         case UIStyle.female:
548           // App.vue --shadow-convex: 0 8px 32px rgba(brand-rgb, 0.15)
549           return [
550             BoxShadow(
551               color: tokens.brand.withOpacity(0.15),
552               offset: const Offset(0, 8),
553               blurRadius: 32,
554             ),
555           ];
556       }
557     }
558     /// 女性美学专属：多渐变卡片背景（5 层视觉）
559     /// 仅在 style == UIStyle.female 时由 Widget 调用
560     /// 层 1: 线性渐变基底（brand→brandLight，135deg）
```

```
   561    /// 层 2: 径向高光（brandSubtle 中心 70%透明度）
   562    /// 层 3: 表面 75% 透明度（surfaceAlpha）
   563    /// 层 4: 白色 1px 边框（女性美学实际 cardBorder 为 null，但多渐变卡
片本身需细边框强化层次）
   564    /// 层 5: 品牌色 0.15 阴影（cardShadow 已提供）
   565    MultiGradientSpec? get multiGradient {
   566      if (style != UIStyle.female) return null;
   567      return MultiGradientSpec(
   568        linear: LinearGradient(
   569          begin: Alignment.topLeft,
   570          end: Alignment.bottomRight,
   571          colors: [
   572            tokens.brandSubtle.withOpacity(0.75),
   573            tokens.surface.withOpacity(0.75),
   574          ],
   575        ),
   576        radialHighlight: RadialGradient(
   577          center: Alignment.topLeft,
   578          radius: 0.85,
   579          colors: [
   580            tokens.brandLight.withOpacity(0.45),
   581            tokens.brandLight.withOpacity(0.0),
   582          ],
   583        ),
   584        hairlineBorder: Border.all(color: const
Color(0xFFFFFFFF).withOpacity(0.6), width: 0.5),
   585      );
   586    }
   587    ThemeData toThemeData() {
   588      final isLight = tokens.canvas.computeLuminance() > 0.5;
   589      return ThemeData(
   590        brightness: isLight ? Brightness.light : Brightness.dark,
   591        primaryColor: tokens.brand,
   592        scaffoldBackgroundColor: tokens.canvas,
   593        cardColor: tokens.surface,
   594        dividerColor: tokens.divider,
   595        textTheme: TextTheme(
   596          displayLarge: TextStyle(
   597            fontSize: 38,
   598            fontWeight: FontWeight.w600,
   599            color: tokens.textPrimary,
   600            letterSpacing: 0.04,
   601          ),
   602          bodyLarge: TextStyle(fontSize: 30, color:
tokens.textPrimary),
   603          bodyMedium: TextStyle(fontSize: 26, color:
tokens.textSecondary),
   604          bodySmall: TextStyle(fontSize: 22, color:
tokens.textTertiary),
   605        ),
   606      );
   607    }
   608  }
   609  /// 女性美学多渐变卡片规格
   610  class MultiGradientSpec {
```

```
  611    final LinearGradient linear;
  612    final RadialGradient radialHighlight;
  613    final Border hairlineBorder;
  614    const MultiGradientSpec({
  615      required this.linear,
  616      required this.radialHighlight,
  617      required this.hairlineBorder,
  618    });
  619  }
```

模块 7：路由名与观察者（lib/core/router/route_names.dart + route_observers.dart）

第 23 页

如画 Lumira 摄影辅助应用 V1.0.0 第 23 页

```
   1  // ===== lib/core/router/route_names.dart =====
   2  /// 所有路由名与路径常量。
   3  /// 来源：uni-app lumira-app/src/pages.json（34 个页面，1:1 映射）
   4  class RouteNames {
   5    RouteNames._();
   6    // === 路径常量 ===
   7    // 注意：GoRouter 路径以 / 开头，对应 uni-app pages/ 下的文件路径
   8    static const String splash = '/splash';
   9    static const String home = '/home';
  10    static const String templates = '/templates';
  11    static const String challenge = '/challenge';
  12    static const String profile = '/profile';
  13    static const String capture = '/capture';
  14    static const String capturePreview = '/capture/preview';
  15    static const String capturePreviewTemplate = '/capture/preview-
template';
  16    static const String captureSceneGuide = '/capture/scene-guide';
  17    static const String captureSceneManage = '/capture/scene-manage';
  18    static const String captureSceneDetail = '/capture/scene-detail';
  19    static const String templatesDetail = '/templates/detail';
  20    static const String templatesUnlock = '/templates/unlock';
  21    static const String templatesEditor = '/templates/editor';
  22    static const String templatesDrafts = '/templates/drafts';
  23    static const String templatesRecommend = '/templates/recommend';
  24    static const String templatesAll = '/templates/all';
  25    static const String challengeDetail = '/challenge/detail';
  26    static const String inspiration = '/inspiration';
  27    static const String gallery = '/gallery';
  28    static const String galleryDetail = '/gallery/detail';
  29    static const String galleryDiary = '/gallery/diary';
  30    static const String galleryMonthlyDigest = '/gallery/monthly-
digest';
```

```
   31    static const String profileSettings = '/profile/settings';
   32    static const String profileSettingsTheme =
'/profile/settings/theme';
   33    static const String profileGrowth = '/profile/growth';
   34    static const String profileInvite = '/profile/invite';
   35    static const String profileAcademy = '/profile/academy';
   36    static const String profileAcademyDetail = '/profile/academy-
detail';
   37    static const String profileCollections = '/profile/collections';
   38    static const String profileCollectionDetail =
'/profile/collection-detail';
   39    static const String profileMyTemplates = '/profile/my-templates';
   40    static const String profileAbout = '/profile/about';
   41    static const String scenes = '/scenes';
   42    static const String shootkitEditor = '/shootkit/editor';
   43    // === 查询参数键名 ===
   44    static const String paramTemplateId = 'templateId';
   45    static const String paramSceneId = 'sceneId';
   46    static const String paramScene = 'scene';
   47    static const String paramTab = 'tab';
   48    static const String paramChallengeId = 'challengeId';
   49    static const String paramPhotoId = 'photoId';
   50    static const String paramCollectionId = 'collectionId';
```

```
   51    static const String paramAcademyId = 'academyId';
   52    static const String paramMode = 'mode';
   53    static const String paramKitId = 'kitId';
   54    // === 工具方法 ===
   55    /// 构建 templateId 查询参数 URL
   56    /// 用于模板详情/解锁/编辑/拍摄预览页等需要传递 templateId 的场景
   57    static String withTemplateId(String path, String templateId) {
   58      return '$path?$paramTemplateId=$templateId';
   59    }
   60    /// 构建 sceneId 查询参数 URL
   61    static String withSceneId(String path, String sceneId) {
   62      return '$path?$paramSceneId=$sceneId';
   63    }
   64    /// 构建多参数 URL
   65    static String build(String path, Map<String, String> params) {
   66      if (params.isEmpty) return path;
   67      final query = params.entries.map((e) =>
'${e.key}=${e.value}').join('&');
   68      return '$path?$query';
   69    }
   70  }
```

第 24 页


```
   71  // ===== lib/core/router/route_observers.dart =====
   72  import 'package:flutter/material.dart';
   73  import 'package:flutter_riverpod/flutter_riverpod.dart';
   74  import '../../features/capture/data/capture_state.dart';
   75  import 'route_names.dart';
   76  /// 监听路由变化，用于：
   77  /// 1. 页面切换埋点（onPush/onPop/onReplace 记录）
   78  /// 2. 离开拍摄页时清理 template 相关状态（修复 uni-app 的状态残留 bug）
   79  /// 3. 进入特定页面时预热数据
   80  class LumiraRouteObserver extends RouteObserver<PageRoute<dynamic>>
{
   81    LumiraRouteObserver(this._ref);
   82    final Ref _ref;
   83    @override
   84    void didPush(Route<dynamic> route, Route<dynamic>? previousRoute)
{
   85      super.didPush(route, previousRoute);
   86      _handleRouteChange(previousRoute, route);
   87    }
   88    @override
   89    void didPop(Route<dynamic> route, Route<dynamic>? previousRoute) {
   90      super.didPop(route, previousRoute);
   91      _handleRouteChange(route, previousRoute);
   92    }
   93    @override
   94    void didReplace({Route<dynamic>? newRoute, Route<dynamic>?
oldRoute}) {
   95      super.didReplace(newRoute: newRoute, oldRoute: oldRoute);
   96      if (oldRoute != null && newRoute != null) {
```

```
   97        _handleRouteChange(oldRoute, newRoute);
   98      }
   99    }
  100    void _handleRouteChange(Route<dynamic>? from, Route<dynamic>? to)
{
  101      final fromPath = _extractPath(from);
  102      final toPath = _extractPath(to);
  103      // 离开拍摄页（capture/preview/preview-template）时清理模板状态
  104      // 修复 uni-app 中"退出拍摄页后 currentTemplateId 残留影响后续自由拍
摄"的 bug
  105      if (_isCapturePage(fromPath) && !_isCapturePage(toPath)) {
  106        _clearCaptureState();
  107      }
  108    }
  109    String? _extractPath(Route<dynamic>? route) {
  110      if (route is PageRoute) {
  111        final settings = route.settings;
  112        if (settings.name != null) {
  113          return settings.name;
  114        }
  115      }
  116      return null;
  117    }
  118    /// 拍摄页路径与名称集合
  119    /// go_router 6.5.9 在 GoRoute 设置了 name: 时，settings.name 返回
NAME（如 'capture'）
  120    /// 而非 PATH（如 '/capture'）。_isCapturePage 同时匹配两者。
```

```
  121    /// 修复 Minor finding #1。
  122    static const _capturePaths = <String>{
  123      RouteNames.capture,           // '/capture'
  124      RouteNames.capturePreview,    // '/capture/preview'
  125      RouteNames.capturePreviewTemplate, // '/capture/preview-
template'
  126    };
  127    static const _captureNames = <String>{
  128      'capture',
  129      'capturePreview',
  130      'capturePreviewTemplate',
  131    };
  132    bool _isCapturePage(String? pathOrName) {
  133      if (pathOrName == null) return false;
  134      return _capturePaths.contains(pathOrName) ||
_captureNames.contains(pathOrName);
  135    }
  136    void _clearCaptureState() {
  137      // Forced fix: CaptureState.resetAll 接收 ProviderContainer（见
capture_state.dart 注释）
  138      CaptureState.resetAll(_ref.container);
  139    }
  140  }
  141  /// 全局路由观察者 Provider
  142  final lumiraRouteObserverProvider =
Provider<LumiraRouteObserver>((ref) {
  143    return LumiraRouteObserver(ref);
  144  });
```

模块 8：数字格式化工具 + 拍摄页状态（lib/core/utils/number_format.dart + lib/features/capture/data/capture_state.dart）

第 25 页

如画 Lumira 摄影辅助应用 V1.0.0 第 25 页

```
    1  // ===== lib/core/utils/number_format.dart =====
    2  /// 数字千分位格式化（如 1280 → "1,280"）
    3  ///
    4  /// 用于 Profile HeroCard 总作品数、Growth LevelCard 经验值等场景。
    5  /// 来源：uni-app lumira-app/src/pages/profile/index.vue 和
growth.vue 中
    6  /// 的 `_formatNum` 函数（两处实现完全一致，此处提取为共享工具）。
    7  ///
    8  /// 历史修复说明：原 `_formatNum` 在 4+ 位数字上逗号位置错误
    9  /// （如 1280 错误返回 "128,0"，2000 错误返回 "200,0"）。
```

```
  /// 本实现采用正向 StringBuffer + 索引取字符的方式，
  /// 通过 `(str.length - i) % 3 == 0` 在正确位置插入逗号，
  /// 输出 "1,280" / "2,000" 等标准千分位格式，已由单元测试覆盖。
  String formatThousands(int value) {
    final str = value.toString();
    final buffer = StringBuffer();
    for (var i = 0; i < str.length; i++) {
      if (i > 0 && (str.length - i) % 3 == 0) {
        buffer.write(',');
      }
      buffer.write(str[i]);
    }
    return buffer.toString();
  }
  // ===== lib/features/capture/data/capture_state.dart =====
  import 'package:flutter_riverpod/flutter_riverpod.dart';
  /// 闪光灯模式
  enum CaptureFlashMode {
    off,
    on,
    auto,
    torch,
  }
  /// 拍摄页状态 providers
  ///
  /// 修复 uni-app 中"退出拍摄页后 currentTemplateId 残留影响后续自由拍摄"的 bug。
  /// 所有状态在 LumiraRouteObserver.didPop 离开拍摄页时通过 _clearCaptureState 重置。
  class CaptureState {
    CaptureState._();
    /// 当前模板 ID（null = 自由拍摄模式）
    /// 来源：URL query param `?templateId=xxx`
    static final currentTemplateIdProvider = StateProvider<String?>((ref) => null);
    /// 闪光灯模式
    static final flashModeProvider = StateProvider<CaptureFlashMode>((ref) => CaptureFlashMode.off);
    /// 全屏取景器开关
    static final isFullscreenProvider = StateProvider<bool>((ref) => false);
    /// 显示/隐藏模板叠图
    static final showTemplateProvider = StateProvider<bool>((ref) => true);
    /// 显示/隐藏姿势剪影
    static final showSilhouetteProvider = StateProvider<bool>((ref) => true);
    /// 最近拍摄照片路径（点击缩略图跳转预览页）
```

```
   51    static final lastPhotoPathProvider = StateProvider<String?>((ref)
=> null);
   52    /// 当前摄像头方向（front / back）
   53    static final cameraFacingProvider = StateProvider<String>((ref) =>
'back');
   54    /// 重置所有拍摄页状态（在 LumiraRouteObserver._clearCaptureState 中
调用）
   55    ///
   56    /// Forced fix: brief 原签名 `resetAll(Ref ref)` 与测试调用
   57    /// `CaptureState.resetAll(container.read)` 不兼容
（`container.read` 是
   58    /// 函数引用而非 `Ref`）。改为接收 `ProviderContainer`，因为
   59    /// `ProviderContainer.read(provider.notifier).state = ...` 与
`Ref` 行为一致。
   60    /// 调用方 `LumiraRouteObserver._clearCaptureState` 传
`_ref.container`。
   61    static void resetAll(ProviderContainer container) {
   62      container.read(currentTemplateIdProvider.notifier).state = null;
   63      container.read(flashModeProvider.notifier).state =
CaptureFlashMode.off;
   64      container.read(isFullscreenProvider.notifier).state = false;
   65      container.read(showTemplateProvider.notifier).state = true;
   66      container.read(showSilhouetteProvider.notifier).state = true;
   67      container.read(lastPhotoPathProvider.notifier).state = null;
   68      container.read(cameraFacingProvider.notifier).state = 'back';
   69    }
   70  }
```

模块 9：拍摄页

（lib/features/capture/pages/capture_page.dart）

第 26 页


```
    1  import 'package:flutter/material.dart';
    2  import 'package:flutter_riverpod/flutter_riverpod.dart';
    3  import 'package:go_router/go_router.dart';
    4  import '../../../core/router/route_names.dart';
    5  import '../data/capture_state.dart';
    6  import '../widgets/capture_button.dart';
    7  import '../widgets/capture_nav.dart';
    8  import '../widgets/camera_preview.dart';
    9  /// 拍摄页（Phase 2 MVP）
   10  ///
   11  /// 视觉规格来源：lumira-app/src/pages/capture/index.vue
   12  /// 本任务范围：导航栏 + 相机预览 + 拍摄按钮 + 缩略图 + 切换摄像头 + 横竖
屏自适应
```

```
/// 范围外（后续任务）：构图叠图 / 姿势剪影 / 参数面板 / 滤镜选择器 / 场景预设 / Raw 模式 / 水平仪 / 双击
class CapturePage extends ConsumerStatefulWidget {
  const CapturePage({super.key, this.templateId});
  /// 来自 URL ?templateId=xxx，null 表示自由拍摄
  final String? templateId;
  @override
  ConsumerState<CapturePage> createState() => _CapturePageState();
}
class _CapturePageState extends ConsumerState<CapturePage>
    with WidgetsBindingObserver {
  bool _isLandscape = false;
  @override
  void initState() {
    super.initState();
    // 监听系统 metrics 变化（横竖屏切换）
    WidgetsBinding.instance.addObserver(this);
    // 初始化 templateId 状态（来自 URL）
    WidgetsBinding.instance.addPostFrameCallback((_) {
ref.read(CaptureState.currentTemplateIdProvider.notifier).state =
          widget.templateId;
    });
  }
  @override
  void dispose() {
    WidgetsBinding.instance.removeObserver(this);
    super.dispose();
  }
  @override
  void didChangeMetrics() {
    super.didChangeMetrics();
    final size = WidgetsBinding.instance.window.physicalSize;
    final newIsLandscape = size.width > size.height;
    if (newIsLandscape != _isLandscape) {
      setState(() => _isLandscape = newIsLandscape);
    }
  }
  void _onBack() {
    // Forced fix: 使用 canPop 保护，避免从深链接直接进入 capture 时 pop 到空栈退出应用
```

```
51      if (Navigator.of(context).canPop()) {
52        Navigator.of(context).pop();
53      } else {
54        GoRouter.of(context).go(RouteNames.home);
55      }
56    }
57    void _onCapture() {
58      // 实际拍照由 CameraAwesomeBuilder.onMediaTap 处理
59      // 此处仅做 UI 反馈（按钮动画已在 CaptureButton 中处理）
60    }
61    void _onCaptured(String path) {
62      ref.read(CaptureState.lastPhotoPathProvider.notifier).state =
path;
63    }
64    void _switchCamera() {
65      final current = ref.read(CaptureState.cameraFacingProvider);
66      ref.read(CaptureState.cameraFacingProvider.notifier).state =
67          current == 'back' ? 'front' : 'back';
68    }
69    @override
70    Widget build(BuildContext context) {
71      final isFullscreen =
ref.watch(CaptureState.isFullscreenProvider);
72      final lastPhoto = ref.watch(CaptureState.lastPhotoPathProvider);
73      return Scaffold(
74        backgroundColor: Colors.black,
75        body: Stack(
76          fit: StackFit.expand,
77          children: [
78            // 相机预览（全屏）
79            CameraPreview(onCaptured: _onCaptured),
80            // 顶部导航栏（全屏模式下隐藏）
81            if (!isFullscreen)
82              Positioned(
83                top: 0,
84                left: 0,
85                right: 0,
86                child: CaptureNav(onBack: _onBack),
87              ),
88            // 底部操作栏（拍摄按钮 + 缩略图 + 切换摄像头）
89            if (!isFullscreen)
90              Positioned(
91                left: 0,
92                right: 0,
93                bottom: 0,
94                child: _BottomBar(
95                  isLandscape: _isLandscape,
96                  lastPhotoPath: lastPhoto,
97                  onCapture: _onCapture,
98                  onSwitchCamera: _switchCamera,
99                  onThumbnailTap: (path) {
100                   GoRouter.of(context).push(
```

```
  101
'${RouteNames.capturePreview}?${RouteNames.paramPhotoId}=$path',
  102                     );
  103                   },
  104                 ),
  105               ),
  106           ],
  107         ),
  108       ),
  109     );
  110     }
  111   }
```

第 27 页


```
   1  /// 底部操作栏
   2  class _BottomBar extends StatelessWidget {
   3    const _BottomBar({
   4      required this.isLandscape,
   5      required this.lastPhotoPath,
   6      required this.onCapture,
   7      required this.onSwitchCamera,
   8      required this.onThumbnailTap,
   9    });
  10    final bool isLandscape;
  11    final String? lastPhotoPath;
  12    final VoidCallback onCapture;
  13    final VoidCallback onSwitchCamera;
  14    final void Function(String path) onThumbnailTap;
  15    @override
  16    Widget build(BuildContext context) {
  17      return SafeArea(
  18        top: false,
  19        child: Container(
  20          decoration: BoxDecoration(
  21            gradient: LinearGradient(
  22              begin: Alignment.topCenter,
  23              end: Alignment.bottomCenter,
  24              colors: [
  25                Colors.transparent,
  26                Colors.black.withOpacity(0.6),
  27              ],
  28            ),
  29          ),
  30          padding: const EdgeInsets.symmetric(horizontal: 24, vertical:
16),
  31          child: Row(
  32            crossAxisAlignment: CrossAxisAlignment.center,
  33            children: [
  34              // 缩略图（左）
  35              GestureDetector(
  36                onTap: lastPhotoPath != null
  37                    ? () => onThumbnailTap(lastPhotoPath!)
```

```
38                     : null,
39                 child: Container(
40                   width: 48,
41                   height: 48,
42                   decoration: BoxDecoration(
43                     color: Colors.white.withOpacity(0.1),
44                     borderRadius: BorderRadius.circular(8),
45                     border: Border.all(
46                       color: Colors.white.withOpacity(0.3),
47                       width: 1.5,
48                     ),
49                   ),
50                   child: lastPhotoPath != null
```

```
                    ? ClipRRect(
                        borderRadius: BorderRadius.circular(6.75),
                        child: Image.network(
                          lastPhotoPath!,
                          fit: BoxFit.cover,
                          errorBuilder: (_, __, ___) => const Icon(
                            Icons.photo,
                            color: Colors.white54,
                            size: 24,
                          ),
                        ),
                      )
                    : const Icon(
                        Icons.photo_camera,
                        color: Colors.white54,
                        size: 24,
                      ),
              ),
            ),
            const Spacer(),
```

第 28 页


```
            // 拍摄按钮（中）
            CaptureButton(onTap: onCapture),
            const Spacer(),
            // 切换摄像头（右）
            GestureDetector(
              onTap: onSwitchCamera,
              child: Container(
                width: 48,
                height: 48,
                decoration: BoxDecoration(
                  color: Colors.white.withOpacity(0.1),
                  shape: BoxShape.circle,
                  border: Border.all(
                    color: Colors.white.withOpacity(0.3),
                    width: 1.5,
                  ),
                ),
                child: const Icon(
                  Icons.cameraswitch_outlined,
                  color: Colors.white,
                  size: 24,
                ),
              ),
            ),
          ],
        ),
      ),
    );
    }
}
```

```
// ===== lib/features/capture/widgets/camera_preview.dart =====
   31  import 'package:camerawesome_ohos/camerawesome_plugin.dart';
   32  import 'package:flutter/material.dart';
   33  import 'package:flutter_riverpod/flutter_riverpod.dart';
   34  import '../data/capture_state.dart';
   35  /// 相机预览组件
   36  ///
   37  /// 使用 camerawesome_ohos 1.0.2（CPF-Flutter Harmony 适配版本，对应原
camerawesome 1.4.0）实现。
   38  /// 在真实设备上渲染相机预览；在 widget 测试中通过
cameraPreviewOverrideProvider 覆盖为占位 widget。
   39  ///
   40  /// 视觉规格来源：lumira-app/src/pages/capture/index.vue line 44-68
   41  ///
   42  /// Forced fix: brief 原代码使用 camerawesome 1.5+ 的 API
（sensorConfig / SingleCaptureRequest /
   43  /// CaptureRequestType / mediaCapture.capture(onSuccess:, onImage:)
/ builder:），与已锁定的
   44  /// camerawesome 1.4.0 实际 API 不匹配。1.4.0 的 `.awesome()` 工厂仅接
受 `sensor:` 和 `flashMode:`
   45  /// 两个独立参数；`SaveConfig.photo(pathBuilder:)` 的 pathBuilder 返回
`Future<String>`；
   46  /// `onMediaTap` 在用户点击已保存媒体缩略图时触发，参数为 `MediaCapture`
数据类（无 capture 方法）。
   47  ///
   48  /// Harmony 适配：pub.dev 上的 camerawesome 无 ohos 实现，平台通道无人
响应会导致取景器一直转圈。
   49  /// 改用 CPF-Flutter fork 的 camerawesome_ohos 包（gitcode.com/CPF-
Flutter/fluttertpc_camerawesome）。
```

```
class CameraPreview extends ConsumerWidget {
  const CameraPreview({super.key, required this.onCaptured});
  /// 拍照完成回调（传入文件路径）
  final void Function(String path) onCaptured;
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final overrideWidget = ref.watch(cameraPreviewOverrideProvider);
    if (overrideWidget != null) {
      return overrideWidget;
    }
    final flashMode = ref.watch(CaptureState.flashModeProvider);
    final facing = ref.watch(CaptureState.cameraFacingProvider);
    return CameraAwesomeBuilder.awesome(
      saveConfig: SaveConfig.photo(
        pathBuilder: () async {
          // 1.4.0: pathBuilder 返回 Future<String>（路径），由调用方决定保存位置
          // 实际生产中应使用 path_provider 生成时间戳路径；MVP 阶段先返回空字符串占位
          return '';
        },
      ),
      sensor: facing == 'front' ? Sensors.front : Sensors.back,
```

模块 10 续：相机预览组件

（lib/features/capture/widgets/camera_preview.dart）

第 29 页

如画 Lumira 摄影辅助应用 V1.0.0 第 29 页

```
        flashMode: _mapFlashMode(flashMode),
      onMediaTap: (mediaCapture) {
        // 1.4.0: onMediaTap 在用户点击已保存媒体的缩略图时触发
        // mediaCapture 是数据类，包含 filePath 与 status（无 capture 方法）
        if (mediaCapture.status == MediaCaptureStatus.success &&
            mediaCapture.filePath.isNotEmpty) {
          onCaptured(mediaCapture.filePath);
        }
      },
    );
  }
  FlashMode _mapFlashMode(CaptureFlashMode mode) {
    switch (mode) {
      case CaptureFlashMode.off:
        return FlashMode.none;
      case CaptureFlashMode.on:
        return FlashMode.always;
```

```
        case CaptureFlashMode.auto:
          return FlashMode.auto;
        case CaptureFlashMode.torch:
          return FlashMode.always;
      }
    }
  }
  /// 测试覆写 provider（生产环境为 null，测试中注入占位 widget）
  final cameraPreviewOverrideProvider = Provider<Widget?>((ref) =>
null);
// ===== lib/features/splash/pages/splash_page.dart =====
  import 'dart:async';
  import 'package:flutter/material.dart';
  import 'package:flutter_riverpod/flutter_riverpod.dart';
  import 'package:go_router/go_router.dart';
  import '../../../core/router/route_names.dart';
  import '../../../core/theme/theme_controller.dart';
  import '../../../shared/widgets/common/fade_up.dart';
  /// Splash 启动页
  ///
  /// 视觉规格来源：lumira-app/src/pages/splash/index.vue
  /// - 居中布局：logo + 标题 + 副标题
  /// - logo 120rpx→60dp aperture 图标，brand 色
  /// - 标题 48rpx→24dp Noto Serif SC，600 字重
  /// - 副标题 26rpx→13dp，textTertiary 色，letter-spacing 0.04em
  /// - 1.8s 后 uni.reLaunch → home（Flutter 用 context.go 替换栈顶）
  class SplashPage extends ConsumerStatefulWidget {
    const SplashPage({super.key});
    @override
    ConsumerState<SplashPage> createState() => _SplashPageState();
  }
  class _SplashPageState extends ConsumerState<SplashPage> {
    static const Duration _redirectDelay = Duration(milliseconds:
1800);
    Timer? _redirectTimer;
```

```
   50    @override
   51    void initState() {
   52      super.initState();
   53      // 1.8s 后跳转 home（用 context.go 替换路由栈，对齐 uni-app 的
reLaunch 语义）
   54      _redirectTimer = Timer(_redirectDelay, () {
   55        if (mounted) {
   56          context.go(RouteNames.home);
   57        }
   58      });
   59    }
```

模块 11 续：Splash 启动页

（lib/features/splash/pages/splash_page.dart）

第 30 页

如画 Lumira 摄影辅助应用 V1.0.0 第 30 页

```
    1    @override
    2    void dispose() {
    3      _redirectTimer?.cancel();
    4      super.dispose();
    5    }
    6    @override
    7    Widget build(BuildContext context) {
    8      final tokens = ref.watch(appThemeProvider).tokens;
    9      return Scaffold(
   10        // #FAF7F2 默认主题 canvas；用 tokens.canvas 让所有主题都对齐
   11        backgroundColor: tokens.canvas,
   12        body: SafeArea(
   13          child: Center(
   14            child: Column(
   15              mainAxisAlignment: MainAxisAlignment.center,
   16              crossAxisAlignment: CrossAxisAlignment.center,
   17              children: [
   18                // logo
   19                FadeUp(
   20                  child: Icon(
   21                    Icons.camera_outlined, // ph-aperture →
Icons.camera_outlined
   22                    size: 60, // 120rpx → 60dp
   23                    color: tokens.brand,
   24                  ),
   25                ),
   26                const SizedBox(height: 24), // margin-bottom 48rpx →
24dp
   27                // 文字组
   28                FadeUp(
```

```
                delay: const Duration(milliseconds: 200),
                child: Column(
                  mainAxisSize: MainAxisSize.min,
                  children: [
                    Text(
                      '如画 Lumira',
                      style: TextStyle(
                        fontSize: 24, // 48rpx → 24dp
                        fontWeight: FontWeight.w600,
                        color: tokens.textPrimary,
                        letterSpacing: -0.01 * 24,
                        height: 1.3,
                      ),
                    ),
                    const SizedBox(height: 6), // gap 12rpx → 6dp
                    Text(
                      '如你所见，皆成画卷',
                      style: TextStyle(
                        fontSize: 13, // 26rpx → 13dp
                        color: tokens.textTertiary,
                        letterSpacing: 0.04 * 13,
                        height: 1.3,
```

```
                      ),
                    ),
                  ],
                ),
              ),
            ],
          ),
        ),
      ),
    ),
  );
  }
}
```

模块 12：模板页

（lib/features/templates/pages/templates_page.dart）

第 31 页


```
// ===== lib/features/templates/pages/templates_page.dart =====
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import '../../../core/router/route_names.dart';
import '../../../core/theme/theme_controller.dart';
import '../../../core/theme/theme_tokens.dart';
import '../../../shared/widgets/common/fade_up.dart';
import '../../../shared/widgets/common/glass_background.dart';
import '../../../shared/widgets/nav/lumira_nav.dart';
import '../../../shared/widgets/tabbar/floating_tabbar.dart';
import '../data/templates_mock_data.dart';
import '../widgets/recommendation_card.dart';
import '../widgets/template_grid_card.dart';
import '../widgets/user_preference_card.dart';
/// 模板页（Tab 页，FloatingTabBar active="templates"）
///
/// 视觉规格来源：lumira-app/src/pages/templates/index.vue
/// 页面结构：
/// 1. LumiraNav（无返回，右侧"查看全部"按钮跳 /templates/all）
/// 2. Hero 推荐区（横向滚动卡片列表）
/// 3. 拍摄偏好（仅 totalPhotos > 0 显示）
/// 4. 更多模板（2 列网格）
/// 5. FloatingTabBar
class TemplatesPage extends ConsumerStatefulWidget {
  const TemplatesPage({super.key});
  @override
```

```
   28    ConsumerState<TemplatesPage> createState() =>
_TemplatesPageState();
   29  }
   30  class _TemplatesPageState extends ConsumerState<TemplatesPage> {
   31    late ScrollController _scrollController;
   32    bool _scrolled = false;
   33    @override
   34    void initState() {
   35      super.initState();
   36      _scrollController = ScrollController();
   37      _scrollController.addListener(_onScroll);
   38    }
   39    @override
   40    void dispose() {
   41      _scrollController.removeListener(_onScroll);
   42      _scrollController.dispose();
   43      super.dispose();
   44    }
   45    void _onScroll() {
   46      final scrolled = _scrollController.offset > 8;
   47      if (scrolled != _scrolled) {
   48        setState(() => _scrolled = scrolled);
   49      }
   50    }
```

```
   51    void _goAll() {
   52      GoRouter.of(context).push(RouteNames.templatesAll);
   53    }
   54    void _goDetail(String templateId) {
   55      GoRouter.of(context).push(
   56        RouteNames.withTemplateId(RouteNames.templatesDetail,
templateId),
   57      );
   58    }
   59    @override
   60    Widget build(BuildContext context) {
   61      final appTheme = ref.watch(appThemeProvider);
   62      final tokens = appTheme.tokens;
   63      return Scaffold(
   64        backgroundColor: tokens.canvas,
   65        body: Stack(
   66          children: [
   67            // 背景装饰（glass 风格可见性）
   68            _BackgroundDecoration(tokens: tokens),
   69            // Forced fix: glass 风格彩色斑点背景
   70            const Positioned.fill(child: GlassBackground(variant:
GlassBackgroundVariant.templates)),
   71            // 主内容
   72            SafeArea(
   73              child: Column(
   74                children: [
   75                  LumiraNav(
   76                    title: '模板',
   77                    transparent: true,
   78                    scrolled: _scrolled,
   79                    showBackButton: false,
   80                    actions: [
   81                      IconButton(
   82                        icon: const Icon(Icons.apps_outlined, size:
20),
   83                        onPressed: _goAll,
   84                        tooltip: '查看全部',
   85                      ),
   86                    ],
   87                  ),
   88                  Expanded(
   89                    child: ListView(
   90                      controller: _scrollController,
   91                      padding: EdgeInsets.zero,
   92                      children: [
   93                        _HeroSection(
   94                          onTap: _goDetail,
   95                        ),
   96                        if
(TemplatesMockData.userPreference.totalPhotos > 0)
   97                          FadeUp(
   98                            delay: const Duration(milliseconds: 80),
   99                            child: _PreferenceSection(),
  100                          ),
```

```
                        FadeUp(
                          delay: const Duration(milliseconds: 160),
                          child: _OtherSection(onTap: _goDetail),
                        ),
                        const SizedBox(height: 140), // bottom spacer
避开 FloatingTabBar
                      ],
                    ),
                  ),
                ],
              ),
            ),
            // FloatingTabBar
            const Positioned(
              left: 0,
              right: 0,
              bottom: 0,
              child: FloatingTabBar(active: 'templates'),
            ),
          ],
        ),
      );
    }
  }
  /// 背景径向渐变装饰（glass 风格 backdrop-filter 可见性）
  class _BackgroundDecoration extends StatelessWidget {
    const _BackgroundDecoration({required this.tokens});
    final ThemeTokens tokens;
    @override
    Widget build(BuildContext context) {
      return Positioned.fill(
       child: IgnorePointer(
         child: Container(
           decoration: BoxDecoration(
             gradient: RadialGradient(
               center: const Alignment(-0.6, -0.8),
               radius: 1.4,
               colors: [
                 tokens.brandSubtle.withOpacity(0.45),
                 tokens.canvas.withOpacity(0),
               ],
             ),
           ),
         ),
       ),
      );
  }
}
/// Hero 推荐区
class _HeroSection extends StatelessWidget {
  const _HeroSection({required this.onTap});
```

```
  final void Function(String templateId) onTap;
  @override
  Widget build(BuildContext context) {
    return FadeUp(
      child: Padding(
        padding: const EdgeInsets.fromLTRB(0, 12, 0, 16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Padding(
              padding: EdgeInsets.fromLTRB(20, 0, 20, 10),
              child: Text(
                '今日为你推荐',
                style: TextStyle(
                  fontSize: 20,
                  fontWeight: FontWeight.w700,
                  letterSpacing: -0.01 * 20,
                  height: 1.2,
                ),
              ),
            ),
            SizedBox(
              height: 244,
              child: ListView.separated(
                scrollDirection: Axis.horizontal,
                padding: const EdgeInsets.symmetric(horizontal: 20),
                itemCount: TemplatesMockData.recommendations.length,
                separatorBuilder: (_, __) => const SizedBox(width: 10),
                itemBuilder: (_, index) {
                  final rec = TemplatesMockData.recommendations[index];
                  return RecommendationCard(
                    recommendation: rec,
                    onTap: () => onTap(rec.id),
                  );
                },
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

第 32 页


```
131  /// 拍摄偏好 section
132  class _PreferenceSection extends StatelessWidget {
133    @override
134    Widget build(BuildContext context) {
135      return Consumer(
```

```
136        builder: (context, ref, _) {
137          final tokens = ref.watch(appThemeProvider).tokens;
138          return Padding(
139            padding: const EdgeInsets.fromLTRB(20, 8, 20, 12),
140            child: Column(
141              crossAxisAlignment: CrossAxisAlignment.start,
142              children: [
143                Padding(
144                  padding: const EdgeInsets.only(bottom: 8),
145                  child: Text(
146                    '你的拍摄偏好',
147                    style: TextStyle(
148                      fontSize: 16,
149                      fontWeight: FontWeight.w600,
150                      color: tokens.textPrimary,
151                      letterSpacing: -0.01 * 16,
152                      height: 1.2,
153                    ),
154                  ),
155                ),
156                const UserPreferenceCard(
157                  preference: TemplatesMockData.userPreference,
158                ),
159              ],
160            ),
161          );
162        },
163      );
164    }
165  }
166  /// 更多模板 section
167  class _OtherSection extends StatelessWidget {
168    const _OtherSection({required this.onTap});
169    final void Function(String templateId) onTap;
170    @override
171    Widget build(BuildContext context) {
172      return Consumer(
173        builder: (context, ref, _) {
174          final tokens = ref.watch(appThemeProvider).tokens;
175          const others = TemplatesMockData.otherTemplates;
176          return Padding(
177            padding: const EdgeInsets.fromLTRB(20, 8, 20, 0),
178            child: Column(
179              crossAxisAlignment: CrossAxisAlignment.start,
180              children: [
```

```
  181                 Padding(
  182                   padding: const EdgeInsets.only(bottom: 8),
  183                   child: Row(
  184                     mainAxisAlignment: MainAxisAlignment.spaceBetween,
  185                     children: [
  186                     Text(
  187                       '更多模板',
  188                       style: TextStyle(
  189                         fontSize: 16,
  190                         fontWeight: FontWeight.w600,
  191                         color: tokens.textPrimary,
  192                         letterSpacing: -0.01 * 16,
  193                         height: 1.2,
  194                       ),
  195                     ),
  196                     GestureDetector(
  197                       onTap: () =>
  198 
GoRouter.of(context).push(RouteNames.templatesAll),
  199                       child: Text(
  200                         '查看全部 ›',
  201                         style: TextStyle(
  202                           fontSize: 12,
  203                           fontWeight: FontWeight.w500,
  204                           color: tokens.brand,
  205                         ),
  206                       ),
  207                     ),
  208                   ],
  209                 ),
  210               ),
  211               if (others.isEmpty)
  212                 _EmptyState(tokens: tokens)
  213               else
  214                 GridView.builder(
  215                   shrinkWrap: true,
  216                   physics: const NeverScrollableScrollPhysics(),
  217                   gridDelegate: const
SliverGridDelegateWithFixedCrossAxisCount(
  218                     crossAxisCount: 2,
  219                     mainAxisSpacing: 12,
  220                     crossAxisSpacing: 12,
  221                     childAspectRatio: 0.70,
  222                   ),
  223                   itemCount: others.length > 6 ? 6 : others.length,
  224                   itemBuilder: (_, index) {
  225                     final tpl = others[index];
  226                     return TemplateGridCard(
  227                       template: tpl,
  228                       onTap: () => onTap(tpl.id),
  229                     );
  230                   },
```

```
231                ),
232              ],
233            ),
234          );
235        },
236      );
237  }
238  }
239  /// 空状态
240  class _EmptyState extends StatelessWidget {
241    const _EmptyState({required this.tokens});
242    final ThemeTokens tokens;
243    @override
244    Widget build(BuildContext context) {
245      return Container(
246        padding: const EdgeInsets.symmetric(vertical: 32, horizontal:
20),
247        child: Column(
248          children: [
249            Icon(
250              Icons.folder_open_outlined,
251              size: 40,
252              color: tokens.textTertiary.withOpacity(0.4),
253            ),
254            const SizedBox(height: 8),
255            Text(
256              '暂无更多模板',
257              style: TextStyle(
258                fontSize: 13,
259                color: tokens.textTertiary,
260              ),
261            ),
262          ],
263        ),
264      );
265    }
266  }
```

第 33 页


```
1  // ===== lib/features/templates/data/templates_mock_data.dart =====
2  /// 模板推荐来源类型
3  enum TemplateSource {
4    /// 最近使用
5    recentUsed,
6    /// 场景匹配
7    sceneMatch,
8    /// 同分类
9    categoryMatch,
10   /// 系统精选
11   systemPick,
```

```
}
/// 模板推荐项（Hero 推荐区）
class TemplateRecommendation {
  const TemplateRecommendation({
    required this.id,
    required this.name,
    required this.reason,
    required this.source,
    required this.imageSeed,
    required this.category,
  });
  final String id;
  final String name;
  final String reason;
  final TemplateSource source;
  final String imageSeed; // picsum seed
  final String category; // 'portrait' / 'landscape' / ... 原始值
}
/// 模板卡片项（更多模板 section）
class TemplateItem {
  const TemplateItem({
    required this.id,
    required this.name,
    required this.category,
    required this.imageSeed,
    required this.price,
  });
  final String id;
  final String name;
  final String category;
  final String imageSeed;
  final int price; // 0 = 免费
}
/// 用户拍摄偏好（仅有照片时显示）
class UserPreference {
  const UserPreference({
    required this.totalPhotos,
    required this.topCategory,
    required this.topCategoryPercentage,
```

```
  51    });
  52    final int totalPhotos;
  53    final String topCategory; // 'portrait' / ...
  54    final int topCategoryPercentage;
   55  }
   56  /// 模板页 mock 数据
   57  /// 来源：lumira-app/src/pages/templates/index.vue 的 recommendations
/ otherTemplates / userPreference
   58  /// 以及 lumira-app/src/composables/useRecommendation.ts 的 mock 示例
   59  class TemplatesMockData {
   60    TemplatesMockData._();
   61    /// 今日推荐（4 项，不同来源）
   62    static const List<TemplateRecommendation> recommendations = [
   63      TemplateRecommendation(
   64        id: 'soft_portrait',
   65        name: '柔光人像',
   66        reason: '你最近经常使用此模板（5 次）',
   67        source: TemplateSource.recentUsed,
   68        imageSeed: 'tpl-soft-portrait',
   69        category: 'portrait',
   70      ),
   71      TemplateRecommendation(
   72        id: 'golden_landscape',
   73        name: '金色风光',
   74        reason: '匹配当前日落场景',
   75        source: TemplateSource.sceneMatch,
   76        imageSeed: 'tpl-golden-landscape',
   77        category: 'landscape',
   78      ),
   79      TemplateRecommendation(
   80        id: 'food_flat_lay',
   81        name: '美食俯拍',
   82        reason: '同分类热门模板',
   83        source: TemplateSource.categoryMatch,
   84        imageSeed: 'tpl-food-flat-lay',
   85        category: 'food',
   86      ),
   87      TemplateRecommendation(
   88        id: 'night_cityscape',
   89        name: '夜景城市',
   90        reason: '系统精选：适合当前时段',
   91        source: TemplateSource.systemPick,
   92        imageSeed: 'tpl-night-cityscape',
   93        category: 'night',
   94      ),
   95    ];
   96    /// 更多模板（6 项，部分免费）
   97    static const List<TemplateItem> otherTemplates = [
   98      TemplateItem(
   99        id: 'street_bw',
  100        name: '街拍黑白',
```

```
  101         category: 'street',
  102         imageSeed: 'tpl-street-bw',
  103         price: 0,
  104       ),
  105       TemplateItem(
  106         id: 'macro_flower',
  107         name: '微距花卉',
  108         category: 'macro',
  109         imageSeed: 'tpl-macro-flower',
  110         price: 0,
  111       ),
  112     ];
  113   }
```

第 34 页


```
  114       // 用户拍摄偏好（模拟数据，totalPhotos > 0 触发显示）
  115       static const UserPreference userPreference = UserPreference(
  116         totalPhotos: 24,
  117         topCategory: 'portrait',
  118         topCategoryPercentage: 42,
  119       );
  120       /// 分类中文标签
  121       /// 来源：lumira-app/src/pages/templates/index.vue 的
categoryLabelMap
  122       static String categoryLabel(String category) {
  123         const map = {
  124           'portrait': '人像',
  125           'landscape': '风光',
  126           'food': '美食',
  127           'night': '夜景',
  128           'street': '街拍',
  129           'macro': '微距',
  130           'still-life': '静物',
  131         };
  132         return map[category] ?? category;
  133       }
  134       /// 推荐来源中文标签
  135       /// 来源：lumira-app/src/pages/templates/index.vue 的 sourceLabel
  136       static String sourceLabel(TemplateSource source) {
  137         switch (source) {
  138           case TemplateSource.recentUsed:
  139             return '最近使用';
  140           case TemplateSource.sceneMatch:
  141             return '场景匹配';
  142           case TemplateSource.categoryMatch:
  143             return '同分类';
  144           case TemplateSource.systemPick:
  145             return '系统精选';
  146         }
```

```
    }
}
// ===== lib/features/home/pages/home_page.dart =====
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import '../../../core/router/route_names.dart';
import '../../../core/theme/theme_controller.dart';
import '../../../core/theme/theme_tokens.dart';
import '../../../shared/widgets/common/fade_up.dart';
import '../../../shared/widgets/common/glass_background.dart';
import '../../../shared/widgets/nav/lumira_nav.dart';
import '../../../shared/widgets/tabbar/floating_tabbar.dart';
import '../data/home_mock_data.dart';
import '../widgets/hero_card.dart';
import '../widgets/quick_actions.dart';
import '../widgets/recent_shot_card.dart';
```

```
import '../widgets/scene_reco_card.dart';
import '../widgets/stats_card.dart';
import '../widgets/streak_card.dart';
import '../widgets/tip_card.dart';
/// 首页
/// 视觉规格来源：lumira-app/src/pages/home/index.vue
/// 8 个 section:
/// 1. LumiraNav（带位置 + 通知/二维码 actions）
/// 2. HeroCard（今日灵感）
/// 3. QuickActions（4 快捷入口）
/// 4. StreakCard（连续打卡）
/// 5. TipCard（今日拍摄小贴士）
/// 6. SceneRecoCard 网格（场景推荐，2 列）
/// 7. RecentShotCard 网格（最近拍摄，2 列）
/// 8. StatsCard（统计）
/// 改进点（vs uni-app）：
/// - 用 ScrollController + listener 替代 window scroll 监听，更可靠
/// - 径向渐变背景装饰用 Container + BoxDecoration 实现（glass 风格可见性）
/// - Section 入场动画用 FadeUp（统一组件，可复用）
class HomePage extends ConsumerStatefulWidget {
  const HomePage({super.key});
  @override
  ConsumerState<HomePage> createState() => _HomePageState();
}
class _HomePageState extends ConsumerState<HomePage> {
  final ScrollController _scrollController = ScrollController();
  bool _scrolled = false;
  static const double _scrollThreshold = 10.0; // uni-app 用 20px → 10dp
  @override
  void initState() {
    super.initState();
    _scrollController.addListener(_onScroll);
  }
  @override
  void dispose() {
    _scrollController.removeListener(_onScroll);
    _scrollController.dispose();
    super.dispose();
  }
  void _onScroll() {
    final offset = _scrollController.offset;
    final newScrolled = offset > _scrollThreshold;
    if (newScrolled != _scrolled) {
      setState(() => _scrolled = newScrolled);
    }
  }
  void _goCapture() => GoRouter.of(context).push(RouteNames.capture);
  void _goTemplates() => context.go(RouteNames.templates);
  void _goChallenge() => context.go(RouteNames.challenge);
  void _goInspiration() => GoRouter.of(context).push(RouteNames.inspiration);
```

```
  214    void _goGallery() => GoRouter.of(context).push(RouteNames.gallery);
  215    void _goSceneGuide(String sceneId) {
  216      // 项目记忆规则：场景推荐卡片跳转 scene-guide?scene=xxx（不是 scene-
detail）
  217      // Forced fix: 改为 push 而非 context.go，避免返回时跳到其他页面或退
出应用
  218      GoRouter.of(context).push(
  219        RouteNames.build(
  220          RouteNames.captureSceneGuide,
  221          {RouteNames.paramScene: sceneId},
  222        ),
  223      );
  224    }
  225    @override
  226    Widget build(BuildContext context) {
  227      final appTheme = ref.watch(appThemeProvider);
  228      final tokens = appTheme.tokens;
  229      return Scaffold(
  230        backgroundColor: tokens.canvas,
  231        // 透明 LumiraNav 作为 appBar（PreferredSizeWidget）
  232        appBar: LumiraNav(
  233          title: '如画',
  234          transparent: true,
  235          scrolled: _scrolled,
  236          leading: _NavLocation(tokens: tokens),
  237          actions: [
  238            _NavAction(
  239              icon: Icons.notifications_outlined,
  240              tokens: tokens,
  241              onTap: () {}, // 占位：通知中心
  242            ),
  243            _NavAction(
  244              icon: Icons.qr_code_outlined,
  245              tokens: tokens,
  246              onTap: () {}, // 占位：扫一扫
  247            ),
  248          ],
  249        ),
  250        body: Stack(
  251          children: [
  252            // 1. 径向渐变背景装饰（glass 风格 backdrop-filter 可见性必
需）
  253            _BackgroundDecoration(tokens: tokens),
  254            // Forced fix: glass 风格彩色斑点背景（让毛玻璃效果可见）
  255            const Positioned.fill(child: GlassBackground()),
  256            // 2. 主内容层（可滚动）
  257            SafeArea(
  258              child: ListView(
  259                controller: _scrollController,
  260                padding: const EdgeInsets.only(bottom: 100), // 给
FloatingTabBar 留空间
  261                children: [
  262                  // Section 1: Hero
  263                  FadeUp(
```

```
  264                   child: HeroCard(onCapture: _goCapture),
  265                 ),
  266                 const SizedBox(height: 20), // section margin-bottom
40rpx → 20dp
  267                 // Section 2: QuickActions
  268                 FadeUp(
  269                   delay: const Duration(milliseconds: 100),
  270                   child: QuickActions(
  271                     onCapture: _goCapture,
  272                     onTemplates: _goTemplates,
  273                     onInspiration: _goInspiration,
  274                     onGallery: _goGallery,
  275                   ),
  276                 ),
  277                 const SizedBox(height: 20),
  278               ],
  279             ),
  280           ),
  281         ],
  282       ),
  283     );
  284   }
  285 }
```

第 35 页


```
   1 // ===== lib/features/gallery/pages/gallery_page.dart =====
   2 import 'package:flutter/material.dart';
   3 import 'package:flutter_riverpod/flutter_riverpod.dart';
   4 import 'package:go_router/go_router.dart';
   5 import '../../../core/db/dao/gallery_dao.dart';
   6 import '../../../core/db/database_provider.dart';
   7 import '../../../core/router/route_names.dart';
   8 import '../../../core/theme/theme_controller.dart';
   9 import '../../../core/theme/theme_tokens.dart';
  10 import '../../../shared/widgets/common/fade_up.dart';
  11 import '../../../shared/widgets/nav/lumira_nav.dart';
  12 import '../data/gallery_models.dart';
  13 import '../widgets/photo_cell.dart';
  14 import '../widgets/scene_filter_pills.dart';
  15 import '../widgets/view_toggle.dart';
  16 /// 相册主页（首个接入 DAO 的页面）
  17 /// 视觉规格来源：lumira-app/src/pages/gallery/index.vue（318 行）
  18 /// - LumiraNav + 顶部信息条（张数 + 视图切换）+ 场景筛选 pills + 3 列网
格 + 空状态
  19 /// - 数据：通过 galleryDaoProvider 异步读取 GalleryDao.getAll()
  20 class GalleryPage extends ConsumerStatefulWidget {
  21   const GalleryPage({super.key});
  22   @override
  23   ConsumerState<GalleryPage> createState() => _GalleryPageState();
  24 }
```

```
  25  class _GalleryPageState extends ConsumerState<GalleryPage> {
  26    String _activeFilter = 'all';
  27    final String _viewTab = 'photo'; // 默认照片视图；切到 diary 直接跳页
  28    // Forced fix: FutureBuilder 在测试环境下不会从
ConnectionState.waiting
  29    // 切到 done（即使 future 已解析），导致 pumpAndSettle timed out。
  30    // 改用 state-variable-based loading + setState，绕过 FutureBuilder
  31    // 的内部 listener。CircularProgressIndicator（_isLoading=true 时显
示，
  32    // 无限动画）让 pumpAndSettle 持续 pump 直到 setState 触发重建。
  33    List<GalleryItemRecord> _photos = const [];
  34    bool _isLoading = true;
  35    bool _isInitialLoaded = false;
  36    Future<List<GalleryItemRecord>> _fetchPhotos(GalleryDao dao) async
{
  37      if (_activeFilter == 'all') {
  38        return dao.getAll();
  39      }
  40      if (_activeFilter == 'uncategorized') {
  41        final all = await dao.getAll();
  42        return all.where((p) => p.sceneId == null ||
p.sceneId!.isEmpty).toList();
  43      }
  44      if (_activeFilter.startsWith('scene_')) {
  45        final sceneId = _activeFilter.replaceFirst('scene_', '');
  46        return dao.getByScene(sceneId);
  47      }
  48      return dao.getAll();
  49    }
  50    Future<void> _loadPhotos(GalleryDao dao) async {
```

```
   51      try {
   52        final photos = await _fetchPhotos(dao);
   53        if (mounted) {
   54          setState(() {
   55            _photos = photos;
   56            _isLoading = false;
   57          });
   58        }
   59      } catch (_) {
   60        if (mounted) {
   61          setState(() {
   62            _photos = const [];
   63            _isLoading = false;
   64          });
   65        }
   66      }
   67    }
   68    void _setFilter(String key) {
   69      final dao = ref.read(galleryDaoProvider).value;
   70      if (dao == null) return;
   71      setState(() {
   72        _activeFilter = key;
   73        _isLoading = true;
   74      });
   75      _loadPhotos(dao);
   76    }
   77    @override
   78    Widget build(BuildContext context) {
   79      final appTheme = ref.watch(appThemeProvider);
   80      final tokens = appTheme.tokens;
   81      final daoAsync = ref.watch(galleryDaoProvider);
   82      return Scaffold(
   83        backgroundColor: tokens.canvas,
   84        appBar: LumiraNav(
   85          title: '相册',
   86          transparent: true,
   87          leading: _BackButton(tokens: tokens),
   88          actions: [
   89            _CollectionsAction(tokens: tokens),
   90          ],
   91        ),
   92        body: Stack(
   93          children: [
   94            // 1. 径向渐变背景装饰（glass 风格可见性）
   95            _BackgroundDecoration(tokens: tokens),
   96            // 2. 主内容层
   97            SafeArea(
   98              child: daoAsync.when(
   99                loading: () => const Center(child:
CircularProgressIndicator()),
  100                error: (e, _) => Center(
```

```
  101                   child: Text('加载失败：$e', style: TextStyle(color:
tokens.textSecondary)),
  102                 ),
  103                 data: (dao) {
  104                   if (!_isInitialLoaded) {
  105                     _isInitialLoaded = true;
  106                     _loadPhotos(dao);
  107                   }
  108                   return _buildBody(tokens);
  109                 },
  110               ),
  111             ),
  112           ],
  113         ),
  114       );
  115     }
  116     Widget _buildBody(ThemeTokens tokens) {
  117       final photoViews = _photos.map(GalleryPhoto.fromRecord).toList();
  118       final pills = _buildPills(_photos);
  119       return Column(
  120         crossAxisAlignment: CrossAxisAlignment.stretch,
  121         children: [
  122           // 顶部信息条：张数 + 视图切换
  123           Padding(
  124             padding: const EdgeInsets.fromLTRB(24, 16, 24, 16),
  125             child: Row(
  126               mainAxisAlignment: MainAxisAlignment.spaceBetween,
  127               children: [
  128                 Text(
  129                   '${_photos.length} 张照片',
  130                   style: TextStyle(
  131                     fontSize: 13,
  132                     color: tokens.textTertiary,
  133                     height: 1.3,
  134                   ),
  135                 ),
  136                 ViewToggle(
  137                   activeTab: _viewTab,
  138                   onPhotoTap: () {},
  139                   onDiaryTap: () =>
GoRouter.of(context).push(RouteNames.galleryDiary),
  140                 ),
  141               ],
  142             ),
  143           ),
  144           // 场景筛选 pills
  145           SceneFilterPills(
  146             pills: pills,
  147             activeKey: _activeFilter,
  148             onTap: _setFilter,
  149           ),
  150           const SizedBox(height: 12),
```

```
  151          // 网格或空状态
  152          Expanded(
  153            child: _isLoading
  154                ? const Center(child: CircularProgressIndicator())
  155                : photoViews.isEmpty
  156                    ? _EmptyState(tokens: tokens)
  157                    : GridView.builder(
  158                        padding: const EdgeInsets.fromLTRB(24, 0, 24,
100),
  159                        gridDelegate: const
SliverGridDelegateWithFixedCrossAxisCount(
  160                          crossAxisCount: 3,
  161                          mainAxisSpacing: 6, // 12rpx → 6dp
  162                          crossAxisSpacing: 6,
  163                        ),
  164                        itemCount: photoViews.length,
  165                        itemBuilder: (_, i) {
  166                          final photo = photoViews[i];
  167                          return FadeUp(
  168                            delay: Duration(milliseconds: (i % 6) *
50),
  169                            child: PhotoCell(
  170                              photo: photo,
  171                              onTap: () => GoRouter.of(context).push(
  172                                RouteNames.build(
  173                                  RouteNames.galleryDetail,
  174                                  {RouteNames.paramPhotoId: photo.id},
  175                                ),
  176                              ),
  177                            ),
  178                          );
  179                        },
  180                      ),
  181          ),
  182        ],
  183      );
  184    }
```

第 36 页


```
   1  // ===== lib/features/profile/pages/profile_page.dart =====
   2  import 'package:flutter/material.dart';
   3  import 'package:flutter_riverpod/flutter_riverpod.dart';
   4  import 'package:go_router/go_router.dart';
   5  import '../../../core/router/route_names.dart';
   6  import '../../../core/theme/theme_controller.dart';
   7  import '../../../core/theme/theme_tokens.dart';
   8  import '../../../shared/widgets/common/fade_up.dart';
   9  import '../../../shared/widgets/nav/lumira_nav.dart';
  10  import '../../../shared/widgets/tabbar/floating_tabbar.dart';
  11  import '../data/profile_mock_data.dart';
```

```dart
import '../widgets/achievement_section.dart';
import '../widgets/growth_section.dart';
import '../widgets/profile_header.dart';
import '../widgets/settings_section.dart';
import '../widgets/stats_section.dart';
/// 个人中心
/// 视觉规格来源：lumira-app/src/pages/profile/index.vue
/// - ProfileHeader（头像 + 昵称 + 等级 + 个性签名）
/// - StatsSection（统计：连续天数 / 拍摄张数 / 收藏张数）
/// - GrowthSection（成长：当前等级 + 进度条 + 升级提示）
/// - AchievementSection（成就：徽章网格）
/// - SettingsSection（设置：主题 / 通知 / 隐私 / 关于）
class ProfilePage extends ConsumerStatefulWidget {
  const ProfilePage({super.key});
  @override
  ConsumerState<ProfilePage> createState() => _ProfilePageState();
}
class _ProfilePageState extends ConsumerState<ProfilePage> {
  final ScrollController _scrollController = ScrollController();
  bool _scrolled = false;
  static const double _scrollThreshold = 10.0;
  @override
  void initState() {
    super.initState();
    _scrollController.addListener(_onScroll);
  }
  @override
  void dispose() {
    _scrollController.removeListener(_onScroll);
    _scrollController.dispose();
    super.dispose();
  }
  void _onScroll() {
    final offset = _scrollController.offset;
    final newScrolled = offset > _scrollThreshold;
    if (newScrolled != _scrolled) {
      setState(() => _scrolled = newScrolled);
    }
  }
```

```
  53    void _goSettings() {
  54      GoRouter.of(context).push(RouteNames.profileSettings);
  55    }
  56    void _goTheme() {
  57      GoRouter.of(context).push(RouteNames.profileTheme);
  58    }
  59    void _goAcademy() {
  60      GoRouter.of(context).push(RouteNames.profileAcademy);
  61    }
  62    void _goCollections() {
  63      GoRouter.of(context).push(RouteNames.profileCollections);
  64    }
  65    void _goMyTemplates() {
  66      GoRouter.of(context).push(RouteNames.profileMyTemplates);
  67    }
  68    @override
  69    Widget build(BuildContext context) {
  70      final appTheme = ref.watch(appThemeProvider);
  71      final tokens = appTheme.tokens;
  72      return Scaffold(
  73        backgroundColor: tokens.canvas,
  74        appBar: LumiraNav(
  75          title: '我的',
  76          transparent: true,
  77          scrolled: _scrolled,
  78          showBackButton: false,
  79        ),
  80        body: Stack(
  81          children: [
  82            _BackgroundDecoration(tokens: tokens),
  83            SafeArea(
  84              child: ListView(
  85                controller: _scrollController,
  86                padding: const EdgeInsets.fromLTRB(20, 8, 20, 100),
  87                children: [
  88                  const FadeUp(child: _HeroCard(user:
ProfileMockData.userProfile)),
  89                  const SizedBox(height: 20),
  90                  const FadeUp(
  91                    delay: Duration(milliseconds: 100),
  92                    child: _StatsCard(user:
ProfileMockData.userProfile),
  93                  ),
  94                  const SizedBox(height: 20),
  95                  const FadeUp(
  96                    delay: Duration(milliseconds: 200),
  97                    child: _FragmentCard(fragments:
ProfileMockData.fragments),
  98                  ),
  99                  const SizedBox(height: 20),
 100                  FadeUp(
 101                    delay: const Duration(milliseconds: 300),
 102                    child: _QuickActionsRow(onTap: (p) =>
_goPage(context, p)),
```

```
  103                 ),
  104                 const SizedBox(height: 20),
  105                 FadeUp(
  106                   delay: const Duration(milliseconds: 400),
  107                   child: _MenuCard(onTap: (p) => _goPage(context,
p)),
  108                 ),
  109                 const SizedBox(height: 16),
  110               ],
  111             ),
  112           ),
  113           const Positioned(
  114             left: 0,
  115             right: 0,
  116             bottom: 0,
  117             child: FloatingTabBar(active: 'profile'),
  118           ),
  119         ],
  120       ),
  121     );
  122   }
  123 }
  124 /// HeroCard：用户头像 + 名字 + 等级徽章 + 经验进度
  125 class _HeroCard extends StatelessWidget {
  126   const _HeroCard({required this.user});
  127   final UserProfile user;
  128   @override
  129   Widget build(BuildContext context) {
  130     return Container(
  131       padding: const EdgeInsets.fromLTRB(32, 32, 24, 24),
  132       decoration: BoxDecoration(
  133         gradient: const LinearGradient(
  134           begin: Alignment(-0.4, -1),
  135           end: Alignment(0.4, 1),
  136           colors: [Color(0xFFFFF8EE), Color(0xFFF5EDDB),
Color(0xFFEDE3D0)],
  137           stops: [0.0, 0.4, 1.0],
  138         ),
  139         border: Border.all(color: const
Color(0xFFC9A96E).withOpacity(0.12), width: 1),
  140         borderRadius: BorderRadius.circular(24),
  141         boxShadow: [
  142           BoxShadow(
  143             color: const Color(0xFFC9A96E).withOpacity(0.08),
  144             offset: const Offset(0, 4),
  145             blurRadius: 24,
  146           ),
  147         ],
  148       ),
  149       child: Column(
  150         children: [
  151           Stack(
  152             clipBehavior: Clip.none,
```

```
  153              children: [
  154                Container(
  155                  width: 88,
  156                  height: 88,
  157                  decoration: BoxDecoration(
  158                    shape: BoxShape.circle,
  159                    border: Border.all(color: Colors.white, width: 3),
  160                    boxShadow: [
  161                      BoxShadow(
  162                        color: const
Color(0xFFC9A96E).withOpacity(0.2),
  163                        offset: const Offset(0, 4),
  164                        blurRadius: 16,
  165                      ),
  166                    ],
  167                  ),
  168                  child: ClipOval(
  169                    child: Image.network(
  170
'https://picsum.photos/seed/${user.avatarSeed}/200/200',
  171                      fit: BoxFit.cover,
  172                    ),
  173                  ),
  174                ),
  175                Positioned(
  176                  bottom: 0,
  177                  right: 0,
  178                  child: Container(
  179                    width: 22,
  180                    height: 22,
  181                    decoration: BoxDecoration(
  182                      shape: BoxShape.circle,
  183                      gradient: const LinearGradient(
  184                        colors: [Color(0xFFC9A96E), Color(0xFFA88550)],
  185                      ),
  186                      border: Border.all(color: Colors.white, width:
2.5),
  187                    ),
  188                    child: const Icon(
  189                      Icons.keyboard_arrow_up,
  190                      size: 14,
  191                      color: Colors.white,
  192                    ),
  193                  ),
  194                ),
  195              ],
  196            ),
```

第 37 页


```
1  // ===== lib/features/challenge/pages/challenge_page.dart =====
```

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import '../../../core/router/route_names.dart';
import '../../../core/theme/theme_controller.dart';
import '../../../core/theme/theme_tokens.dart';
import '../../../shared/widgets/common/fade_up.dart';
import '../../../shared/widgets/nav/lumira_nav.dart';
import '../../../shared/widgets/tabbar/floating_tabbar.dart';
import '../data/challenge_mock_data.dart';
import '../widgets/challenge_card.dart';
import '../widgets/section_header.dart';
/// 挑战页
/// 视觉规格来源：lumira-app/src/pages/challenge/index.vue
/// - 顶部欢迎语 + 每日挑战卡片
/// - 合拍挑战列表
/// - 历史挑战
class ChallengePage extends ConsumerStatefulWidget {
  const ChallengePage({super.key});
  @override
  ConsumerState<ChallengePage> createState() =>
_ChallengePageState();
}
class _ChallengePageState extends ConsumerState<ChallengePage> {
  final ScrollController _scrollController = ScrollController();
  bool _scrolled = false;
  @override
  void initState() {
    super.initState();
    _scrollController.addListener(_onScroll);
  }
  @override
  void dispose() {
    _scrollController.removeListener(_onScroll);
    _scrollController.dispose();
    super.dispose();
  }
  void _onScroll() {
    final offset = _scrollController.offset;
    final newScrolled = offset > 10.0;
    if (newScrolled != _scrolled) {
      setState(() => _scrolled = newScrolled);
    }
  }
  void _goChallengeDetail(String id) {
    GoRouter.of(context).push(RouteNames.challengeDetail);
  }
  @override
  Widget build(BuildContext context) {
    final tokens = ref.watch(themeTokensProvider);
```

```
   51      return Scaffold(
   52        backgroundColor: tokens.canvas,
   53        body: Stack(
   54          children: [
   55            // 径向渐变背景装饰（glass 风格可见性）
   56            Positioned.fill(
   57              child: DecoratedBox(
   58                decoration: BoxDecoration(
   59                  gradient: RadialGradient(
   60                    center: const Alignment(-0.7, -0.5),
   61                    radius: 1.2,
   62                    colors: [
   63                      tokens.brand.withOpacity(0.06),
   64                      tokens.canvas,
   65                    ],
   66                    stops: const [0.0, 0.6],
   67                  ),
   68                ),
   69              ),
   70            ),
   71            // Forced fix: glass 风格彩色斑点背景
   72            const Positioned.fill(child: GlassBackground(variant:
GlassBackgroundVariant.challenge)),
   73            // 主内容
   74            SafeArea(
   75              child: Column(
   76                children: [
   77                  LumiraNav(
   78                    title: '每日挑战',
   79                    scrolled: _scrolled,
   80                    transparent: true,
   81                    showBackButton: false,
   82                    actions: [
   83                      IconButton(
   84                        icon: Icon(
   85                          Icons.assignment_outlined,
   86                          size: 20,
   87                          color: tokens.textPrimary,
   88                        ),
   89                        onPressed: () {},
   90                        tooltip: '挑战记录',
   91                      ),
   92                    ],
   93                  ),
   94                  Expanded(
   95                    child: ListView(
   96                      controller: _scrollController,
   97                      padding: const EdgeInsets.fromLTRB(20, 12, 20,
100),
   98                      children: [
   99                        // 1. 主挑战卡
  100                        const FadeUp(
```

```
                        child: MainChallengeCard(
                          challenge: ChallengeMockData.mainChallenge,
                        ),
                      ),
                      const SizedBox(height: 32),
                      // 2. 附加挑战区块
                      const FadeUp(
                        delay: Duration(milliseconds: 80),
                        child: _SectionTitle(
                          title: '附加挑战',
                          subtitle: '1+2 弹性模式',
                        ),
                      ),
                      const SizedBox(height: 16),

                    ...ChallengeMockData.subChallenges.asMap().entri
es.map(
                          (entry) => Padding(
                            padding: const EdgeInsets.only(bottom:
12),
                            child: FadeUp(
                              delay: Duration(
                                milliseconds: 160 + entry.key * 80,
                              ),
                              child: SubChallengeRow(
                                challenge: entry.value,
                                onGoComplete: () =>
                                    _goDetail(entry.value.id),
                              ),
                            ),
                          ),
                        ),
                      const SizedBox(height: 32),
                      // 3. 明日挑战预览
                      FadeUp(
                        delay: const Duration(milliseconds: 320),
                        child: _SectionTitle(
                          title: '明日挑战预览',
                          trailing: _SectionLink(
                            text: '全部',
                            onTap: _goAllTomorrow,
                          ),
                        ),
                      ),
                      const SizedBox(height: 16),
                      const FadeUp(
                        delay: Duration(milliseconds: 320),
                        child: TomorrowPreviewCard(
                          preview: ChallengeMockData.tomorrowPreview,
                        ),
                      ),
                      const SizedBox(height: 32),
                      // 4. 连续打卡
```

```
  151                         const FadeUp(
  152                           delay: Duration(milliseconds: 400),
  153                           child: StreakCard(streak:
ChallengeMockData.streak),
  154                         ),
  155                       ],
  156                     ),
  157                   ),
  158                 ],
  159               ),
  160             ),
  161             // FloatingTabBar
  162             const Positioned(
  163               left: 0,
  164               right: 0,
  165               bottom: 0,
  166               child: FloatingTabBar(active: 'challenge'),
  167             ),
  168           ],
  169         ),
  170       );
  171     }
  172   }
```

第 38 页


```
   1  // ===== lib/features/inspiration/pages/inspiration_page.dart =====
   2  import 'package:flutter/material.dart';
   3  import 'package:flutter_riverpod/flutter_riverpod.dart';
   4  import 'package:go_router/go_router.dart';
   5  import '../../../core/router/route_names.dart';
   6  import '../../../core/theme/theme_controller.dart';
   7  import '../../../shared/widgets/cards/neu_card.dart';
   8  import '../../../shared/widgets/common/fade_up.dart';
   9  import '../../../shared/widgets/nav/lumira_nav.dart';
  10  import '../../home/widgets/scene_reco_card.dart';
  11  import '../data/inspiration_mock_data.dart';
  12  import '../widgets/checkin_card.dart';
  13  import '../widgets/mood_card.dart';
  14  import '../widgets/outfit_diary_card.dart';
  15  /// 灵感页
  16  /// 视觉规格来源：lumira-app/src/pages/inspiration/index.vue（549 行）
   17  /// - 5 个 section：今日心情 / 穿搭日记 / 推荐场景 / 探店打卡 / 加载更多
   18  /// - 所有 section 用 FadeUp 错峰入场（d0/d1/d2/d3/d4）
   19  /// - 路由跳转：goSceneDetail(id) → captureSceneDetail?sceneId=xxx；
goSceneManage → captureSceneManage
   20  class InspirationPage extends ConsumerWidget {
   21    const InspirationPage({super.key});
   22    void _goSceneDetail(BuildContext context, String sceneId) {
   23      GoRouter.of(context).push(
```

```
   24
'${RouteNames.captureSceneDetail}?${RouteNames.paramSceneId}=$sceneId',
   25      );
   26    }
   27    void _goSceneManage(BuildContext context) {
   28      GoRouter.of(context).push(RouteNames.captureSceneManage);
   29    }
   30    void _goViewDiary(BuildContext context) {
   31      // uni-app 跳 gallery/diary（查看日记）
   32      GoRouter.of(context).push(RouteNames.galleryDiary);
   33    }
   34    @override
   35    Widget build(BuildContext context, WidgetRef ref) {
   36      final tokens = ref.watch(appThemeProvider).tokens;
   37      return Scaffold(
   38        backgroundColor: tokens.canvas,
   39        extendBodyBehindAppBar: true,
   40        appBar: const LumiraNav(title: '灵感', transparent: true),
   41        body: Container(
   42          // 径向渐变背景装饰（glass 风格可见性）
   43          decoration: BoxDecoration(
   44            gradient: RadialGradient(
   45              center: const Alignment(-0.8, -0.6), // 左上
   46              radius: 1.2,
   47              colors: [
   48                tokens.brandSubtle.withOpacity(0.35),
   49                tokens.canvas.withOpacity(0.0),
   50              ],
```

```
   51            ),
   52          ),
   53          child: SafeArea(
   54            child: SingleChildScrollView(
   55              padding: const EdgeInsets.symmetric(horizontal: 24,
vertical: 16),
   56              child: Column(
   57                crossAxisAlignment: CrossAxisAlignment.stretch,
   58                children: [
   59                  // 1. 今日心情卡片
   60                  const FadeUp(
   61                    child: MoodCard(),
   62                  ),
   63                  const SizedBox(height: 20),
   64                ],
   65              ),
   66            ),
   67          ),
   68        ),
   69      );
   70    }
   71  }
   72  /// 推荐场景卡片
   73  class _RecommendScenesCard extends ConsumerWidget {
   74    const _RecommendScenesCard({required this.onSceneTap, required
this.onMore});
   75    final void Function(String sceneId) onSceneTap;
   76    final VoidCallback onMore;
   77    @override
   78    Widget build(BuildContext context, WidgetRef ref) {
   79      final tokens = ref.watch(appThemeProvider).tokens;
   80      return NeuCard(
   81        child: Column(
   82          crossAxisAlignment: CrossAxisAlignment.start,
   83          children: [
   84            // 标题 + 副标题
   85            Row(
   86              children: [
   87                Icon(
   88                  Icons.explore_outlined,
   89                  size: 18,
   90                  color: tokens.brand,
   91                ),
   92                const SizedBox(width: 8),
   93                Expanded(
   94                  child: Column(
   95                    crossAxisAlignment: CrossAxisAlignment.start,
   96                    children: [
   97                      Text(
   98                        '根据你的喜好推荐',
   99                        style: TextStyle(
  100                          fontSize: 17,
```

```
                          fontWeight: FontWeight.w600,
                          color: tokens.textPrimary,
                          fontFamily: 'Noto Serif SC',
                        ),
                      ),
                      const SizedBox(height: 4),
                      Text(
                        InspirationMockData.recommendSubtitle,
                        style: TextStyle(
                          fontSize: 12,
                          color: tokens.textTertiary,
                        ),
                      ),
                    ],
                  ),
                ),
              ],
            ),
            const SizedBox(height: 16),
            // 2x2 场景网格
            GridView.count(
              shrinkWrap: true,
              physics: const NeverScrollableScrollPhysics(),
              crossAxisCount: 2,
              mainAxisSpacing: 10,
              crossAxisSpacing: 10,
              childAspectRatio: 0.46,
              children:
InspirationMockData.scenes.map((inspirationScene) {
                return SceneRecoCard(
                  scene: inspirationScene.scene,
                  onTap: () => onSceneTap(inspirationScene.scene.id),
                  showPhotoCount: false,
                  footer: _SceneTagFooter(tagInfo:
inspirationScene.tagInfo, photoCount: inspirationScene.scene.photoCount),
                );
              }).toList(),
            ),
            const SizedBox(height: 16),
            // 发现更多场景链接
            Center(
              child: GestureDetector(
                onTap: onMore,
                behavior: HitTestBehavior.opaque,
                child: Row(
                  mainAxisSize: MainAxisSize.min,
                  children: [
                    Text(
                      '发现更多场景',
                      style: TextStyle(
                        fontSize: 13,
                        color: tokens.brand,
```

```
  151                     ),
  152                   ),
  153                   Icon(Icons.arrow_right_alt, size: 14, color:
tokens.brand),
  154                 ],
  145               ),
  146             ),
  147           ),
  148         ],
  149       ),
  150     );
  151   }
  152 }
```

第 39 页


```
   1  // ===== lib/features/scenes/pages/scenes_page.dart =====
   2  import 'package:flutter/material.dart';
   3  import 'package:flutter_riverpod/flutter_riverpod.dart';
   4  import 'package:go_router/go_router.dart';
   5  import '../../../core/router/route_names.dart';
   6  import '../../../shared/widgets/nav/lumira_nav.dart';
   7  import '../../capture/data/capture_scene_mock_data.dart';
   8  import '../data/scenes_mock_data.dart';
   9  /// Scenes 独立场景库页（Task 2.11）
  10  /// 视觉规格来源：lumira-app/src/pages/scenes/index.vue (159 行)
  11  /// - 顶部导航：返回 + 标题「场景库」+ 右侧搜索按钮（toast 占位）
  12  /// - 分类 tab 横滑条：全部 / 光线 / 室外 / 室内 / 情绪（5 项 pill）
  13  /// - 场景 grid 2 列：封面图 + 名称 + vibe + 照片数 badge（>0 时显示）
  14  /// - FAB：右下角圆形 + 按钮 → /capture/scene-manage?tab=custom
  15  /// 跳转：
  16  /// - 点击场景卡 → /capture/scene-detail?sceneId=xxx
  17  /// - 点击搜索按钮 → SnackBar「搜索功能开发中」（与 uni-app showToast 一
致）
  18  class ScenesPage extends ConsumerStatefulWidget {
  19    const ScenesPage({super.key});
  20    @override
  21    ConsumerState<ScenesPage> createState() => _ScenesPageState();
  22  }
  23  class _ScenesPageState extends ConsumerState<ScenesPage> {
   24    String _activeCategoryId = 'all';
   25    List<ScenePreset> get _filteredScenes {
   26      final allScenes = ref.read(scenesListProvider);
   27      if (_activeCategoryId == 'all') return allScenes;
   28      final pill =
   29          scenesCategoryPills.firstWhere((p) => p.id ==
_activeCategoryId);
   30      return allScenes.where((s) => s.category ==
pill.category).toList();
   31    }
```

```
32    void _back() {
33      if (Navigator.of(context).canPop()) {
34        Navigator.of(context).pop();
35      } else {
36        GoRouter.of(context).go(RouteNames.home);
37      }
38    }
39    void _onSearch() {
40      ScaffoldMessenger.of(context).showSnackBar(
41        const SnackBar(content: Text('搜索功能开发中')),
42      );
43    }
44    void _goDetail(String id) {
45      GoRouter.of(context).push(
46        RouteNames.withSceneId(RouteNames.captureSceneDetail, id),
47      );
48    }
49    void _goCreate() {
50      GoRouter.of(context).push(
```

```
51        RouteNames.build(RouteNames.captureSceneManage, {
52          RouteNames.paramTab: 'custom',
53        }),
54      );
55    }
56    void _onCategorySelect(String id) {
57      setState(() {
58        _activeCategoryId = id;
59      });
60    }
61    @override
62    Widget build(BuildContext context) {
63      return Scaffold(
64        // 硬编码颜色，与 uni-app scenes-container bg #FAF7F2 一致
65        backgroundColor: const Color(0xFFFAF7F2),
66        body: SafeArea(
67          child: Column(
68            children: [
69              _ScenesNav(onBack: _back, onSearch: _onSearch),
70              _CategoryBar(
71                activeId: _activeCategoryId,
72                onSelect: _onCategorySelect,
73              ),
74              Expanded(
75                child: _SceneGrid(
76                  scenes: _filteredScenes,
77                  onTap: _goDetail,
78                ),
79              ),
80            ],
81          ),
82        ),
83        floatingActionButton: _Fab(onTap: _goCreate),
84      );
85    }
86  }
87  /// 顶部导航：返回 + 标题「场景库」+ 搜索
88  class _ScenesNav extends StatelessWidget {
89    const _ScenesNav({required this.onBack, required this.onSearch});
90    final VoidCallback onBack;
91    final VoidCallback onSearch;
92    @override
93    Widget build(BuildContext context) {
94      return LumiraNav(
95        title: '场景库',
96        transparent: true,
97        leading: _NavIconButton(
98          icon: Icons.arrow_back_ios_new,
99          onTap: onBack,
100       ),
```

```
        actions: [
          _NavIconButton(
            icon: Icons.search,
            onTap: onSearch,
          ),
        ],
      );
    }
  }
  class _NavIconButton extends StatelessWidget {
    const _NavIconButton({required this.icon, required this.onTap});
    final IconData icon;
    final VoidCallback onTap;
    @override
    Widget build(BuildContext context) {
      return GestureDetector(
        onTap: onTap,
        behavior: HitTestBehavior.opaque,
        child: Padding(
          padding: const EdgeInsets.all(8),
          child: Icon(
            icon,
            size: 20,
            color: const Color(0xFF2A2520),
          ),
        ),
      );
    }
  }
  /// 分类 tab 横滑条（5 项 pill）
  class _CategoryBar extends StatelessWidget {
    const _CategoryBar({required this.activeId, required
this.onSelect});
    final String activeId;
    final ValueChanged<String> onSelect;
    @override
    Widget build(BuildContext context) {
      return Padding(
        padding: const EdgeInsets.symmetric(vertical: 8),
        child: SingleChildScrollView(
          scrollDirection: Axis.horizontal,
          padding: const EdgeInsets.symmetric(horizontal: 12),
          child: Row(
            children: [
              for (final pill in scenesCategoryPills) ...[
                _CategoryPill(
                  label: pill.name,
                  active: activeId == pill.id,
                  onTap: () => onSelect(pill.id),
                ),
                const SizedBox(width: 8),
```

```
              ],
            ],
          ),
        ),
      );
    }
  }
  class _CategoryPill extends StatelessWidget {
    const _CategoryPill({
      required this.label,
      required this.active,
      required this.onTap,
    });
    final String label;
    final bool active;
    final VoidCallback onTap;
    @override
    Widget build(BuildContext context) {
      return GestureDetector(
        onTap: onTap,
        behavior: HitTestBehavior.opaque,
        child: Container(
          padding: const EdgeInsets.symmetric(
            horizontal: 14,
            vertical: 6,
          ),
          decoration: BoxDecoration(
            // 与 uni-app 一致：active 用 brand-primary，inactive 用
card bg
```

第 40 页


```
// ===== lib/shared/widgets/tabbar/floating_tabbar.dart =====
import 'dart:ui';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import '../../../core/theme/theme_controller.dart';
import '../../../core/theme/theme_tokens.dart';
import '../../../core/router/route_names.dart';
/// 如画应用悬浮 Tab 栏
/// 视觉规格来源：lumira-app/src/components/FloatingTabBar.vue +
App.vue line 488-555
/// - fixed 定位，bottom 28rpx→14dp
/// - width: calc(100% - 80rpx) → 横向 padding 40rpx→20dp
/// - height: 108rpx → 54dp
/// - border-radius: 9999rpx → 完全圆角（pill 形）
/// - canvas 背景 + backdrop-filter blur 24px
/// - shadow-convex
/// - 5 个 item：首页/模板/(中心拍摄按钮)/挑战/我的
```

```
  18  /// - 中心按钮：100rpx→50dp 圆形 + brand bg + shadow-convex-brand +
translateY(-12rpx)→-6dp
  19  /// - active item: brand 色
  20  /// - 女性美学：active item + center 有呼吸光晕动画
  21  class FloatingTabBar extends ConsumerStatefulWidget {
  22    const FloatingTabBar({
  23      super.key,
  24      required this.active,
  25    });
  26    /// 当前激活的 tab key：'home' | 'templates' | 'challenge' |
'profile'
  27    final String active;
  28    @override
  29    ConsumerState<FloatingTabBar> createState() =>
_FloatingTabBarState();
  30  }
  31  class _FloatingTabBarState extends ConsumerState<FloatingTabBar> {
  32    @override
  33    Widget build(BuildContext context) {
  34      final appTheme = ref.watch(appThemeProvider);
  35      final tokens = appTheme.tokens;
  36      final isFemale = appTheme.style == UIStyle.female;
  37      final isGlass = appTheme.style == UIStyle.glass;
  38      // Forced fix: 之前 _CenterCaptureButton 在 ClipRRect 内部，
  39      // Transform.translate(0, -6) 的部分被 ClipRRect 剪切。
  40      // 改为 Stack 结构：底层是带 ClipRRect 的 tab bar（4 个 tab + 中间空
位），
  41      // 顶层是悬浮的 capture button（可自由超出 tab bar 范围）。
  42      return Align(
  43        alignment: Alignment.bottomCenter,
  44        child: Padding(
  45          padding: const EdgeInsets.only(
  46            left: 20,
  47            right: 20,
  48            bottom: 14,
  49          ),
  50          child: SizedBox(
```

```
51            height: 70,
52            child: Stack(
53              clipBehavior: Clip.none,
54              alignment: Alignment.bottomCenter,
55              children: [
56                // 底层：tab bar（带 ClipRRect + BackdropFilter）
57                Positioned(
58                  left: 0,
59                  right: 0,
60                  bottom: 0,
61                  child: ClipRRect(
62                    borderRadius: BorderRadius.circular(1000),
63                    child: BackdropFilter(
64                      filter: ImageFilter.blur(
65                        sigmaX: isGlass ? 25 : 0,
66                        sigmaY: isGlass ? 25 : 0,
67                      ),
68                      child: Container(
69                        height: 54,
70                        decoration: BoxDecoration(
71                          gradient: isGlass
72                              ? LinearGradient(
73                                  begin: Alignment.topLeft,
74                                  end: Alignment.bottomRight,
75                                  colors: [
76                                    Colors.white.withOpacity(0.75),
77                                    Colors.white.withOpacity(0.45),
78                                    Colors.white.withOpacity(0.30),
79                                  ],
80                                  stops: const [0.0, 0.5, 1.0],
81                                )
82                              : null,
83                          color: isGlass ? null : tokens.canvas,
84                          borderRadius: BorderRadius.circular(1000),
85                        ),
86                      ),
87                    ),
88                  ),
89                ),
90              ],
91            ),
92          ),
93        ),
94      );
95    }
 96  }
 97  class _TabItem extends StatelessWidget {
 98    const _TabItem({
 99      required this.icon,
100      required this.label,
```

```
  101      required this.active,
  102      required this.onTap,
  103      required this.tokens,
  104      required this.isFemale,
  105    });
  106    final IconData icon;
  107    final String label;
  108    final bool active;
  109    final VoidCallback onTap;
  110    final ThemeTokens tokens;
  111    final bool isFemale;
  112    @override
  113    Widget build(BuildContext context) {
  114      final color = active ? tokens.brand : tokens.textTertiary;
  115      return Expanded(
  116        child: GestureDetector(
  117          onTap: onTap,
  118          behavior: HitTestBehavior.opaque,
  119          child: Column(
  120            mainAxisAlignment: MainAxisAlignment.center,
  121            children: [
  122              Icon(icon, size: 22, color: color),
  123              const SizedBox(height: 2),
  124              Text(
  125                label,
  126                style: TextStyle(
  127                  fontSize: 10, // 20rpx → 10dp
  128                  fontWeight: active ? FontWeight.w600 :
FontWeight.w400,
  129                  color: color,
  130                  height: 1.2,
  131                ),
  132              ),
  133            ],
  134          ),
  135        ),
  136      );
  137    }
  138  }
  139  /// 中心拍摄按钮（悬浮于 tab bar 之上，可超出范围）
  140  class _CenterCaptureButton extends ConsumerWidget {
  141    const _CenterCaptureButton({required this.tokens, required
this.isFemale});
  142    final ThemeTokens tokens;
  143    final bool isFemale;
  144    @override
  145    Widget build(BuildContext context, WidgetRef ref) {
  146      return GestureDetector(
  147        onTap: () => GoRouter.of(context).push(RouteNames.capture),
  148        child: Transform.translate(
  149          offset: const Offset(0, -6), // translateY(-12rpx) → -6dp
  150          child: Container(
```

```
151            width: 50, // 100rpx → 50dp
152            height: 50,
153            decoration: BoxDecoration(
154              shape: BoxShape.circle,
155              color: tokens.brand,
156              boxShadow: isFemale
157                  ? [
158                      BoxShadow(
159                        color: tokens.brand.withOpacity(0.30),
160                        offset: const Offset(0, 6),
161                        blurRadius: 16,
162                      ),
163                      BoxShadow(
164                        color: tokens.brand.withOpacity(0.15),
165                        offset: const Offset(0, 2),
166                        blurRadius: 6,
167                      ),
168                    ]
169                  : tokens.shadowConvexBrand,
170            ),
171            child: Icon(
172              Icons.camera_alt, // ph-camera-icon 替代
173              size: 22, // 44rpx → 22dp
174              color: Colors.white,
175            ),
176          ),
177        ),
178      );
179    }
180  }
```

第 41 页

如画 Lumira 摄影辅助应用 V1.0.0 第 41 页

```
1  // ===== lib/shared/widgets/cards/neu_card.dart =====
2  import 'dart:ui';
3  import 'package:flutter/material.dart';
4  import 'package:flutter_riverpod/flutter_riverpod.dart';
5  import '../../../core/theme/app_theme.dart';
6  import '../../../core/theme/theme_controller.dart';
7  import '../../../core/theme/theme_tokens.dart';
8  /// 如画应用统一卡片组件
9  /// 视觉规格来源：lumira-app/src/App.vue line 640-660 + 900-1200
10 /// 4 种 UI 风格分支渲染：
11 /// - neumorphic: canvas 背景 + shadowConvex 双向外阴影 + 14dp 圆角
12 /// - flat: canvas 背景 + divider 0.5dp 边框 + 10dp 圆角 + 无阴影
13 /// - glass: rgba(255,255,255,0.55) 背景 + white 30% 0.5dp 边框 + 14dp
圆角 + backdrop blur 20px
14 /// - female: multiGradient 5 层（linear + radial + hairline + brand
shadow）+ 24dp 圆角
15 class NeuCard extends ConsumerWidget {
```

```
  16    const NeuCard({
  17      super.key,
  18      required this.child,
  19      this.padding = const EdgeInsets.all(20), // 40rpx → 20dp
  20      this.margin,
  21      this.onTap,
  22      this.enableHoverScale = false,
  23    });
  24    final Widget child;
  25    final EdgeInsetsGeometry padding;
  26    final EdgeInsetsGeometry? margin;
  27    final VoidCallback? onTap;
  28    final bool enableHoverScale;
  29    @override
  30    Widget build(BuildContext context, WidgetRef ref) {
  31      final appTheme = ref.watch(appThemeProvider);
  32      final tokens = appTheme.tokens;
  33      // rpx → dp：app_theme.cardRadius 存储的是 rpx 原值
（28/20/28/48），/2 得 dp
  34      final radius = appTheme.cardRadius / 2;
  35      Widget card = _buildCard(appTheme, tokens, radius);
  36      if (enableHoverScale && onTap != null) {
  37        card = _ScaleTap(
  38          scale: appTheme.style == UIStyle.female ? 0.96 : 0.98,
  39          onTap: onTap!,
  40          child: card,
  41        );
  42      } else if (onTap != null) {
  43        card = GestureDetector(onTap: onTap, child: card);
  44      }
  45      if (margin != null) {
  46        card = Padding(padding: margin!, child: card);
  47      }
  48      return card;
  49    }
  50    Widget _buildCard(AppThemeData appTheme, ThemeTokens tokens, double
radius) {
```

```
51      switch (appTheme.style) {
52        case UIStyle.neumorphic:
53          // Forced fix: 新拟态需卡片与背景同色 + 双向阴影才有效果。
54          return Container(
55            padding: padding,
56            decoration: BoxDecoration(
57              color: tokens.surface, // surface 比 canvas 稍亮，增强阴影
对比
58              borderRadius: BorderRadius.circular(radius),
59              boxShadow: tokens.shadowConvex,
60            ),
61            child: child,
62          );
63        case UIStyle.flat:
64          // Forced fix: 扁平化使用 surfaceAlt 区分卡片与背景，无阴影、细边
框。
65          return Container(
66            padding: padding,
67            decoration: BoxDecoration(
68              color: tokens.surfaceAlt, // 区分于 canvas 背景
69              borderRadius: BorderRadius.circular(radius),
70              border: Border.all(color: tokens.divider, width: 1),
71            ),
72            child: child,
73          );
74      }
 75    }
 76    Widget _buildGlassCard(ThemeTokens tokens, double radius) {
 77      // Forced fix: 玻璃拟态彻底重做，加强 5 个视觉特征让 glass 效果明
显：
 78      // 1. blur 25（强模糊）
 79      // 2. 3 段渐变：顶部高光白 0.85 → 中部白 0.50 → 底部白 0.30（玻璃边
缘反射）
 80      // 3. 双层边框：外白 0.6 + 内白 0.15（玻璃厚度感）
 81      // 4. 顶部高光反射：Stack 顶层放一个 LinearGradient（白 0.45 → 透
明）只覆盖顶部 1/3
 82      // 5. 深阴影：brand 色微染 + 黑色阴影双层
 83      return ClipRRect(
 84        borderRadius: BorderRadius.circular(radius),
 85        child: Stack(
 86          children: [
 87            // 层 1: BackdropFilter blur
 88            Positioned.fill(
 89              child: BackdropFilter(
 90                filter: ImageFilter.blur(sigmaX: 25, sigmaY: 25),
 91                child: Container(
 92                  decoration: BoxDecoration(
 93                    // 3 段渐变模拟玻璃边缘反射
 94                    gradient: LinearGradient(
 95                      begin: Alignment.topLeft,
 96                      end: Alignment.bottomRight,
 97                      colors: [
 98                        Colors.white.withOpacity(0.85),
 99                        Colors.white.withOpacity(0.50),
```

```
100                         Colors.white.withOpacity(0.30),
```

```
101                     ],
102                     stops: const [0.0, 0.5, 1.0],
103                   ),
104                   border: Border.all(
105                     color: Colors.white.withOpacity(0.6),
106                     width: 1.2,
109                   ),
110                 ),
111               ),
112             ),
113           ),
114           // 层 2: 顶部高光反射（玻璃边缘高光）
115           Positioned(
116             top: 0,
117             left: 0,
118             right: 0,
119             height: 40,
120             child: IgnorePointer(
121               child: ClipRRect(
122                 borderRadius: BorderRadius.only(
123                   topLeft: Radius.circular(radius),
124                   topRight: Radius.circular(radius),
125                 ),
126                 child: DecoratedBox(
127                   decoration: BoxDecoration(
128                     gradient: LinearGradient(
129                       begin: Alignment.topCenter,
130                       end: Alignment.bottomCenter,
131                       colors: [
132                         Colors.white.withOpacity(0.45),
133                         Colors.white.withOpacity(0.0),
134                       ],
135                     ),
136                   ),
137                 ),
138               ),
139             ),
140           ),
141           // 层 3: 内容
142           Padding(
143             padding: padding,
144             child: child,
146           ),
147         ],
148       ),
149     );
150   }
151 }
152 class _ScaleTap extends StatefulWidget {
153   const _ScaleTap({
```

```
154      required this.child,
155      required this.onTap,
156      this.scale = 0.97,
157    });
158    final Widget child;
159    final VoidCallback? onTap;
160    final double scale;
161    @override
162    State<_ScaleTap> createState() => _ScaleTapState();
163  }
164  class _ScaleTapState extends State<_ScaleTap> {
165    bool _pressed = false;
166    @override
167    Widget build(BuildContext context) {
168      return GestureDetector(
169        onTapDown: widget.onTap == null
170            ? null
171            : (_) => setState(() => _pressed = true),
172        onTapUp: widget.onTap == null
173            ? null
174            : (_) {
175                setState(() => _pressed = false);
176                widget.onTap!();
177              },
178        onTapCancel: widget.onTap == null
179            ? null
180            : () => setState(() => _pressed = false),
181        child: AnimatedScale(
182          scale: _pressed ? widget.scale : 1.0,
183          duration: const Duration(milliseconds: 150),
184          curve: Curves.easeOut,
185          child: widget.child,
186        ),
187      );
188    }
189  }
```

第 42 页


```
1  // ===== lib/shared/widgets/buttons/lumira_buttons.dart =====
2  import 'package:flutter/material.dart';
3  import 'package:flutter_riverpod/flutter_riverpod.dart';
4  import '../../../core/theme/app_theme.dart';
5  import '../../../core/theme/theme_controller.dart';
6  import '../../../core/theme/theme_tokens.dart';
7  /// 如画应用按钮变体
8  enum LumiraButtonVariant {
9    /// 主按钮：textPrimary 背景 + canvas 文字
10   primary,
11   /// 品牌按钮：brand 背景 + inverse 文字 + brand 阴影
12   brand,
```

```
  13    /// 描边按钮：透明背景 + divider 边框
  14    outline,
  15    /// 幽灵按钮：surfaceAlt 背景 + textSecondary 文字（小尺寸）
  16    ghost,
  17  }
  18  /// 如画应用统一按钮
  19  /// 视觉规格来源：lumira-app/src/App.vue line 558-637
  20  /// - primary: textPrimary bg + canvas text + 16rpx radius +
28rpx×48rpx padding
  21  /// - brand: brand bg + inverse text + brand shadow + 16rpx radius +
28rpx×48rpx padding
  22  /// - outline: transparent + textPrimary text + 3rpx divider border +
16rpx radius + 28rpx×48rpx padding
  23  /// - ghost: surfaceAlt bg + textSecondary text + 16rpx radius +
20rpx×32rpx padding
  24  /// 所有变体：
  25  /// - width: 100%（除非 expand=false）
  26  /// - max-width: 100% + box-sizing: border-box（防右溢出）
  27  /// - :active scale(0.97)
  28  class LumiraButton extends ConsumerWidget {
  29    const LumiraButton({
  30      super.key,
  31      required this.label,
  32      this.onPressed,
  33      this.variant = LumiraButtonVariant.primary,
  34      this.icon,
  35      this.expand = true,
  36      this.enabled = true,
  37    });
  38    final String label;
  39    final VoidCallback? onPressed;
  40    final LumiraButtonVariant variant;
  41    final IconData? icon;
  42    final bool expand;
  43    final bool enabled;
  44    @override
  45    Widget build(BuildContext context, WidgetRef ref) {
  46      final appTheme = ref.watch(appThemeProvider);
  47      final tokens = appTheme.tokens;
  48      final spec = _ButtonSpec.from(variant, tokens, appTheme);
  49      final content = Row(
  50        mainAxisSize: MainAxisSize.min,
```

```
        children: [
          if (icon != null) ...[
            Icon(icon, size: 18, color: spec.foregroundColor),
            const SizedBox(width: 8), // gap 16rpx → 8dp
          ],
          Text(
            label,
            style: TextStyle(
              fontSize: spec.fontSize,
              fontWeight: FontWeight.w500,
              color: spec.foregroundColor,
              height: 1,
            ),
          ),
        ],
      );
      final decoration = BoxDecoration(
        color: spec.backgroundColor,
        borderRadius: BorderRadius.circular(spec.radius),
        border: spec.border,
      );
      Widget button = _ScaleTap(
        scale: 0.97,
        onTap: enabled ? onPressed : null,
        child: AnimatedContainer(
          duration: const Duration(milliseconds: 200),
          padding: spec.padding,
          decoration: decoration,
          child: content,
        ),
      );
      if (!enabled) {
        button = Opacity(opacity: 0.5, child: button);
      }
      // expand: width 100% + max-width 100% + box-sizing border-box
      if (expand) {
        return ConstrainedBox(
          constraints: const BoxConstraints(maxWidth: double.infinity),
          child: SizedBox(width: double.infinity, child: button),
        );
      }
      return button;
    }
  }
  class _ButtonSpec {
    final Color backgroundColor;
    final Color foregroundColor;
    final double radius;
    final double fontSize;
    final EdgeInsets padding;
```

```
  101    final Border? border;
  102    final List<BoxShadow>? boxShadow;
  103    _ButtonSpec({
  104      required this.backgroundColor,
  105      required this.foregroundColor,
  106      required this.radius,
  107      required this.fontSize,
  108      required this.padding,
  109      this.border,
  110      this.boxShadow,
  111    });
  112    factory _ButtonSpec.from(
  113      LumiraButtonVariant variant,
  114      ThemeTokens tokens,
  115      AppThemeData appTheme,
  116    ) {
  117      final brandShadow = appTheme.style == UIStyle.neumorphic ||
  118              appTheme.style == UIStyle.female
  119          ? tokens.shadowConvexBrand
  120          : null;
  121      switch (variant) {
  122        case LumiraButtonVariant.primary:
  123          return _ButtonSpec(
  124            backgroundColor: tokens.textPrimary,
  125            foregroundColor: tokens.canvas,
  126            radius: 8, // 16rpx → 8dp
  127            fontSize: 15, // 30rpx → 15dp
  128            padding: const EdgeInsets.symmetric(
  129              horizontal: 24, // 48rpx → 24dp
  130              vertical: 14, // 28rpx → 14dp
  131            ),
  132          );
  133        case LumiraButtonVariant.brand:
  134          return _ButtonSpec(
  135            backgroundColor: tokens.brand,
  136            foregroundColor: tokens.textInverse,
  137            radius: 8,
  138            fontSize: 15,
  139            padding: const EdgeInsets.symmetric(horizontal: 24,
vertical: 14),
  140            boxShadow: brandShadow,
  141          );
  142        case LumiraButtonVariant.outline:
  143          return _ButtonSpec(
  144            backgroundColor: Colors.transparent,
  145            foregroundColor: tokens.textPrimary,
  146            radius: 8,
  147            fontSize: 15,
  148            padding: const EdgeInsets.symmetric(horizontal: 24,
vertical: 14),
  149            border: Border.all(color: tokens.divider, width: 1.5), //
3rpx → 1.5dp
  150          );
```

```
151         case LumiraButtonVariant.ghost:
152           return _ButtonSpec(
153             backgroundColor: tokens.surfaceAlt,
154             foregroundColor: tokens.textSecondary,
155             radius: 8,
156             fontSize: 13, // 26rpx → 13dp
157             padding: const EdgeInsets.symmetric(
158               horizontal: 16, // 32rpx → 16dp
159               vertical: 10, // 20rpx → 10dp
160             ),
161           );
162       }
163     }
164   }
```

第 43 页


```
1  // ===== lib/shared/widgets/nav/lumira_nav.dart =====
2  import 'dart:ui';
3  import 'package:flutter/material.dart';
4  import 'package:flutter_riverpod/flutter_riverpod.dart';
5  import '../../../core/theme/theme_controller.dart';
6  import '../../../core/theme/theme_tokens.dart';
7  /// 如画应用统一顶部导航栏
8  /// 视觉规格来源：lumira-app/src/App.vue line 373-490
9  /// - 透明背景（不阻挡页面背景色）
10 /// - 滚动后毛玻璃态（.scrolled 类）
11 /// - 标题居中（position absolute + left 50% + transform translate -
50%）
12 /// - 可选左侧返回按钮（圆形 neumorphic 背景）
13 /// - 可选右侧操作按钮组
14 class LumiraNav extends ConsumerStatefulWidget implements
PreferredSizeWidget {
15   const LumiraNav({
16     super.key,
17     this.title,
18     this.leading,
19     this.actions,
20     this.centerTitle = true,
21     this.scrolled = false,
22     this.transparent = true,
23     this.showBackButton = true,
24   });
25   final String? title;
26   final Widget? leading;
27   final List<Widget>? actions;
28   final bool centerTitle;
29   final bool scrolled;
30   final bool transparent;
31   /// 是否在 leading 为 null 且 canPop 时自动显示返回按钮。
```

```
  32    /// Tab 页（home/templates/challenge/profile）应传 false，
  33    /// 避免 go_router canPop 误判导致 tab 页显示返回按钮（点击退出应用）。
  34    final bool showBackButton;
  35    @override
  36    Size get preferredSize => const Size.fromHeight(56);
  37    @override
  38    ConsumerState<LumiraNav> createState() => _LumiraNavState();
  39  }
  40  class _LumiraNavState extends ConsumerState<LumiraNav> {
  41    @override
  42    Widget build(BuildContext context) {
  43      final appTheme = ref.watch(appThemeProvider);
  44      final tokens = appTheme.tokens;
  45      final isGlass = appTheme.style == UIStyle.glass;
  46      // Forced fix: glass 风格默认就启用半透明 + blur（不只 scrolled
时），
  47      // 让导航栏在 glass 风格下也有毛玻璃效果。
  48      final Color backgroundColor;
  49      if (isGlass) {
  50        // glass 风格：默认半透明白 0.55，scrolled 时加深至 0.72
```

```
  51        backgroundColor = Colors.white.withOpacity(widget.scrolled ?
0.72 : 0.55);
  52      } else if (widget.transparent && !widget.scrolled) {
  53        backgroundColor = Colors.transparent;
  54      } else if (widget.scrolled) {
  55        // .lumira-nav.scrolled: rgba(canvas, 0.72) + blur 28px
saturate 1.8
  56        backgroundColor = tokens.canvas.withOpacity(0.72);
  57      } else {
  58        backgroundColor = tokens.canvas;
  59      }
  60      final border = widget.scrolled || isGlass
  61          ? Border(
  62              bottom: BorderSide(
  63                color: isGlass ? Colors.white.withOpacity(0.4) :
tokens.divider,
  64                width: 0.5,
  65              ),
  66            )
  67          : null;
  68      final shadow = widget.scrolled
  69          ? const [
  70              BoxShadow(
  71                color: Color(0x08000000),
  72                offset: Offset(0, 0.5),
  73                blurRadius: 6,
  74              ),
  75            ]
  76          : null;
  77      // Forced fix: 计算 leading widget
  78      // - 如果显式传了 leading，用它
  79      // - 否则如果 showBackButton=true 且 canPop=true，显示返回按钮
  80      // - 否则占位 SizedBox（保持标题居中）
  81      // Tab 页传 showBackButton=false，避免 canPop 误判导致 tab 页显示返
回按钮
  82      final Widget leadingWidget = widget.leading ??
  83          (widget.showBackButton && Navigator.of(context).canPop()
  84              ? _NavBackButton()
  85              : const SizedBox(width: 40));
  86      // Forced fix: glass 风格默认 blur 20，scrolled 时 blur 28
  87      final double blurSigma = isGlass
  88          ? (widget.scrolled ? 28.0 : 20.0)
  89          : (widget.scrolled ? 14.0 : 0.0);
  90      return ClipRect(
  91        child: BackdropFilter(
  92          filter: ImageFilter.blur(sigmaX: blurSigma, sigmaY:
blurSigma),
  93          child: AnimatedContainer(
  94            duration: const Duration(milliseconds: 400),
  95            curve: Curves.easeOutCubic,
  96            decoration: BoxDecoration(
  97              color: backgroundColor,
  98              border: border,
  99              boxShadow: shadow,
 100            ),
```

```
  101            child: SafeArea(
  102              bottom: false,
  103              child: SizedBox(
  104                height: 48, // min-height 96rpx → 48dp
  105                child: Stack(
  106                  alignment: Alignment.center,
  107                  children: [
  108                    // 左侧
  109                    Positioned(
  110                      left: 8,
  111                      child: leadingWidget,
  112                    ),
  113                    // 居中标题
  114                    if (widget.title != null)
  115                      Positioned(
  116                        left: 0,
  117                        right: 0,
  118                        child: Center(
  119                          child: Text(
  120                            widget.title!,
  121                            style: TextStyle(
  122                              fontSize: 19, // 38rpx → 19dp
  123                              fontWeight: FontWeight.w600,
  124                              color: tokens.textPrimary,
  125                              letterSpacing: 0.04 * 19,
  126                              height: 1.3,
  127                            ),
  128                            maxLines: 1,
  129                            overflow: TextOverflow.ellipsis,
  130                          ),
  131                        ),
  132                      ),
  133                    // 右侧
  134                    Positioned(
  135                      right: 8,
  136                      child: Row(
  137                        mainAxisSize: MainAxisSize.min,
  138                        children: widget.actions ?? [const
SizedBox(width: 40)],
  139                      ),
  140                    ),
  141                  ],
  142                ),
  143              ),
  144            ),
  145          ),
  146        ),
  147      );
  148    }
  149  }
  150  /// 默认返回按钮（圆形 neumorphic 背景）
```

```
  /// 来自 App.vue line 466-486: 64rpx 圆形 + surface 背景 + shadow-convex-subtle
  class _NavBackButton extends ConsumerWidget {
    @override
    Widget build(BuildContext context, WidgetRef ref) {
      final appTheme = ref.watch(appThemeProvider);
      final tokens = appTheme.tokens;
      return GestureDetector(
        onTap: () => Navigator.of(context).maybePop(),
        child: Container(
          width: 32, // 64rpx → 32dp
          height: 32,
          decoration: BoxDecoration(
            color: tokens.surface,
            shape: BoxShape.circle,
            boxShadow: tokens.shadowConvexSubtle,
          ),
          child: Icon(
            Icons.chevron_left,
            size: 18, // 36rpx → 18dp
            color: tokens.textPrimary,
          ),
        ),
      );
    }
  }
```

第 44 页


```
  // ===== lib/features/capture/data/capture_state.dart =====
  import 'package:flutter_riverpod/flutter_riverpod.dart';
  /// 闪光灯模式
  enum CaptureFlashMode {
    off,
    on,
    auto,
    torch,
  }
  /// 拍摄页状态 providers
  /// 修复 uni-app 中"退出拍摄页后 currentTemplateId 残留影响后续自由拍摄"的 bug。
  /// 所有状态在 LumiraRouteObserver.didPop 离开拍摄页时通过 _clearCaptureState 重置。
  class CaptureState {
    CaptureState._();
    /// 当前模板 ID（null = 自由拍摄模式）
    /// 来源：URL query param `?templateId=xxx`
    static final currentTemplateIdProvider = StateProvider<String?>((ref) => null);
    /// 闪光灯模式
```

```
  19    static final flashModeProvider =
StateProvider<CaptureFlashMode>((ref) => CaptureFlashMode.off);
  20    /// 全屏取景器开关
  21    static final isFullscreenProvider = StateProvider<bool>((ref) =>
false);
  22    /// 显示/隐藏模板叠图
  23    static final showTemplateProvider = StateProvider<bool>((ref) =>
true);
  24    /// 显示/隐藏姿势剪影
  25    static final showSilhouetteProvider = StateProvider<bool>((ref) =>
true);
  26    /// 最近拍摄照片路径（点击缩略图跳转预览页）
  27    static final lastPhotoPathProvider = StateProvider<String?>((ref)
=> null);
  28    /// 当前摄像头方向（front / back）
  29    static final cameraFacingProvider = StateProvider<String>((ref) =>
'back');
  30    /// 重置所有拍摄页状态（在 LumiraRouteObserver._clearCaptureState 中调
用）
  31    /// Forced fix: brief 原签名 `resetAll(Ref ref)` 与测试调用
  32    /// `CaptureState.resetAll(container.read)` 不兼容（`container.read`
是
  33    /// 函数引用而非 `Ref`）。改为接收 `ProviderContainer`，因为
  34    /// `ProviderContainer.read(provider.notifier).state = ...` 与 `Ref`
行为一致。
  35    /// 调用方 `LumiraRouteObserver._clearCaptureState` 传
`_ref.container`。
  36    static void resetAll(ProviderContainer container) {
  37      container.read(currentTemplateIdProvider.notifier).state = null;
  38      container.read(flashModeProvider.notifier).state =
CaptureFlashMode.off;
  39      container.read(isFullscreenProvider.notifier).state = false;
  40      container.read(showTemplateProvider.notifier).state = true;
  41      container.read(showSilhouetteProvider.notifier).state = true;
  42      container.read(lastPhotoPathProvider.notifier).state = null;
  43      container.read(cameraFacingProvider.notifier).state = 'back';
  44    }
   45  }
```

第 45 页


```
   1  // ===== lib/features/templates/pages/templates_editor_page.dart
=====
   2  import 'dart:async';
   3  import 'package:flutter/material.dart';
   4  import 'package:flutter_riverpod/flutter_riverpod.dart';
   5  import 'package:go_router/go_router.dart';
   6  import '../../../core/router/route_names.dart';
   7  import '../../../core/theme/theme_controller.dart';
   8  import '../../../core/theme/theme_tokens.dart';
```

```dart
import '../../../shared/widgets/buttons/lumira_buttons.dart';
import '../../../shared/widgets/common/fade_up.dart';
import '../../../shared/widgets/nav/lumira_nav.dart';
import '../data/templates_editor_mock_data.dart';
import '../widgets/composition_overlay.dart';
import '../widgets/pose_silhouette.dart';
import '../widgets/silhouette_editor.dart';
/// 选项对（label/value）
class EditorOption {
  const EditorOption(this.value, this.label);
  final String value;
  final String label;
}
// ===== 选项数组（editor.vue lines 819-888 verbatim） =====
const List<EditorOption> categoryOptions = [
  EditorOption('portrait', '人像'),
  EditorOption('landscape', '风光'),
  EditorOption('food', '美食'),
  EditorOption('night', '夜景'),
  EditorOption('street', '街拍'),
  EditorOption('macro', '微距'),
  EditorOption('still-life', '静物'),
];
const List<EditorOption> overlayTypeOptions = [
  EditorOption('rule_of_thirds', '三分法'),
  EditorOption('golden_ratio', '黄金比例'),
  EditorOption('diagonal', '对角线'),
  EditorOption('grid', '网格'),
  EditorOption('leading_lines', '引导线'),
  EditorOption('center', '中心构图'),
  EditorOption('none', '无'),
];
const List<EditorOption> silhouetteSourceOptions = [
  EditorOption('builtin', '内置库'),
  EditorOption('image', '导入图片'),
  EditorOption('svg', '绘制剪影'),
];
const List<EditorOption> whiteBalanceOptions = [
  EditorOption('daylight', '日光'),
  EditorOption('cloudy', '阴天'),
  EditorOption('shade', '阴影'),
  EditorOption('tungsten', '白炽灯'),
```

```
  51    EditorOption('fluorescent', '荧光灯'),
  52    EditorOption('custom', '自定义'),
  53  ];
  54  const List<EditorOption> flashModeOptions = [
  55    EditorOption('off', '关闭'),
  56    EditorOption('on', '开启'),
  57    EditorOption('auto', '自动'),
  58    EditorOption('torch', '常亮'),
  59  ];
  60  const List<EditorOption> focusModeOptions = [
  61    EditorOption('auto', '自动'),
  62    EditorOption('manual', '手动'),
  63    EditorOption('continuous', '连续'),
  64  ];
  65  const List<EditorOption> lensOptions = [
  66    EditorOption('wide', '广角'),
  67    EditorOption('main', '主摄'),
  68    EditorOption('telephoto', '长焦'),
  69    EditorOption('ultra_wide', '超广角'),
  70  ];
  71  const List<EditorOption> isoModeOptions = [
  72    EditorOption('auto', '自动'),
  73    EditorOption('manual', '手动'),
  74  ];
  75  const List<EditorOption> lutOptions = [
  76    EditorOption('none', '无'),
  77    EditorOption('cinematic', '电影感'),
  78    EditorOption('vintage', '复古'),
  79    EditorOption('bw', '黑白'),
  80    EditorOption('warm_film', '暖色胶片'),
  81    EditorOption('cool_film', '冷色胶片'),
  82    EditorOption('pastel', '柔色'),
  83    EditorOption('fuji', '富士'),
  84  ];
  85  /// 模板编辑器页
  86  /// 视觉规格来源：lumira-app/src/pages/templates/editor.vue (1534 行)
  87  /// 6 个 step：模板信息 / 构图叠图 / 姿势剪影 / 相机参数 / 场景指南 / 后
期参数
  88  /// + footer（草稿/预览/保存/导出）+ auto-save（1000ms debounce）+ pose
drag
  89  class TemplatesEditorPage extends ConsumerStatefulWidget {
  90    const TemplatesEditorPage({
  91      super.key,
  92      this.templateId,
  93      this.draftId,
  94    });
  95    /// 路由参数：templateId（编辑已有模板）或 draftId（恢复草稿），二选一
或都为 null（新建）
  96    final String? templateId;
  97    final String? draftId;
  98    @override
  99    ConsumerState<TemplatesEditorPage> createState() =>
 100        _TemplatesEditorPageState();
```

```
  101  }
  102  class _TemplatesEditorPageState extends
ConsumerState<TemplatesEditorPage> {
  103    late EditorForm _form;
  104    bool _isEditMode = false;
  105    String _currentDraftId = '';
  106    Timer? _autoSaveTimer;
  107    final ScrollController _scrollController = ScrollController();
  108    // 文本缓冲（数组字段与输入框双向同步）
  109    late TextEditingController _tagsController;
  110    late TextEditingController _propsController;
  111    late TextEditingController _tipsController;
  112    // 姿势预览拖动状态
  113    bool _isDraggingPose = false;
  114    @override
  115    void initState() {
  116      super.initState();
  117      _form = _loadInitialForm();
  118      _isEditMode = widget.templateId != null &&
  119
TemplatesEditorMockData.loadTemplateById(widget.templateId) != null;
  120      _tagsController =
  121          TextEditingController(text: _form.meta.tags.join(', '));
  122      _propsController =
  123          TextEditingController(text: _form.sceneGuide.props.join(',
'));
  124      _tipsController =
  125          TextEditingController(text:
_form.sceneGuide.tips.join('\n'));
  126    }
  127    EditorForm _loadInitialForm() {
  128      if (widget.templateId != null) {
  129        final tpl =
TemplatesEditorMockData.loadTemplateById(widget.templateId);
  130        if (tpl != null) return tpl;
  131      }
  132      if (widget.draftId != null) {
  133        final draft =
TemplatesEditorMockData.loadDraftById(widget.draftId);
  134        if (draft != null) {
  135          _currentDraftId = widget.draftId!;
  136          return draft;
  137        }
  138      }
  139      return createBlankEditorForm();
  140    }
  141    @override
  142    void dispose() {
  143      _autoSaveTimer?.cancel();
  144      _scrollController.dispose();
  145      _tagsController.dispose();
  146      _propsController.dispose();
  147      _tipsController.dispose();
  148      super.dispose();
  149    }
  150    // ===== 表单变更处理 =====
```

```
  151    void _onChange(void Function() mutator) {
  152      setState(mutator);
  153      _scheduleAutoSave();
  154    }
  155    void _onTagsChanged(String text) {
  156      _form.meta.tags =
  157          text.split(',').map((t) => t.trim()).where((t) =>
t.isNotEmpty).toList();
  158      _scheduleAutoSave();
  159    }
  160    void _onPropsChanged(String text) {
  161      _form.sceneGuide.props =
  162          text.split(',').map((t) => t.trim()).where((t) =>
t.isNotEmpty).toList();
  163      _scheduleAutoSave();
  164    }
  165    void _onTipsChanged(String text) {
  166      _form.sceneGuide.tips = text
  167          .split('\n')
  168          .map((t) => t.trim())
  169          .where((t) => t.isNotEmpty)
  170          .toList();
  171      _scheduleAutoSave();
  172    }
```

第 46 页


```
  173    // ===== 自动保存 =====
  174    void _scheduleAutoSave() {
  175      _autoSaveTimer?.cancel();
  176      _autoSaveTimer = Timer(const Duration(milliseconds: 1000), () {
  177        _saveDraft();
  178      });
  179    }
  180    void _saveDraft() {
  181      if (_currentDraftId.isEmpty) {
  182        _currentDraftId =
'draft_${DateTime.now().millisecondsSinceEpoch}';
  183      }
  184      TemplatesEditorMockData.saveDraft(_currentDraftId, _form);
  185    }
  186    void _preview() {
  187      _saveDraft();
  188      GoRouter.of(context).push(
  189        RouteNames.build(
  190          RouteNames.capturePreviewTemplate,
  191          {
  192            RouteNames.paramTemplateId: _form.meta.id,
  194            'draftId': _currentDraftId,
  195          },
  196        ),
```

```
197       );
198     }
199     void _save() {
200       _saveDraft();
201       if (_isEditMode) {
202         // 编辑模式：直接更新已有模板
203         TemplatesEditorMockData.updateTemplate(_form.meta.id, _form);
204       } else {
205         // 新建模式：创建新模板
206         TemplatesEditorMockData.createTemplate(_form);
207       }
208       GoRouter.of(context).pop();
209     }
210     void _export() {
211       // 导出 .pptpl 文件（包含 base64 资源自包含）
212       final json = _form.toJson();
213       // 在实际实现中会写入文件系统；此处简化为 SnackBar 提示
214       ScaffoldMessenger.of(context).showSnackBar(
215         SnackBar(content: Text('导出 ${_form.meta.name}.pptpl
（${json.length} 字节）')),
216       );
217     }
218     @override
219     Widget build(BuildContext context) {
220       final appTheme = ref.watch(appThemeProvider);
221       final tokens = appTheme.tokens;
222       final stepLabels = ['模板信息', '构图叠图', '姿势剪影', '相机参数',
'场景指南', '后期参数'];
223       return Scaffold(
```

```
224         backgroundColor: tokens.canvas,
225         appBar: LumiraNav(
226           title: _isEditMode ? '编辑模板' : '新建模板',
227           transparent: true,
228           scrolled: false,
229         ),
230         body: Stack(
231           children: [
232             // 主内容：6 个 step 的表单
233             SafeArea(
234               child: ListView(
235                 controller: _scrollController,
236                 padding: const EdgeInsets.fromLTRB(20, 8, 20, 120),
237                 children: [
238                   // 1. 模板信息
239                   FadeUp(
240                     child: _StepSection(
241                       index: 0,
242                       title: stepLabels[0],
243                       child: _buildTemplateInfoSection(tokens),
244                     ),
245                   ),
246                   const SizedBox(height: 16),
247                   // 2. 构图叠图
248                   FadeUp(
249                     delay: const Duration(milliseconds: 80),
250                     child: _StepSection(
251                       index: 1,
252                       title: stepLabels[1],
253                       child: _buildCompositionSection(tokens),
254                     ),
255                   ),
256                   const SizedBox(height: 16),
257                 ],
258               ),
259             ),
260           ],
261         ),
262       );
263     }
264   }
```

第 47 页


```
266     // ===== 姿势预览拖动 =====
268     void _onPoseDragStart(DragStartDetails details) {
269       setState(() => _isDraggingPose = true);
270     }
272     void _onPoseDragUpdate(DragUpdateDetails details, BoxConstraints
constraints) {
273       if (!_isDraggingPose) return;
```

```
274       setState(() {
275         final dx = details.delta.dx / constraints.maxWidth;
276         final dy = details.delta.dy / constraints.maxHeight;
277         _form.pose.position.x =
278             (_form.pose.position.x + dx).clamp(0.0, 1.0);
279         _form.pose.position.y =
280             (_form.pose.position.y + dy).clamp(0.0, 1.0);
281       });
282     }
284     void _onPoseDragEnd(DragEndDetails details) {
285       setState(() => _isDraggingPose = false);
286       _scheduleAutoSave();
287     }
289     // ===== 自动保存草稿（debounce 1000ms） =====
291     void _scheduleAutoSave() {
292       _autoSaveTimer?.cancel();
293       _autoSaveTimer = Timer(const Duration(milliseconds: 1000), () {
294         // mock 自动保存（Task 2.9+ 接入真实 DAO）
295         if (_currentDraftId.isEmpty) {
296           _currentDraftId =
297               'draft-editor-${DateTime.now().millisecondsSinceEpoch}';
298         }
299       });
300     }
302     // ===== Footer 操作 =====
304     void _onSave() {
305       if (_form.meta.name.trim().isEmpty) {
306         ScaffoldMessenger.of(context).showSnackBar(
307           const SnackBar(content: Text('请输入模板名称')),
308         );
309         return;
310       }
311       // 简化：mock 保存（不接入真实 DAO，Task 2.9+ 替换）
312       _currentDraftId = '';
313       ScaffoldMessenger.of(context).showSnackBar(
314         const SnackBar(content: Text('保存成功')),
315       );
316       // 800ms 后返回上一页（与 uni-app setTimeout 一致）
317       Future.delayed(const Duration(milliseconds: 800), () {
318         if (!mounted) return;
319         if (Navigator.of(context).canPop()) {
320           Navigator.of(context).pop();
321         } else {
322           GoRouter.of(context).go(RouteNames.templates);
```

```
323         }
324       });
325     }
```

第 48 页


```
327     void _onSaveDraft() {
328       // 简化：mock 草稿保存
329       if (_currentDraftId.isEmpty) {
330         _currentDraftId =
331             'draft-editor-${DateTime.now().millisecondsSinceEpoch}';
332       }
333       ScaffoldMessenger.of(context).showSnackBar(
334         const SnackBar(content: Text('草稿已保存')),
335       );
336     }
338     void _onExport() {
339       // 简化：mock 导出（Task 2.9+ 接入真实导出 .pptpl）
340       ScaffoldMessenger.of(context).showSnackBar(
341         const SnackBar(content: Text('已导出（mock）')),
342       );
343     }
345     void _onPreview() {
346       // 保存草稿后跳转预览页（capture/preview-template?draftId=xxx）
347       if (_currentDraftId.isEmpty) {
348         _currentDraftId =
349             'draft-editor-${DateTime.now().millisecondsSinceEpoch}';
350       }
351       GoRouter.of(context)
352           .push('/capture/preview-template?draftId=$_currentDraftId');
353     }
355     void _back() {
356       if (Navigator.of(context).canPop()) {
357         Navigator.of(context).pop();
358       } else {
359         GoRouter.of(context).go(RouteNames.templates);
360       }
361     }
363     void _goDrafts() {
364       GoRouter.of(context).push(RouteNames.templatesDrafts);
365     }
367     String get _pageTitle => _isEditMode ? '编辑模板' : '新建模板';
369     @override
370     Widget build(BuildContext context) {
371       final tokens = ref.watch(themeTokensProvider);
373       return Scaffold(
374         backgroundColor: tokens.canvas,
375         extendBodyBehindAppBar: true,
376         body: Stack(
377           children: [
378             _BackgroundDecoration(tokens: tokens),
```

```
379            SafeArea(
380              top: false,
381              bottom: false,
382              child: Column(
383                children: [
```

```
  384                   LumiraNav(
  385                     title: _pageTitle,
```

第 49 页


```
  386                     transparent: true,
  387                     leading: _BackButton(tokens: tokens, onTap: _back),
  388                     actions: [
  389                       _DraftsNavButton(tokens: tokens, onTap:
_goDrafts),
  390                     ],
  391                   ),
  392                 Expanded(
  393                   child: Stack(
  394                     children: [
  395                       SingleChildScrollView(
  396                         padding: const EdgeInsets.fromLTRB(24, 12, 24,
120),
  397                         child: Column(
  398                           crossAxisAlignment:
CrossAxisAlignment.stretch,
  399                           children: [
  400                             _Step1TemplateInfo(
  401                               tokens: tokens,
  402                               form: _form,
  403                               tagsController: _tagsController,
  404                               onTagsChanged: _onTagsChanged,
  405                               onChange: _onChange,
  406                             ),
  407                             const SizedBox(height: 12),
  408                             _Step2Composition(
  409                               tokens: tokens,
  410                               form: _form,
  411                               onChange: _onChange,
  412                             ),
  413                             const SizedBox(height: 12),
  414                             _Step3Pose(
  415                               tokens: tokens,
  416                               form: _form,
  417                               isDragging: _isDraggingPose,
  418                               onSourceChange:
_onSilhouetteSourceChange,
  419                               onSelectBuiltin:
_selectBuiltinSilhouette,
  420                               onImportImage: _importSilhouetteImage,
  421                               onOpenEditor: _openSilhouetteEditor,
  422                               onChange: _onChange,
  423                               onPoseDragStart: _onPoseDragStart,
  424                               onPoseDragUpdate: _onPoseDragUpdate,
  425                               onPoseDragEnd: _onPoseDragEnd,
  426                             ),
```

```
427                         const SizedBox(height: 12),
428                         _Step4Camera(
429                           tokens: tokens,
430                           form: _form,
431                           onChange: _onChange,
432                           onIsoInput: _onIsoInput,
433                           onWbKInput: _onWbKInput,
434                         ),
435                         const SizedBox(height: 12),
```

```
436                             _Step5SceneGuide(
437                               tokens: tokens,
438                               form: _form,
439                               propsController: _propsController,
440                               tipsController: _tipsController,
```

第 50 页


```
441                               onPropsChanged: _onPropsChanged,
442                               onTipsChanged: _onTipsChanged,
443                               onChange: _onChange,
444                             ),
445                             const SizedBox(height: 12),
446                             _Step6PostProcess(
447                               tokens: tokens,
448                               form: _form,
449                               onChange: _onChange,
450                             ),
451                           ],
452                         ),
453                       ),
454                       _EditorFooter(
455                         tokens: tokens,
456                         isExportVisible: _isEditMode &&
_form.meta.id.isNotEmpty,
457                         onSaveDraft: _onSaveDraft,
458                         onPreview: _onPreview,
459                         onSave: _onSave,
460                         onExport: _onExport,
461                       ),
462                     ],
463                   ),
464                 ),
465               ],
466             ),
467           ),
468         ],
469       ),
470     );
471     }
472   }
474   /// 背景径向渐变装饰（glass 风格 backdrop-filter 可见性）
475   class _BackgroundDecoration extends StatelessWidget {
476     const _BackgroundDecoration({required this.tokens});
477     final ThemeTokens tokens;
479     @override
480     Widget build(BuildContext context) {
481       return Positioned.fill(
482         child: IgnorePointer(
483           child: Container(
484             decoration: BoxDecoration(
```

```
485              gradient: RadialGradient(
486                center: const Alignment(-0.6, -0.8),
487                radius: 1.4,
488                colors: [
489                  tokens.brandSubtle.withOpacity(0.45),
490                  tokens.canvas.withOpacity(0),
491                ],
492              ),
```

```
  493           ),
  494         ),
  495       ),
  496     );
  497   }
  498 }
  500 class _BackButton extends StatelessWidget {
```

第 51 页

如画 Lumira 摄影辅助应用 V1.0.0 第 51 页

```
  501   const _BackButton({required this.tokens, required this.onTap});
  502   final ThemeTokens tokens;
  503   final VoidCallback onTap;
  505   @override
  506   Widget build(BuildContext context) {
  507     return GestureDetector(
  508       onTap: onTap,
  509       behavior: HitTestBehavior.opaque,
  510       child: Padding(
  511         padding: const EdgeInsets.all(8),
  512         child: Icon(
  513           Icons.arrow_back_ios_new,
  514           size: 20,
  515           color: tokens.textPrimary,
  516         ),
  517       ),
  518     );
  519   }
  520 }
  522 class _DraftsNavButton extends StatelessWidget {
  523   const _DraftsNavButton({required this.tokens, required
this.onTap});
  524   final ThemeTokens tokens;
  525   final VoidCallback onTap;
  527   @override
  528   Widget build(BuildContext context) {
  529     return GestureDetector(
  530       onTap: onTap,
  531       behavior: HitTestBehavior.opaque,
  532       child: Padding(
  533         padding: const EdgeInsets.all(8),
  534         child: Icon(
  535           Icons.edit_note,
  536           size: 22,
  537           color: tokens.textSecondary,
  538         ),
  539       ),
  540     );
  541   }
  542 }
  544 // ===== 通用 Step 卡片 + 字段组件 =====
```

```
546  class _StepCard extends StatelessWidget {
547    const _StepCard({
548      required this.tokens,
549      required this.stepNumber,
550      required this.title,
551      required this.child,
552      this.delay = Duration.zero,
553    });
555    final ThemeTokens tokens;
556    final int stepNumber;
```

```
  final String title;
  final Widget child;
  final Duration delay;
```

第 52 页


```
  @override
  Widget build(BuildContext context) {
    return FadeUp(
      delay: delay,
      child: Container(
        padding: const EdgeInsets.all(20),
        decoration: BoxDecoration(
          color: tokens.canvas,
          borderRadius: BorderRadius.circular(14),
          border: Border.all(color: tokens.divider, width: 0.5),
          boxShadow: tokens.shadowConvex,
        ),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
          Row(
            children: [
              Container(
                width: 24,
                height: 24,
                decoration: BoxDecoration(
                  color: tokens.brand,
                  shape: BoxShape.circle,
                  boxShadow: tokens.shadowConvexBrand,
                ),
                alignment: Alignment.center,
                child: Text(
                  '$stepNumber',
                  style: const TextStyle(
                    fontSize: 12,
                    fontWeight: FontWeight.w600,
                    color: Colors.white,
                    height: 1,
                  ),
                ),
              ),
              const SizedBox(width: 10),
              Text(
                title,
                style: TextStyle(
                  fontFamily: 'Noto Serif SC',
                  fontSize: 16,
                  fontWeight: FontWeight.w600,
                  color: tokens.textPrimary,
                ),
```

```
606                ),
607              ],
608            ),
609            const SizedBox(height: 18),
610            child,
```

```
611           ],
612         ),
613       ),
614     );
615     }
616   }
618   class _FieldLabel extends StatelessWidget {
619     const _FieldLabel({required this.tokens, required this.text});
620     final ThemeTokens tokens;
```

第 53 页


```
621     final String text;
623     @override
624     Widget build(BuildContext context) {
625       return Padding(
626         padding: const EdgeInsets.only(bottom: 6),
627         child: Text(
628           text,
629           style: TextStyle(
630             fontSize: 13,
631             fontWeight: FontWeight.w500,
632             color: tokens.textSecondary,
633           ),
634         ),
635       );
636     }
637   }
639   class _FieldInput extends StatelessWidget {
640     const _FieldInput({
641       required this.tokens,
642       this.controller,
643       this.initialValue,
644       this.placeholder,
645       this.multiline = false,
646       this.keyboardType,
647       this.onChanged,
648     });
650     final ThemeTokens tokens;
651     final TextEditingController? controller;
652     final String? initialValue;
653     final String? placeholder;
654     final bool multiline;
655     final TextInputType? keyboardType;
656     final ValueChanged<String>? onChanged;
658     @override
659     Widget build(BuildContext context) {
660       final InputDecoration decoration = InputDecoration(
661         hintText: placeholder,
662         filled: true,
663         fillColor: tokens.canvasDeep,
```

```
664         contentPadding: EdgeInsets.symmetric(
665           horizontal: 12,
666           vertical: multiline ? 12 : 10,
667         ),
668         border: OutlineInputBorder(
669           borderRadius: BorderRadius.circular(8),
670           borderSide: BorderSide.none,
671         ),
672         enabledBorder: OutlineInputBorder(
673           borderRadius: BorderRadius.circular(8),
674           borderSide: BorderSide.none,
```

```
675         ),
676         focusedBorder: OutlineInputBorder(
677           borderRadius: BorderRadius.circular(8),
678           borderSide: BorderSide(color: tokens.brand, width: 1),
679         ),
680       );
```

第 54 页


```
682     final style = TextStyle(
683       fontSize: 14,
684       color: tokens.textPrimary,
685     );
687     if (controller != null) {
688       return TextField(
689         controller: controller,
690         decoration: decoration,
691         style: style,
692         maxLines: multiline ? null : 1,
693         minLines: multiline ? 3 : 1,
694         keyboardType: keyboardType,
695         onChanged: onChanged,
696       );
697     }
698     return TextFormField(
699       initialValue: initialValue,
700       decoration: decoration,
701       style: style,
702       maxLines: multiline ? null : 1,
703       minLines: multiline ? 3 : 1,
704       keyboardType: keyboardType,
705       onChanged: onChanged,
706     );
707     }
708   }
710   class _FieldDropdown extends StatelessWidget {
711     const _FieldDropdown({
712       required this.tokens,
713       required this.value,
714       required this.options,
715       required this.onChanged,
716     });
718     final ThemeTokens tokens;
719     final String value;
720     final List<EditorOption> options;
721     final ValueChanged<String> onChanged;
723     @override
724     Widget build(BuildContext context) {
725       return Container(
726         padding: const EdgeInsets.symmetric(horizontal: 12),
727         decoration: BoxDecoration(
```

```
728           color: tokens.canvasDeep,
729           borderRadius: BorderRadius.circular(8),
730         ),
731         child: DropdownButtonHideUnderline(
732           child: DropdownButton<String>(
733             value: value,
734             isExpanded: true,
735             icon: Icon(
```

```
  736             Icons.keyboard_arrow_down_rounded,
  737             size: 18,
  738             color: tokens.textTertiary,
  739           ),
  740           style: TextStyle(
```

第 55 页


```
  741             fontSize: 14,
  742             color: tokens.textPrimary,
  743           ),
  744           dropdownColor: tokens.surface,
  745           items: options
  746               .map((o) => DropdownMenuItem<String>(
  747                     value: o.value,
  748                     child: Text(o.label),
  749                   ))
  750               .toList(),
  751           onChanged: (v) {
  752             if (v != null) onChanged(v);
  753           },
  754         ),
  755       ),
  756     );
  757     }
  758   }
  760   class _PillGroup extends StatelessWidget {
  761     const _PillGroup({
  762       required this.tokens,
  763       required this.options,
  764       required this.value,
  765       required this.onChanged,
  766     });
  768     final ThemeTokens tokens;
  769     final List<EditorOption> options;
  770     final String value;
  771     final ValueChanged<String> onChanged;
  773     @override
  774     Widget build(BuildContext context) {
  775       return Wrap(
  776         spacing: 8,
  777         runSpacing: 4,
  778         children: options.map((o) {
  779           final active = o.value == value;
  780           return GestureDetector(
  781             onTap: () => onChanged(o.value),
  782             behavior: HitTestBehavior.opaque,
  783             child: Container(
  784             padding: const EdgeInsets.symmetric(horizontal: 14,
vertical: 6),
  785             decoration: BoxDecoration(
```

```
786               color: active ? tokens.brand : tokens.canvasDeep,
787               borderRadius: BorderRadius.circular(9999),
788               border: Border.all(
789                 color: active ? tokens.brand : Colors.transparent,
790                 width: 1,
791               ),
792             ),
793             child: Text(
```

```
  794              o.label,
  795              style: TextStyle(
  796                fontSize: 12,
  797                fontWeight: FontWeight.w500,
  798                color: active ? tokens.textInverse :
tokens.textSecondary,
  799                height: 1,
  800              ),
```

第 56 页


```
  801            ),
  802          ),
  803        );
  804      }).toList(),
  805    );
  806    }
  807  }
  809  class _SliderRow extends StatelessWidget {
  810    const _SliderRow({
  811      required this.tokens,
  812      required this.label,
  813      required this.value,
  814      required this.min,
  815      required this.max,
  816      required this.divisions,
  817      required this.onChanged,
  818      required this.valueText,
  819    });
  821    final ThemeTokens tokens;
  822    final String label;
  823    final double value;
  824    final double min;
  825    final double max;
  826    final int? divisions;
  827    final ValueChanged<double> onChanged;
  828    final String valueText;
  830    @override
  831    Widget build(BuildContext context) {
  832      return Padding(
  833        padding: const EdgeInsets.only(bottom: 14),
  834        child: Row(
  835          children: [
  836            SizedBox(
  837              width: 56,
  838              child: Text(
  839                label,
  840                style: TextStyle(
  841                  fontSize: 13,
  842                  fontWeight: FontWeight.w500,
  843                  color: tokens.textPrimary,
```

```
              ),
            ),
          ),
          Expanded(
            child: Slider(
              value: value,
              min: min,
              max: max,
              divisions: divisions,
              activeColor: tokens.brand,
```

```
854                 onChanged: onChanged,
855               ),
856             ),
857             SizedBox(
858               width: 44,
859               child: Text(
```

第 57 页


```
860                 valueText,
861                 textAlign: TextAlign.right,
862                 style: TextStyle(
863                   fontSize: 11,
864                   fontFamily: 'SF Mono',
865                   color: tokens.textSecondary,
866                 ),
867               ),
868             ),
869           ],
870         ),
871       ),
872     );
873     }
874   }
876   // ===== Step 1: 模板信息 =====
878   class _Step1TemplateInfo extends StatelessWidget {
879     const _Step1TemplateInfo({
880       required this.tokens,
881       required this.form,
882       required this.tagsController,
883       required this.onTagsChanged,
884       required this.onChange,
885     });
887     final ThemeTokens tokens;
888     final EditorForm form;
889     final TextEditingController tagsController;
890     final ValueChanged<String> onTagsChanged;
891     final void Function(void Function() mutator) onChange;
893     @override
894     Widget build(BuildContext context) {
895       return _StepCard(
896         tokens: tokens,
897         stepNumber: 1,
898         title: '模板信息',
899         child: Column(
900           crossAxisAlignment: CrossAxisAlignment.start,
901           children: [
902           _FieldLabel(tokens: tokens, text: '名称'),
903           _FieldInput(
904             tokens: tokens,
905             initialValue: form.meta.name,
```

```
906             placeholder: '输入模板名称',
907             onChanged: (v) => onChange(() => form.meta.name = v),
908           ),
909           const SizedBox(height: 14),
910           _FieldLabel(tokens: tokens, text: '分类'),
911           _FieldDropdown(
912             tokens: tokens,
913             value: form.meta.category,
```

```
  914             options: categoryOptions,
  915             onChanged: (v) => onChange(() => form.meta.category = v),
  916           ),
  917           const SizedBox(height: 14),
  918           _FieldLabel(tokens: tokens, text: '标签'),
  919           _FieldInput(
  920             tokens: tokens,
```

第 58 页

如画 Lumira 摄影辅助应用 V1.0.0 第 58 页

```
  921             controller: tagsController,
  922             placeholder: '标签 1, 标签 2',
  923             onChanged: onTagsChanged,
  924           ),
  925           const SizedBox(height: 14),
  926           _FieldLabel(tokens: tokens, text: '简介'),
  927           _FieldInput(
  928             tokens: tokens,
  929             initialValue: form.meta.description,
  930             placeholder: '模板简介',
  931             multiline: true,
  932             onChanged: (v) => onChange(() => form.meta.description =
v),
  933           ),
  934           const SizedBox(height: 14),
  935           _FieldLabel(tokens: tokens, text: '参数参考来源'),
  936           _FieldInput(
  937             tokens: tokens,
  938             initialValue: form.meta.referenceSource,
  939             placeholder: '如：样片 EXIF',
  940             onChanged: (v) => onChange(() => form.meta.referenceSource
= v),
  941           ),
  942         ],
  943       ),
  944     );
  945     }
  946   }
  948   // ===== Step 2: 构图叠图 =====
  950   class _Step2Composition extends StatelessWidget {
  951     const _Step2Composition({
  952       required this.tokens,
  953       required this.form,
  954       required this.onChange,
  955     });
  957     final ThemeTokens tokens;
  958     final EditorForm form;
  959     final void Function(void Function() mutator) onChange;
  961     @override
  962     Widget build(BuildContext context) {
  963       return _StepCard(
```

```
964         tokens: tokens,
965         stepNumber: 2,
966         title: '构图叠图',
967         delay: const Duration(milliseconds: 80),
968         child: Column(
969           crossAxisAlignment: CrossAxisAlignment.start,
970           children: [
971           _FieldLabel(tokens: tokens, text: '构图类型'),
972           _FieldDropdown(
973             tokens: tokens,
974             value: form.composition.overlayType,
```

```
  975             options: overlayTypeOptions,
  976             onChanged: (v) =>
  977                 onChange(() => form.composition.overlayType = v),
  978           ),
  979           const SizedBox(height: 14),
  980           _FieldLabel(tokens: tokens, text: '宽高比'),
```

第 59 页

如画 Lumira 摄影辅助应用 V1.0.0 第 59 页

```
  981           _FieldInput(
  982             tokens: tokens,
  983             initialValue: form.composition.aspectRatio,
  984             placeholder: '如 3:4',
  985             onChanged: (v) =>
  986                 onChange(() => form.composition.aspectRatio = v),
  987           ),
  988           const SizedBox(height: 14),
  989           _SliderRow(
  990             tokens: tokens,
  991             label: '透明度',
  992             value: form.composition.opacity,
  993             min: 0,
  994             max: 1,
  995             divisions: 10,
  996             onChanged: (v) =>
  997                 onChange(() => form.composition.opacity = v),
  998             valueText: form.composition.opacity.toStringAsFixed(1),
  999           ),
 1000           _FieldLabel(tokens: tokens, text: '构图说明'),
 1001           _FieldInput(
 1002             tokens: tokens,
 1003             initialValue: form.composition.description,
 1004             placeholder: '构图说明',
 1005             multiline: true,
 1006             onChanged: (v) =>
 1007                 onChange(() => form.composition.description = v),
 1008           ),
 1009           const SizedBox(height: 14),
 1010           _PreviewBox(
 1011             tokens: tokens,
 1012             aspectRatio:
parseAspectRatio(form.composition.aspectRatio),
 1013             child: CompositionOverlay(
 1014               overlayType: form.composition.overlayType,
 1015               opacity: form.composition.opacity,
 1016             ),
 1017           ),
 1018         ],
 1019       ),
 1020     );
 1021     }
```

```
1022  }
1024  /// 预览框（preview-box: 渐变背景 + 叠加层）
1025  class _PreviewBox extends StatelessWidget {
1026    const _PreviewBox({
1027      required this.tokens,
1028      required this.aspectRatio,
1029      required this.child,
1030    });
1032    final ThemeTokens tokens;
```

```
 1033    final double aspectRatio;
 1034    final Widget child;
 1036    @override
 1037    Widget build(BuildContext context) {
 1038    return AspectRatio(
  1039      aspectRatio: aspectRatio,
  1040      child: ClipRRect(
```

第 60 页


```
 1041        borderRadius: BorderRadius.circular(10),
 1042        child: Stack(
 1043          fit: StackFit.expand,
 1044          children: [
 1045            // 硬编码颜色，与 uni-app 一致 (preview-bg: linear-
gradient(135deg, #3A3631 0%, #2A2622 100%))
 1046            Container(
 1047              decoration: const BoxDecoration(
 1048                gradient: LinearGradient(
 1049                  begin: Alignment.topLeft,
 1050                  end: Alignment.bottomRight,
 1051                  colors: [Color(0xFF3A3631), Color(0xFF2A2622)],
 1052                ),
 1053              ),
 1054            ),
 1055            child,
 1056          ],
 1057        ),
 1058      ),
 1059    );
 1060    }
 1061  }
 1063  // ===== Step 3: 姿势剪影 =====
 1065  class _Step3Pose extends StatelessWidget {
 1066    const _Step3Pose({
 1067      required this.tokens,
 1068      required this.form,
 1069      required this.isDragging,
 1070      required this.onSourceChange,
 1071      required this.onSelectBuiltin,
 1072      required this.onImportImage,
 1073      required this.onOpenEditor,
 1074      required this.onChange,
 1075      required this.onPoseDragStart,
 1076      required this.onPoseDragUpdate,
 1077      required this.onPoseDragEnd,
 1078    });
 1080    final ThemeTokens tokens;
 1081    final EditorForm form;
 1082    final bool isDragging;
 1083    final ValueChanged<String> onSourceChange;
```

```
 1084    final ValueChanged<String> onSelectBuiltin;
 1085    final VoidCallback onImportImage;
 1086    final VoidCallback onOpenEditor;
 1087    final void Function(void Function() mutator) onChange;
 1088    final void Function(DragStartDetails) onPoseDragStart;
 1089    final void Function(DragUpdateDetails, BoxConstraints)
onPoseDragUpdate;
 1090    final void Function(DragEndDetails) onPoseDragEnd;
 1092    @override
 1093    Widget build(BuildContext context) {
 1094      return _StepCard(
```

```
1095        tokens: tokens,
1096        stepNumber: 3,
1097        title: '姿势剪影',
1098        delay: const Duration(milliseconds: 160),
1099        child: Column(
1100          crossAxisAlignment: CrossAxisAlignment.start,
```